

从零实现 macOS 中文输入法

基于 MyIME 的 InputMethodKit 教程

初级篇：从键盘事件到中文上屏

高级篇：切分、词格、语言模型与工程发布



图 1: 课程路线：从 Echo IME 到可发布输入法

MyIME 教学项目

适用于 macOS 13 及以上版本 · Swift 5.9+ · Xcode 15+

书名：从零实现 macOS 中文输入法：基于 MyIME 的 InputMethodKit 教程

教材定位：面向高年级本科生、研究生、Swift/macOS 开发者，以及希望理解中文输入法内部机制的工程人员。

配套项目：<https://github.com/bennix/MyIME>

版本说明：本教材以 MyIME 1.0.10（构建 76）的源码结构为基线。输入法属于与操作系统深度耦合的软件，后续 macOS 与 Xcode 版本可能改变注册、签名或事件投递行为。课堂实验应以实际运行结果为准。

工程警告：许可与引用

教材引用的 MyIME 源码遵循仓库许可证；词库来源具有各自许可证。教学者分发源码、词库或改编版本前，应阅读仓库中的 `GPL-3.0.txt`、`ThirdPartyNotices.txt` 与 `tools/LICENSES.md`。本教材不把公开词表自动等同于“无条件可再分发”。

前言：输入法是一门小型系统课程

输入法看似只是“把拼音变成汉字”，实际上同时涉及操作系统服务、事件状态机、自然语言处理、本地数据库、图搜索、用户界面、性能工程、安全、签名与发布。它很小，小到一学期可以完成；又足够真实，真实到每一次按键都可能暴露架构问题。

本书不从庞大的商业输入法倒推黑盒行为，而从一个可以阅读、编译、测试和发布的开源项目 MyIME 出发。学生将沿着一条可验证的路径前进：先让一个字母被输入法拦截，再建立组合区与候选列表，继而实现拼音切分、词库查询、整句组合和本地学习，最后处理跨应用兼容、安装注册、Developer ID 签名和 Apple 公证。

全书分为四个部分。初级篇回答“输入法怎样活起来”；高级篇回答“候选怎样产生并排序”；工程篇回答“为什么开发机能用、换一台机器却失效”；课程实验与综合项目则把知识转化为可评分、可演示、可复现的成果。

核心概念：学习方法

不要把本书当作 API 字典。每章都围绕一个可观察现象：按下按键、出现下划线、弹出候选、选择词语、退出进程、换机安装。先预测现象，再运行实验，最后阅读源码解释结果。输入法调试最忌讳“代码看起来应该可以”。

读者对象与先修知识

初级篇只要求读者掌握 Swift 的变量、函数、结构体、类、协议和可选值，能够在 Xcode 中打开项目并阅读编译错误。了解 AppKit 有帮助，但不是前提。高级篇默认读者学过数据结构、离散数学和概率统计的基础内容，至少知道图、动态规划、对数和数据库索引的概念。

完成全书后，读者应能做到：

1. 解释 macOS 输入源、输入法进程、**IMKServer** 与客户端之间的关系；
2. 实现包含 **raw buffer**、**preedit**、候选列表与 **commit** 的输入状态机；
3. 将拼音串建模为切分图，将短句生成建模为词格搜索；
4. 设计可解释的候选评分函数，并区分词频、上下文与惩罚项；
5. 使用 SQLite 保存系统词库和用户学习数据；
6. 在 Notes、浏览器和 Electron 应用中定位输入法兼容问题；
7. 构建 Universal 2 应用，完成签名、公证、装订与 DMG 发布；
8. 为准确率、延迟、确定性和跨应用行为建立测试证据。

课程组织建议

本书适合 16 周课程，每周 3 学时讲授加 2 学时实验。也可以压缩为 8 周项目课。推荐安排如下。

周次	主题	可验收成果
1-2	输入法系统模型、最小 IMK 服务	能注册输入源，拦截并回送字母
3-4	组合状态与候选窗口	能输入 raw buffer、退格、取消和上屏
5-6	拼音切分与词库查询	小词库完成全拼、简拼和硬边界
7-8	候选排序与整句生成	完成可解释评分和词格搜索
9-10	SQLite 与用户学习	重启后保留词频和上下文偏好
11-12	UI、性能和可访问性	候选窗跟随光标，P95 延迟达标
13-14	安装、兼容与安全	在第二台 Mac 的 Notes 和浏览器验收
15-16	发布与答辩	公证 DMG、测试报告和设计说明

核心概念：两条代码路线

课堂前半程使用“教学版”思路：内存词库、最少状态、明确日志。课堂后半程阅读仓库中的完整实现。不要在第一周就让学生修改 699 行的输入控制器；先让他们能独立画出状态机，再去理解真实工程为何复杂。

符号、术语与源码约定

本书使用以下约定：

- `raw` 表示用户尚未转换的原始字母串，如 `jintian`；
- `preedit` 表示显示在插入点附近、尚未最终提交的组合文本；
- `candidate` 表示候选；`commitText` 表示最终上屏文本；
- 拼音路径写作 $P = (s_1, s_2, \dots, s_m)$ ；候选词写作 w ；
- $S(w \mid P, h)$ 表示在拼音路径 P 与历史上下文 h 下候选的评分；
- 源码路径均相对于仓库根目录；代码行号以 1.0.10 为基线；
- `Space`、`Return`、`Esc` 表示键盘按键。



图 2: 从物理按键到最终上屏的端到端数据流

开发环境与第一次构建

建议使用一台非关键工作机或独立 macOS 用户账户完成实验。输入法安装在用户目录并由系统加载，开发过程中的重复注册、崩溃和旧副本残留会影响整个登录会话。开始前请确认 Xcode 命令行工具和 Swift 可用：

```
xcodebuild -version
swift --version
swift test --package-path MyIME/IMEKit
```

第一条测试命令只构建纯逻辑包，不安装输入法，因此非常适合验证环境。随后用 Xcode 打开 `MyIME/MyIME.xcodeproj`，选择 `MyIME Scheme` 构建 `Release`。课堂中每名学生都应维护一份“环境记录”：macOS 版本、芯片架构、Xcode 版本、输入源 ID、安装位置和最后一次成功测试时间。

工程警告：不要用“能编译”代替“能输入”

输入法至少包含四层成功条件：编译成功、安装成功、系统注册成功、目标应用中的组合输入成功。它们互不等价。尤其在换机分发时，签名、公证、App Translocation 和输入源缓存都可能让前两层成功而第四层失败。

动手实验：基线记录

运行纯引擎测试，记录测试总数与耗时；检查系统设置中的当前输入源；在 Notes 中分别用 ABC 和系统简体拼音输入一行文字。此实验不是为了验证 MyIME，而是建立后续对照组。

目录

前言：输入法是一门小型系统课程	iii
读者对象与先修知识	iv
课程组织建议	v
符号、术语与源码约定	vi
开发环境与第一次构建	vii
第一部分 初级篇：让文字真正上屏	1
第一章 把输入法看成一个系统服务	2
1.1 四个容易混淆的对象	2
1.2 从入口函数阅读真实项目	3
1.3 输入法的成功条件	4
思考与练习	4
第二章 输入源元数据与最小 IMK 服务	6
2.1 稳定标识符为何重要	8
2.2 最小服务的责任边界	8
思考与练习	9
第三章 键盘事件：消费、放行与语义化	10
3.1 消费事件的契约	12
3.2 状态决定同一个按键的含义	12
3.3 中英文边界	13
思考与练习	13
第四章 组合输入状态机：raw、preedit 与 commit	14

4.1 三个文本层次	14
4.2 状态不变量	16
4.3 部分消费	17
思考与练习	17
第五章 用协议隔离系统与算法	18
5.1 输入与输出应是值	21
5.2 协议提供替身	21
5.3 线程安全不是加一个标记	23
思考与练习	24
第六章 从小词库生成第一组中文候选	25
6.1 前缀匹配与完整匹配	25
6.2 接口而非文件格式	26
6.3 候选的可解释性	27
思考与练习	27
第七章 候选窗口：把算法结果变成可操作界面	29
7.1 候选窗口不应成为主窗口	32
7.2 定位坐标	32
7.2.1 macOS 中的三层坐标空间	32
7.3 分页与数字映射	36
7.4 VoiceOver 与无障碍高亮	36
思考与练习	37
第八章 偏好设置、繁简输出与可观察性	38
8.1 设置传播的三个问题	41
8.2 繁体输出是映射，不是另一套词库	41
8.3 状态菜单是诊断界面	42
思考与练习	44
第九章 第一次端到端验收	46
9.1 基础验收矩阵	46

9.2 目标应用分层	46
9.3 记录证据	47
思考与练习	47
第二部分 高级篇：从候选词到语言模型	49
第十章 拼音切分：把字符串变成图	50
10.1 路径不仅包含音节	51
10.2 搜索与限制	52
思考与练习	55
第十一章 简拼、混拼、模糊音与错拼容忍	56
11.1 简拼与混拼	56
11.2 模糊音是可配置关系	58
11.3 相邻字母颠倒	59
思考与练习	60
第十二章 SQLite 词库：模式、索引与只读运行时	61
12.1 查询模式决定索引	61
12.2 只读与完整性	64
12.3 用户库写入	64
思考与练习	67
第十三章 候选评分：从词频到对数域	69
13.1 一个可解释的总分	70
13.2 长度偏差	73
13.3 稳定排序	73
思考与练习	73
第十四章 词格与 Beam Search：生成整句候选	75
14.1 宽度为 2 的动态推演	75
14.2 假设状态	77
14.3 转移分数	78

14.4 时间预算	81
思考与练习	81
第十五章 上下文与本地用户学习	83
15.1 三种学习信号	83
15.2 近期组词的边界	87
15.3 隐私与控制权	89
思考与练习	89
第十六章 Emoji、英文辅助与繁体转换的插件化	90
16.1 Emoji 紧随语义候选	90
16.2 英文辅助不是英文模式	92
16.3 后处理次序	94
思考与练习	94
第十七章 准确率、延迟与科学评测	95
17.1 候选指标	95
17.2 延迟指标	95
17.3 MyIME 内部回归集	96
17.4 避免数据泄漏	99
思考与练习	100
第三部分 工程篇：把实验变成可发布的软件	101
第十八章 安装与输入源注册：为什么复制应用还不够	102
18.1 幂等安装	102
18.2 同一 Bundle ID 的多个副本	106
18.3 App Translocation	106
思考与练习	110
第十九章 InputMethodKit 的多条事件路径	111
19.1 统一语义，不统一假设	114
19.2 客户端协议差异	114

思考与练习	115
第二十章 组合会话、失活恢复与幽灵状态	116
20.1 策略矩阵	118
20.2 弱引用与所有权	118
思考与练习	119
第二十一章 日志、诊断与证据链	120
21.1 用 Logger 建立可过滤日志	120
21.2 Console.app 过滤 SOP	121
21.3 诊断问题树	121
21.4 隐私友好的日志	122
21.5 死锁与卡顿调试 SOP	122
21.6 诊断工具	123
思考与练习	124
第二十二章 隐私、安全与词库许可证	126
22.1 威胁模型	126
22.2 安全输入	127
22.3 开源与词库许可	127
思考与练习	130
第二十三章 测试体系：从内存词库到跨机器验收	131
23.1 行为测试	131
23.2 性质与故障测试	134
23.3 性能测试	134
23.4 跨机器门禁	134
思考与练习	134
第二十四章 签名、公证、Universal 2 与 DMG 发布	136
24.1 签名身份	136
24.2 Universal 2	136
24.3 发布脚本	137

24.4 凭据管理	141
思考与练习	141
第二十五章 维护真实输入法：技术债与演进边界	143
25.1 复杂度地图	143
25.2 规格、实现与测试同步	144
25.3 功能边界	144
25.4 把无障碍作为回归契约	144
思考与练习	145
第四部分 课程实验与综合项目	146
第二十六章 实验一至四：从 Echo 到候选窗口	147
26.1 从哪个分支开始	147
26.2 实验一：Echo IME	148
26.3 实验二：组合状态	148
26.4 实验三：小词库候选	149
26.5 实验四：候选窗口	149
26.6 评分量表	149
第二十七章 实验五至八：切分、排序、整句与学习	150
27.1 实验五：切分图	150
27.2 实验六：评分解释器	150
27.3 实验七：词格整句	151
27.4 实验八：用户学习	151
27.5 实验报告模板	151
第二十八章 实验九至十二：兼容、诊断、安全与发布	152
28.1 实验九：跨客户端兼容矩阵	152
28.2 实验十：诊断包	152
28.3 实验十一：离线与供应链审计	153
28.4 实验十二：第二台 Mac 发布验收	153

第二十九章 综合项目：实现一款可解释的离线输入法 **154**

29.1 项目契约 154

29.2 最终交付物 154

29.3 100 分三级评分量表 155

29.4 答辩必问十题 156

附录 A Swift 与并发速查 **157**

A.1 值类型与引用类型 157

A.2 协议与 existential 157

A.3 可选值与 guard 157

A.4 Sendable、锁与队列 157

A.5 输入法线程模型：先画所有权，再写并发代码 158

 A.5.1 @MainActor、Sendable 与 @unchecked Sendable 分别保证什么 159

 A.5.2 NSLock 的两条铁律 159

A.6 String 索引 160

附录 B InputMethodKit 与 TIS 速查 **161**

B.1 核心类型 161

B.2 常见按键语义 161

B.3 AppKit 坐标转换专题 162

附录 C 算法推导与第二方法验证 **164**

C.1 切分的动态规划形式 164

C.2 Viterbi 与 Beam 的关系 164

C.3 对数域数值示例 164

C.4 平滑 165

附录 D 词库流水线与可复现数据工程 **166**

D.1 建议的中间格式 168

D.2 去重与来源保留 168

D.3 可复现性 168

D.4 五分钟编译：从 CIF 到 SQLite 169

附录 E 故障排查手册	170
E.1 先确认系统版本：版本防坑矩阵	170
E.2 升级后的十分钟回归	171
E.3 系统列表没有 MyIME	171
E.4 能看到 MyIME，但中英文都打不出来	171
E.5 有拼音下划线，没有候选	171
E.6 候选出现但无法上屏	171
E.7 Notes 正常，浏览器失败	172
E.8 开发机正常，目标机失败	172
E.9 进程结束后无法恢复	172
附录 F 发布命令与验收清单	173
F.1 纯引擎	173
F.2 应用构建	173
F.3 签名、公证与架构检查	173
F.4 发布前人工清单	173
附录 G 术语表	175
附录 H 练习提示与参考方向	177
H.1 初级篇提示	177
H.2 切分提示	177
H.3 评分提示	177
H.4 词格提示	177
H.5 学习提示	178
H.6 发布提示	178
附录 I 源码阅读路线图	179
结束语：把每一次按键变成证据	181
版本与勘误记录	182

第一部分

初级篇：让文字真正上屏

把输入法看成一个系统服务

学习目标

能够区分键盘布局、输入源、输入法应用和普通文本应用；能够画出客户端、IMKServer 与控制器的通信关系；理解“进程在运行”和“系统正在把按键交给它”不是同一件事。

普通应用拥有窗口，接收键盘事件，再把字符写入自己的文本控件。输入法则站在键盘与普通应用之间：系统把一部分事件交给输入法，输入法维护尚未完成的组合状态，并通过文本输入协议把标记文本或最终文本送回客户端。客户端可能是 Notes，也可能是浏览器的网页输入框、Electron 编辑器或终端。

InputMethodKit(IMK)提供了这条通信链路。输入法进程创建 **IMKServer**，系统根据应用包中的元数据找到服务，再为每个输入会话创建 **IMKInputController**。控制器不是一个独立窗口，也不应该假设自己拥有键盘焦点；它是系统在合适时机调用的对象。

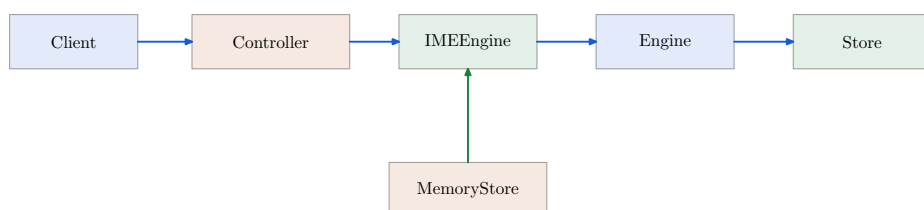


图 1.1: 系统适配层、纯逻辑引擎和可替换存储之间的边界

1.1 四个容易混淆的对象

输入源 系统设置里可以选中的语言输入项，具有稳定的输入源标识符；

输入法应用 安装在 `~/Library/InputMethods` 等位置的应用包；

输入法进程 应用被系统启动后形成的运行实例；

输入会话 某个客户端与输入法之间的一段组合输入关系。

一次常见误诊是：活动监视器里能看到 MyIME，于是断言输入法已经工作。进程存在只说明应用被启动；若系统仍选中 ABC，或者当前客户端没有建立 IMK 会话，按键仍不会进入控制器。反过来，用户强制结束输入法进程后，macOS 通常会在下一次切换输入源时重新启动它。因此输入法不是传统意义上必须常驻的“守护进程”，它更接近由系统托管的服务。

1.2 从入口函数阅读真实项目

MyIME 的入口先处理安装命令，再只在“已安装副本”中创建服务器。这个判断避免从 DMG 或构建目录直接启动的副本冒充系统输入服务。

```

1  import AppKit
2  import Carbon
3  import InputMethodKit
4  import IMKit
5  import os
6  import SwiftUI
7
8  @main @MainActor
9  struct MyIMEApplication {
10     static func main() {
11         let logger = Logger(subsystem: "fudan.miniS.MyIME", category: "command")
12         if SelfInstaller.performCommandIfRequested(logger: logger) {
13             return
14         }
15         let server: IMKServer?
16         if SelfInstaller.isRunningInstalledCopy,
17             let connectionName = Bundle.main.object(forKey: "
InputMethodConnectionName") as? String {
18             server = IMKServer(name: connectionName, bundleIdentifier: Bundle.main.
bundleIdentifier)
19         } else {
20             server = nil
21         }
22         let app = NSApplication.shared
23         let delegate = AppDelegate(server: server)
24         app.delegate = delegate
25         app.setActivationPolicy(.accessory)

```

```
26     app.run()  
27 }  
28 }
```

Listing 1.1: MyIME 的应用入口与 IMKServer 创建

源码导读：阅读顺序

先找 `@main`，再找 `IMKServer`，最后看 `app.run()`。不要一开始就进入候选排序。只要服务器没有正确建立，再优秀的词库和算法也不会收到一个按键。

1.3 输入法的成功条件

我们把一次成功输入拆成五个可单独验证的命题：

1. 应用包位于系统允许的输入法目录；
2. 输入源元数据能被系统枚举；
3. 用户选择了正确的输入源；
4. `IMKServer` 与控制器被创建，事件进入控制器；
5. 控制器向当前客户端正确设置组合文本并提交。

这种分解是本书的第一条工程原则：把“不能输入”改写成一组可证伪的命题。

动手实验：观察系统状态

分别在选中 ABC、系统简体拼音和 MyIME 时观察菜单栏、活动监视器与 Notes。记录哪些状态发生变化。然后结束 MyIME 进程，切换一次 ABC 再切回 MyIME，观察系统是否重启服务。

思考与练习

1. 为什么不能把菜单栏显示“中”当作控制器一定收到按键的证据？
2. 如果 DMG 中的应用能打开设置窗口，但系统列表没有该输入法，应优先检查哪两层？
3. 画出从 Notes 到 MyIME，再回到 Notes 的一次完整调用路径。

本章小结

输入法首先是系统服务，其次才是转换算法。定位故障时应按“安装—注册—选择—事件—上屏”的顺序逐层取证。

输入源元数据与最小 IMK 服务

学习目标

理解 `Info.plist` 在输入源发现过程中的作用；能够解释连接名、控制器类、语言标识和输入源 ID；能够设计一个只完成字母回送的最小输入法。

macOS 不会扫描 Swift 类型来猜测一个应用是不是输入法。系统首先读取应用包中的元数据。MyIME 的 `Info.plist` 同时描述应用身份和输入模式，其中最关键的是服务器连接名、控制器类和 `ComponentInputModeDict`。

```
1      <string>1.0.10</string>
2      <key>CFBundleVersion</key>
3  <string>76</string>
4      <key>InputMethodConnectionName</key>
5      <string>MyIME_1_Connection</string>
6      <key>InputMethodServerControllerClass</key>
7      <string>MyIMEInputController</string>
8      <key>InputMethodServerDelegateClass</key>
9      <string>MyIMEInputController</string>
10     <key>tsInputMethodCharacterRepertoireKey</key>
11     <array>
12         <string>Hans</string>
13         <string>Hant</string>
14         <string>Latn</string>
15     </array>
16     <key>tsInputMethodIconFileKey</key>
17     <string>menu_icon.pdf</string>
18     <key>ComponentInputModeDict</key>
19     <dict>
20         <key>tsInputModeListKey</key>
21         <dict>
22             <key>fudan.miniS.inputmethod.MyIME.Chinese</key>
23             <dict>
```

```

24         <key>TISInputSourceID</key>
25         <string>fudan.miniS.inputmethod.MyIME.Chinese</string>
26         <key>TISIntendedLanguage</key>
27         <string>zh-Hans</string>
28         <key>tsInputModeAlternateMenuIconFileKey</key>
29         <string>menu_icon.pdf</string>
30         <key>tsInputModeCharacterRepertoireKey</key>
31         <array>
32             <string>Hans</string>
33             <string>Hant</string>
34         </array>
35         <key>tsInputModeDefaultStateKey</key>
36         <true/>
37         <key>tsInputModeIsVisibleKey</key>
38         <true/>
39         <key>tsInputModeKeyEquivalentKey</key>
40         <string></string>
41         <key>tsInputModeKeyEquivalentModifiersKey</key>
42         <integer>4608</integer>
43         <key>tsInputModeMenuIconFileKey</key>
44         <string>menu_icon.pdf</string>
45         <key>tsInputModePaletteIconFileKey</key>
46         <string>menu_icon.pdf</string>
47         <key>tsInputModePrimaryInScriptKey</key>
48         <true/>
49         <key>tsInputModeScriptKey</key>
50         <string>smSimpChinese</string>
51     </dict>
52 </dict>
53     <key>tsVisibleInputModeOrderedArrayKey</key>
54     <array>
55         <string>fudan.miniS.inputmethod.MyIME.Chinese</string>
56     </array>
57 </dict>
58 <key>LSBackgroundOnly</key>
59 <false/>
60 <key>LSHasLocalizedDisplayName</key>
61 <true/>
62 <key>LSUIElement</key>
63 <true/>
64 <key>NSPrincipalClass</key>
65 <string>NSApplication</string>
66 <key>NSHighResolutionCapable</key>

```

```

67     <true/>
68     <key>TICapsLockLanguageSwitchCapable</key>
69     <true/>
70 </dict>
71 </plist>

```

Listing 2.1: 输入法身份与简体中文输入模式声明

2.1 稳定标识符为何重要

Bundle ID、输入源 ID 与连接名承担不同职责。Bundle ID 标识应用；输入源 ID 标识用户在系统菜单中选择的模式；连接名帮助系统连接服务器。课堂项目中可以使用学校反向域名，例如 `edu.example.course.MyIME`，但一旦对外发布就不应随意改变。改变标识符会让系统把升级版视作另一个输入法，用户可能同时看到两个同名条目。

核心概念：语言归类

`TISIntendedLanguage=zh-Hans` 告诉系统该模式面向简体中文。若语言声明缺失或不一致，第三方输入法可能出现在错误分类中，或者在“添加输入法”对话框里难以找到。教学时应让学生把 UI 现象追溯到元数据，而不是只记住一个 plist 模板。

2.2 最小服务的责任边界

一个 Echo IME 只做三件事：建立服务器、创建控制器、把收到的可打印字符原样提交。它不需要 SQLite、候选窗、自学习和安装器。最小模型的意义在于证明系统链路畅通。若 Echo IME 都不能工作，加入词库只会制造更多变量。

概念代码如下：

```

1 final class EchoController: IMKInputController {
2     override func inputText(_ string: String!, client sender: Any!) -> Bool {
3         guard let client = sender as? IMKTextInput,
4             let string, !string.isEmpty else { return false }
5         client.insertText(string, replacementRange: NSRange(location: NSNotFound,
6             length: 0))
7         return true
8     }
9 }

```



```
8 }
```

Listing 2.2: 教学用 Echo 控制器骨架

这段代码故意不处理组合状态。它帮助学生回答一个关键问题：当前事件投递路径究竟调用 `inputText`，还是调用 `handle(_:client:)`？不同系统版本、输入服务模式 and 客户端可能表现不同。真实项目因此同时考虑文本数据投递与事件投递。

工程警告：不要硬编码“唯一正确回调”

`InputMethodKit` 的困难之一，是教材示例往往只在作者的系统和文本控件上运行。课堂验收应至少覆盖 `Notes` 与一个浏览器输入框，并在日志中确认真正被调用的方法。

动手实验：最小输入链路

复制项目并新建教学 `target`，只保留入口、`plist` 和 `Echo` 控制器。安装后在 `Notes` 输入 `abc123`。依次回答：数字是否进入控制器？修饰键是否进入？返回 `false` 时客户端发生什么？

思考与练习

1. `Bundle ID` 与输入源 `ID` 可以相同吗？即使可以，为什么仍应在概念上区分？
2. 如果类名在 `plist` 中写错，编译器为什么不会报错？运行时会出现什么证据？
3. 设计一个实验，证明事件被输入法消费后不会再由客户端重复处理。

本章小结

元数据决定系统能否发现和连接输入法。最小 `Echo IME` 是所有后续算法的控制实验，也是排除安装与事件链路问题的最快工具。

键盘事件：消费、放行与语义化

学习目标

能够区分字符、硬件键码和修饰键；理解返回值对事件传播的影响；把按键映射为“追加、删除、选择、提交、取消、移动”六类输入法动作。

输入法不应该把业务逻辑写成一长串神秘键码。更清晰的方法是先把原始 `NSEvent` 翻译成语义动作，再由状态机处理动作。例如：字母产生 `append(Character)`，退格产生 `deleteBackward`，空格产生 `select(0)` 或普通空格，取决于是否存在组合状态。

MyIME 的控制器入口展示了真实工程需要面对的判断顺序。

```
1  override func handle(_ event: NSEvent!, client sender: Any!) -> Bool {
2      guard let event, let client = textClient(from: sender) else {
3          logger.error("无法从输入事件取得文本客户端")
4          return false
5      }
6      activeClient = client
7      logger.debug("收到输入事件: type=\(event.type.rawValue), keyCode=\(event.
      keyCode)")
8      return safelyHandle(event, client: client)
9  }
10
11  /// Some WebKit/Electron text clients fall back to the key-binding delivery
12  path instead
13  /// of forwarding NSEvent objects. Supporting this callback keeps those clients
14  usable.
15  override func inputText(_ string: String!, client sender: Any!) -> Bool {
16      guard let string, !string.isEmpty, let client = textClient(from: sender)
17      else { return false }
18      activeClient = client
19
20      for character in string {
```

```

18         if character.isASCII, character.isLetter {
19             if raw.isEmpty {
20                 composedPhrase = ""
21                 composedPinyin = []
22             }
23             raw.append(String(character).lowercased())
24             refresh(client)
25             continue
26         }
27         if character == "'", !raw.isEmpty {
28             raw.append(character)
29             refresh(client)
30             continue
31         }
32         if character == " " {
33             if !raw.isEmpty {
34                 commitHighlighted(client)
35             } else if showingSuggestions, !output.candidates.isEmpty {
36                 commitSuggestion(highlighted, client: client)
37             } else {
38                 client.insertText(" ", replacementRange: NSRange(location:
147 NSNotFound, length: NSNotFound))
39             }
40             continue
41         }
42         if let number = character.wholeNumberValue, (!raw.isEmpty ||
147 showingSuggestions) {
43             let maximumNumber = expandedCandidates ? 6 : 9
44             if (1...maximumNumber).contains(number) {
45                 let activeRowStart = expandedCandidates ? highlighted / 6 * 6 :
147 0
46                 if showingSuggestions, raw.isEmpty {
47                     commitSuggestion(activeRowStart + number - 1, client:
147 client)
48                 } else {
49                     selectVisible(activeRowStart + number - 1, client: client)
50                 }
51                 continue
52             }
53         }
54         if !raw.isEmpty {
55             commitHighlighted(client)
56             let text = character.isASCII

```

```

57         ? localizedPunctuation(for: character)
58         : String(character)
59         client.insertText(text, replacementRange: NSRange(location:
    NSRange(location: NSNotFound, length: NSNotFound))
60         engine.breakPhraseLearningContext()
61         clearSuggestions()
62     } else {
63         client.insertText(String(character), replacementRange: NSRange(
    NSRange(location: NSNotFound, length: NSNotFound))
64     }
65 }
66 logger.debug("已通过 inputText 回退路径处理输入")
67 return true
68 }
69
70 /// Some custom text controls (including Electron/WebKit-based clients) request
    the

```

Listing 3.1: 事件入口中的修饰键、字母与特殊键分派

3.1 消费事件的契约

回调返回 `true` 通常表示输入法已经处理事件，客户端不应再次处理；返回 `false` 表示放行。错误的返回值会产生两类典型故障：

- 输入法上屏一次、客户端又输入一次，形成重复字符；
- 输入法没有真正处理，却吞掉事件，表现为中英文都无法输入。

所以返回值不是语法细节，而是输入法与客户端之间的所有权声明。

3.2 状态决定同一个按键的含义

`Space` 在空闲态应该是普通空格，在组合态通常选择第一候选；`Return` 在空闲态交给客户端，在组合态可能提交候选或原始英文；数字在候选态选词，在其他状态则应输入数字。正确行为可以写成决策表。

按键	空闲态	组合态	候选导航态
Space	放行	首选上屏	当前候选上屏
Return	放行	提交候选/原文	当前候选上屏
Esc	放行	取消组合	取消组合
Backspace	放行	删除 raw 尾字符	删除 raw 尾字符
1-9	放行	数字选词	数字选词
方向键	放行	进入导航	移动高亮

核心概念：动作层

建议课堂实现 `enumInputAction`，把事件解析与状态改变分开测试。这样无需启动 `IMKServer`，也能验证 `Shift`、`Option`、组合键和数字键的语义。

3.3 中英文边界

MyIME 1.0.10 把“选中 MyIME”定义为中文状态，持续英文输入时切换 ABC；在组合状态下按回车可以提交原始英文，并提供基础拼写建议。这一设计舍弃了隐藏的 `Shift` 中英文会话开关，换取更明确的系统状态。教学中应强调：这是一项产品决策，不是输入法框架的唯一答案。

动手实验：按键矩阵

为 6 种动作与 3 种状态建立至少 18 个测试格。人工测试时同时记录“控制器是否返回 `true`”“客户端最终文本”“raw 是否清空”“候选窗是否关闭”。

思考与练习

- 1. 为什么不应只根据 `charactersIgnoringModifiers` 判断所有快捷键？
- 2. 在组合态按 `Command+C` 时，输入法应该怎样处理？说明理由。
- 3. 设计 `Caps Lock` 切换策略，并指出它与系统输入源切换可能发生的冲突。

本章小结

键盘事件必须先语义化，再交给状态机。返回值决定事件所有权；同一按键的意义取决于当前是否存在组合与候选导航状态。

组合输入状态机：raw、preedit 与 commit

学习目标

掌握输入法的最小状态模型；理解 raw、preedit、marked text 和 committed text 的区别；能够正确处理退格、取消、部分选择与会话结束。

中文输入不是一次按键对应一次字符。用户输入 **ni hao** 时，六个字母先构成 **raw buffer**；引擎可能把它切分并显示为 **ni 'hao**，这是 **preedit**；用户选中“你好”以后才产生最终提交文本。组合期间显示的内容属于客户端文本系统，但仍由输入法管理。

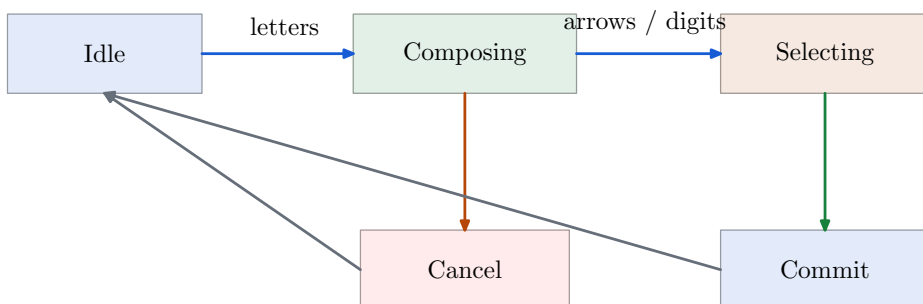


图 4.1: 输入法组合状态机及关键转移

4.1 三个文本层次

raw 用户实际输入、可继续编辑的字母序列；

preedit 为帮助理解而格式化的组合表示，例如插入音节边界；

converted/marked 客户端中带标记样式显示的当前转换结果；

commit 取消标记、成为普通文档内容的最终文本。

初级实现可以让 **marked text** 始终等于 **raw**。高级实现可以显示“已转换前缀 + 未转换拼音”。无论采用哪种 UI，都必须维持一个不变量：**客户端里的 **marked range** 与控制器认为的组合状态一致。**

MyIME 通过专门方法构造并设置组合文本：

```

1      output = addingEnglishCandidates(to: engine.update(raw, prefs: prefs))
2      page = 0
3      highlighted = 0
4      expandedCandidates = false
5      hasNavigatedCandidates = false
6      let converted = localized(composedPhrase, prefs: prefs)
7      let rawPreedit = output.preedit.isEmpty ? raw : output.preedit
8      // Notes / Markdown / Electron clients often ignore plain String marked
   text.
9      // Send an attributed buffer with IMK mark styles, matching Squirrel/落格.
10     setMarkedText(converted: converted, rawPreedit: rawPreedit, client: client)
11     candidateWindow.update(
12         candidates: localizedCandidates(visibleCandidates, prefs: prefs),
13         highlighted: highlighted,
14         prefs: prefs,
15         expanded: expandedCandidates,
16         client: client
17     )
18 }
19
20 /// Builds the inline composition buffer the way InputMethodKit expects.
21 /// Plain `String` works in many AppKit clients, but Notes and several
22 /// WebKit/Electron editors only honor attributed marked text with TSM hilite
   styles.
23 private func setMarkedText(converted: String, rawPreedit: String, client:
   IMKTextInput) {
24     let marked = converted + rawPreedit
25     let attributed = NSMutableAttributedString(string: marked)
26     let convertedLength = (converted as NSString).length
27     let totalLength = attributed.length
28
29     if convertedLength > 0 {
30         let range = NSRange(location: 0, length: convertedLength)
31         if let attrs = mark(forStyle: kTSMHiliteConvertedText, at: range) as? [
   NSAttributedString.Key: Any] {
32             attributed.setAttributes(attrs, range: range)

```

```

33     }
34 }
35 if convertedLength < totalLength {
36     let range = NSRange(location: convertedLength, length: totalLength -
convertedLength)
37     if let attrs = mark(forStyle: kTSMHiliteSelectedRawText, at: range) as?
[NSAttributedString.Key: Any] {
38         attributed.setAttributes(attrs, range: range)
39     }
40 }
41
42 client.setMarkedText(
43     attributed,
44     selectionRange: NSRange(location: totalLength, length: 0),
45     replacementRange: NSRange(location: NSNotFound, length: NSNotFound)
46 )
47 }
48
49 private func clearMarkedText(client: IMKTextInput) {
50     client.setMarkedText(
51         NSAttributedString(string: ""),
52         selectionRange: NSRange(location: 0, length: 0),
53         replacementRange: NSRange(location: NSNotFound, length: NSNotFound)
54     )
55 }

```

Listing 4.1: 引擎更新、候选窗刷新与 marked text 设置

4.2 状态不变量

可以用以下不变量检查控制器:

1. `raw.isEmpty` 时不应保留可见候选窗;
2. 取消或提交后, `raw`、`preedit`、高亮索引与展开状态应一起复位;
3. 高亮索引必须落在当前可见候选范围内;
4. 候选消费的 `raw` 长度不能超过输入长度;
5. 客户端失活时不能把旧组合错误提交到新客户端。

4.3 部分消费

候选不一定消费全部输入。例如用户输入 `beijingdaxue`，如果选择只消费 `beijing` 的“北京”，剩余 `daxue` 应继续参与下一轮候选生成。设输入长度为 n ，候选消费长度为 k ，则：

$$\text{remainingRaw} = \text{raw}[k \dots n].$$

这个看似简单的切片必须明确以字节、Unicode scalar 还是 Swift `String.Index` 计算。MyIME 对 `raw` 进行 ASCII 归一化，因此消费长度可以稳定对应字母数量；若未来允许带声调 Unicode 字符，模型必须重新审视。

工程警告：会话结束

应用切换、输入源切换、窗口关闭和输入法进程退出都可能结束组合。工程实现要明确选择“提交首选”“提交原始拼音”还是“取消”。不能让一段旧组合在下次激活时幽灵般出现。

动手实验：纸上状态机

从空闲态开始，逐步执行：输入 `nihao`、按左方向键、按数字 2、切换到 ABC、再切回 MyIME。每一步写出 `raw`、候选、高亮、`marked text` 与文档最终文本。

思考与练习

1. 为什么 `preedit` 不应被当作最终文档文本？
2. 若候选消费长度计算少 1，会出现哪些用户可见症状？
3. 为“应用失去焦点”设计三种策略，并比较数据安全与用户体验。

本章小结

组合输入的本质是一个有明确所有权的临时事务。`raw` 是输入法状态，`marked text` 是客户端中的映射，`commit` 才是最终文档内容。

用协议隔离系统与算法

学习目标

理解为什么输入法核心不应依赖 AppKit；能够通过协议注入词库和引擎；掌握可测试数据模型的最小集合。

IMKInputController 很难在普通单元测试中实例化，因为它依赖系统输入会话。若拼音切分、候选排序和用户学习都写在控制器里，算法只能通过手工敲字验证。MyIME 将核心放入独立 Swift Package IMEKit，控制器只负责事件与 UI 适配。

```
1 import Foundation
2
3 public struct Candidate: Hashable, Sendable {
4     public let id: Int64
5     public let word: String
6     public let pinyinPath: [String]
7     public let score: Double
8     public let consumedLength: Int
9
10    public init(id: Int64, word: String, pinyinPath: [String], score: Double,
11               consumedLength: Int) {
12        self.id = id
13        self.word = word
14        self.pinyinPath = pinyinPath
15        self.score = score
16        self.consumedLength = consumedLength
17    }
18 }
19
20 public struct EngineOutput: Sendable {
21     public let preedit: String
22     public let candidates: [Candidate]
23     public let hasMore: Bool
```

```

23     public let raw: String
24
25     public init(preedit: String, candidates: [Candidate], hasMore: Bool, raw:
26         String) {
27         self.preedit = preedit
28         self.candidates = candidates
29         self.hasMore = hasMore
30         self.raw = raw
31     }
32
33     public struct SelectResult: Sendable {
34         public let commitText: String
35         public let remainingRaw: String
36
37         public init(commitText: String, remainingRaw: String) {
38             self.commitText = commitText
39             self.remainingRaw = remainingRaw
40         }
41     }
42
43     public struct FuzzyRules: Codable, Equatable, Sendable {
44         public var zZh = false
45         public var cCh = false
46         public var sSh = false
47         public var nL = false
48         public var fH = false
49         public var rL = false
50         public var anAng = false
51         public var enEng = false
52         public var inIng = false
53         public var ianIang = false
54         public var uanUang = false
55
56         public init() {}
57     }
58
59     public enum CandidateOrientation: String, Codable, CaseIterable, Sendable {
60         case horizontal
61         case vertical
62     }
63
64     public struct EnginePrefs: Codable, Equatable, Sendable {

```

```
65     public var pageSize = 5
66     public var fuzzy = FuzzyRules()
67     public var disabledSourceMask = 0
68     public var fontSize = 18.0
69     public var followsSystemAppearance = true
70     public var orientation = CandidateOrientation.horizontal
71     public var outputTraditional = false
72     public var frequencyWeight = 1.0
73     public var userWeight = 0.35
74     public var lengthWeight = 0.12
75     public var contextWeight = 0.2
76     public var fuzzyPenalty = 0.08
77     public var partialPenalty = 0.1
78
79     public init() {}
80 }
81
82 public enum PreferencesStore {
83     public static let suiteName = "group.fudan.miniS.MyIME"
84     private static let key = "enginePreferences"
85
86     public static func load() -> EnginePrefs {
87         guard let data = UserDefaults(suiteName: suiteName)?.data(forKey: key),
88             let value = try? JSONDecoder().decode(EnginePrefs.self, from: data)
89         else {
90             return EnginePrefs()
91         }
92         return value
93     }
94
95     public static func save(_ preferences: EnginePrefs) {
96         guard let data = try? JSONEncoder().encode(preferences) else { return }
97         UserDefaults(suiteName: suiteName)?.set(data, forKey: key)
98         NotificationCenter.default.post(name: .enginePreferencesDidChange, object:
99         nil)
100     }
101 }
102
103 public extension Notification.Name {
104     static let enginePreferencesDidChange = Notification.Name("MyIME.
105     enginePreferencesDidChange")
106 }
```

```

105 public protocol IMEngine: AnyObject {
106     func update(_ raw: String, prefs: EnginePrefs) -> EngineOutput
107     func select(_ index: Int, from output: EngineOutput) -> SelectResult
108     func commitLearning(word: String, pinyin: [String])
109 }

```

Listing 5.1: 候选、引擎输出、偏好和 IMEngine 协议

5.1 输入与输出应是值

`EngineOutput` 是一次更新的快照，包含 `preedit`、候选、是否还有更多结果和归一化 `raw`。值类型让测试可以完整比较结果，也避免候选窗在引擎下一次更新后意外看到被修改的共享数组。

候选至少需要：显示文本、读音路径、分数和消费长度。数据库 ID 便于回溯词条，但 UI 不应依赖 ID 的业务含义。用户词使用负 ID 是当前实现的内部约定，课堂中应把这种约定封装在存储层。

5.2 协议提供替身

词库协议 `CandidateLookup` 允许真实 `SQLiteStore` 与测试 `MemoryStore` 被同一个引擎使用。这样可以构造只有“天气”和“田七”的微型世界，精确验证上下文是否改变排序。

```

1  import Foundation
2  import Testing
3  @testable import IMKit
4
5  private struct MemoryStore: CandidateLookup {
6      let entries: [StoreEntry]
7
8      func lookup(pinyinKey: String, initials: String, disabledSourceMask: Int, limit
        : Int) -> [StoreEntry] {
9          entries.filter {
10              $0.pinyin.replacingOccurrences(of: "'", with: "").hasPrefix(pinyinKey)
11              || $0.pinyin.split(separator: "'").compactMap(\.first).map(String.
12              init).joined() == initials
13          }
14      }

```

```

15     func userBoost(word: String, pinyin: String) -> Double { 0 }
16     func bigramBoost(previous: String?, word: String) -> Double { 0 }
17 }
18
19 private let store = MemoryStore(entries: [
20     StoreEntry(id: 1, word: "你好", pinyin: "ni'hao", weight: 65_535),
21     StoreEntry(id: 2, word: "世界", pinyin: "shi'jie", weight: 65_535),
22     StoreEntry(id: 3, word: "北京", pinyin: "bei'jing", weight: 65_535),
23     StoreEntry(id: 4, word: "中国", pinyin: "zhong'guo", weight: 65_535),
24     StoreEntry(id: 5, word: "我爱你", pinyin: "wo'ai'ni", weight: 65_535),
25     StoreEntry(id: 6, word: "我", pinyin: "wo", weight: 65_535),
26     StoreEntry(id: 7, word: "是", pinyin: "shi", weight: 65_535),
27     StoreEntry(id: 8, word: "人", pinyin: "ren", weight: 65_535),
28     StoreEntry(id: 9, word: "今天", pinyin: "jin'tian", weight: 65_535),
29     StoreEntry(id: 10, word: "天气", pinyin: "tian'qi", weight: 65_535),
30     StoreEntry(id: 11, word: "很好", pinyin: "hen'hao", weight: 65_535),
31     StoreEntry(id: 12, word: "田七", pinyin: "tian'qi", weight: 45_000),
32     StoreEntry(id: 13, word: "医院", pinyin: "yi'yuan", weight: 65_535),
33 ])
34
35 private struct ContextStore: CandidateLookup {
36     let entries: [StoreEntry]
37
38     func lookup(pinyinKey: String, initials: String, disabledSourceMask: Int, limit
39         : Int) -> [StoreEntry] {
40         entries.filter {
41             $0.pinyin.replacingOccurrences(of: "'", with: "").hasPrefix(pinyinKey)
42             || $0.pinyin.split(separator: "'").compactMap(\.first).map(String.
43             init).joined() == initials
44         }
45     }
46
47     func userBoost(word: String, pinyin: String) -> Double { 0 }
48     func bigramBoost(previous: String?, word: String) -> Double {
49         previous == "今天" && word == "天气" ? 10 : 0
50     }
51 }
52
53 private struct PredictingStore: CandidateLookup {
54     let entries: [StoreEntry]
55     let predictions: [String: [StoreEntry]]
56
57     func lookup(pinyinKey: String, initials: String, disabledSourceMask: Int, limit

```

```

56     : Int) -> [StoreEntry] {
57         entries.filter {
58             $0.pinyin.replacingOccurrences(of: "", with: "").hasPrefix(pinyinKey)
59             || $0.pinyin.split(separator: "").compactMap(\.first).map(String.
60             init).joined() == initials
61         }
62     }
63
64     func userBoost(word: String, pinyin: String) -> Double { 0 }
65     func bigramBoost(previous: String?, word: String) -> Double {
66         previous == "今天" && word == "天气" ? 1 : 0
67     }
68     func predictNext(after previous: String, limit: Int) -> [StoreEntry] {
69         Array((predictions[previous] ?? []).prefix(limit))
70     }
71 }

```

Listing 5.2: 测试使用的内存词库与上下文词库

核心概念：依赖方向

系统层可以依赖纯逻辑层，纯逻辑层不能反过来依赖 InputMethodKit。数据库实现依赖 SQLite，但引擎只依赖查询协议。候选窗口依赖模型，却不应该执行词库 SQL。这种单向依赖使每层都能被独立替换。

5.3 线程安全不是加一个标记

源码中部分类型声明为 @uncheckedSendable，意味着编译器把线程安全证明交给程序员。它不是“消除警告”的装饰。Engine 使用锁保护提交历史，UserStore 需要串行化写入。教学答辩应要求学生指出每个共享可变状态由谁保护，而不是只展示通过编译。

动手实验：替换引擎

实现 EchoEngine:IMEEngine：输入任何 raw，都返回一个将 raw 大写的候选。把它注入教学控制器，证明 UI 和事件层不依赖中文算法。

思考与练习

1. 为什么 `EngineOutput` 适合结构体，而控制器适合类？
2. 为 `CandidateLookup` 设计一个网络实现是否违反 MyIME 的离线目标？架构上能否实现？
3. 找出模型中最可能在加入双拼后变化的字段，说明兼容策略。

本章小结

可教学、可测试的输入法需要把系统适配与算法核心分开。协议、值类型快照和内存替身共同形成可验证边界。

从小词库生成第一组中文候选

学习目标

建立“读音键到词条集合”的最小词库模型；理解精确匹配、前缀匹配、同音词和稳定排序；完成第一个能从 `nihao` 输出“你好”的引擎。

教学版词库可以从十行 TSV 开始：每行包含词语、带撇号的拼音和权重。例如：

你好	ni'hao	65535
拟好	ni'hao	12000
世界	shi'jie	65535
北京	bei'jing	65535
背景	bei'jing	48000
我	wo	65535
是	shi	65535
中国	zhong'guo	65535
人	ren	65535

Listing 6.1: 最小教学词库

将拼音中的撇号移除可得到查找键 `nihao`。最简单的查询是筛选所有键等于 `raw` 的词条，再按权重降序排列。即使只有这一步，也必须规定权重相同的稳定次序，否则相同输入在不同运行中可能改变候选位置。

6.1 前缀匹配与完整匹配

用户输入到一半时也需要候选。若 `entryKey.hasPrefix(raw)`，可以把词条作为补全候选，但必须施加补全惩罚；否则只输入 `b` 就可能让长专名淹没单字。一个基础评分可以写成：

$$S(w) = \alpha \cdot \frac{f(w)}{65535} - \lambda \cdot \Delta_{\text{comp}},$$

其中 $f(w)$ 是词库权重, Δ_{comp} 是候选比用户输入多出的音节数。下标 comp 是 completion 的缩写; 本书对具有语义的多字母下标统一使用直立 Roman 字体, 避免与多个变量的乘积混淆。

6.2 接口而非文件格式

真实项目用 SQLiteStore 实现查询接口。控制器和引擎不应该知道词库来自 TSV、SQLite 还是内存。以下协议片段体现了查询、用户加权和上下文加权三类职责:

```

1 import CSQLite
2 import Foundation
3
4 private let sqliteTransient = unsafeBitCast(-1, to: sqlite3_destructor_type.self)
5
6 public struct StoreEntry: Sendable {
7     public let id: Int64
8     public let word: String
9     public let pinyin: String
10    public let weight: Int
11
12    public init(id: Int64, word: String, pinyin: String, weight: Int) {
13        self.id = id
14        self.word = word
15        self.pinyin = pinyin
16        self.weight = weight
17    }
18 }
19
20 public protocol CandidateLookup: Sendable {
21     func lookup(pinyinKey: String, initials: String, disabledSourceMask: Int, limit: Int) -> [StoreEntry]
22     func lookupExact(pinyinKey: String, disabledSourceMask: Int, limit: Int) -> [StoreEntry]
23     func userBoost(word: String, pinyin: String) -> Double
24     func unigramBoost(word: String) -> Double
25     func bigramBoost(previous: String?, word: String) -> Double
26     func sequenceBoost(prefix: String, word: String) -> Double

```

```

27 func predictNext(after previous: String, limit: Int) -> [StoreEntry]
28 }
29
30 public extension CandidateLookup {
31     func lookupExact(pinyinKey: String, disabledSourceMask: Int, limit: Int) -> [
32         StoreEntry] {
33         lookup(pinyinKey: pinyinKey, initials: "", disabledSourceMask:
34             disabledSourceMask, limit: limit)
35         .filter { $0.pinyin.replacingOccurrences(of: "", with: "") ==
36             pinyinKey }
37     }
38
39     func unigramBoost(word: String) -> Double { 0 }
40     func sequenceBoost(prefix: String, word: String) -> Double { 0 }
41     func predictNext(after previous: String, limit: Int) -> [StoreEntry] { [] }
42 }

```

Listing 6.2: CandidateLookup 的查询抽象

动手实验：十词输入法

用十条词构建 MemoryStore，实现 `nihao`、`beijing` 和 `woshizhongguoren`。第一阶段只允许完整词匹配；第二阶段加入前缀匹配。比较只输入 `b` 时的候选变化。

6.3 候选的可解释性

课堂项目应为调试模式提供候选解释：词库权重、完整/前缀状态、消费长度、最终得分。商业 UI 可以隐藏这些信息，教学版必须显示。只有可解释，学生才能回答“为什么背景排在北京前面”，而不是不断调一个神秘常量。

思考与练习

1. 权重相同时，选择词语字典序还是数据库 ID 作为 tie-breaker，各有什么影响？
2. 为什么不能直接用百万词库开始调试？
3. 给出一种让“北京”在输入 `beij` 时出现、但不让它在 `b` 时过度靠前的惩罚函数。

本章小结

词库查询的第一原则是可控：小数据、明确键、稳定排序和可解释分数。规模应在正确性之后加入。

候选窗口：把算法结果变成可操作界面

学习目标

理解候选窗口与普通应用窗口的差异；掌握候选分页、高亮、数字选择和插入点定位；能够分离候选数据更新与窗口布局。

候选窗口是输入法最直接的反馈通道。它必须快速、克制，并且不能抢走客户端焦点。MyIME 使用 SwiftUI 描述候选列表，再由 AppKit 窗口控制器承载。此结构便于样式开发，但坐标、层级和激活策略仍属于 AppKit 世界。

```
1 import AppKit
2 import InputMethodKit
3 import SwiftUI
4 import IMKit
5
6 private struct CandidateListView: View {
7     let candidates: [Candidate]
8     let highlighted: Int
9     let prefs: EnginePrefs
10    let expanded: Bool
11    let onSelect: (Int) -> Void
12    let onExpand: () -> Void
13    let onCollapse: () -> Void
14
15    private let gridColumns = Array(repeating: GridItem(.flexible(), spacing: 0),
16                                     count: 6)
17
18    var body: some View {
19        VStack(spacing: 0) {
20            if expanded {
```

```

20         HStack(spacing: 4) {
21             LazyVGrid(columns: gridColumns, spacing: 0) {
22                 candidateViews
23             }
24             .frame(width: 620)
25             Divider().frame(height: 30)
26             controlButton("chevron.up", label: "收起候选", action:
onCollapse)
27         }
28     } else if prefs.orientation == .horizontal {
29         HStack(spacing: 2) {
30             candidateViews
31             Spacer(minLength: 4)
32             Divider().frame(height: 30)
33             controlButton("chevron.down", label: "展开候选", action:
onExpand)
34         }
35         .frame(minWidth: 620)
36     } else {
37         VStack(alignment: .leading, spacing: 4) {
38             candidateViews
39         }
40         .frame(minWidth: 320)
41     }
42 }
43 .font(.system(size: expanded ? min(max(prefs.fontSize - 2, 15), 19) : max(
prefs.fontSize, 13)))
44 .padding(expanded ? 6 : 7)
45 .background(.regularMaterial, in: RoundedRectangle(cornerRadius: 12))
46 .overlay {
47     RoundedRectangle(cornerRadius: 12)
48         .stroke(Color.primary.opacity(0.18), lineWidth: 1)
49 }
50 }
51
52 @ViewBuilder private var candidateViews: some View {
53     ForEach(Array(candidates.enumerated()), id: \.element.id) { index,
candidate in
54         Button { onSelect(index) } label: {
55             Text(candidateTitle(candidate.word, at: index))
56                 .lineLimit(1)
57                 .frame(maxWidth: expanded ? .infinity : nil, minHeight:
expanded ? 28 : 34, alignment: .leading)

```

```

58         .padding(.horizontal, expanded ? 7 : 9)
59         .padding(.vertical, 2)
60         .foregroundColor(index == highlighted ? Color.white : Color.
primary)
61         .background(index == highlighted ? Color.accentColor : Color.
clear, in: RoundedRectangle(cornerRadius: 8))
62         .contentShape(Rectangle())
63         .overlay(alignment: .bottom) {
64             if expanded, index / 6 < (candidates.count - 1) / 6 {
65                 Divider()
66             }
67         }
68     }
69     .buttonStyle(.plain)
70 }
71 }
72
73 private func controlButton(_ systemName: String, label: String, action:
@escaping () -> Void) -> some View {
74     Button(action: action) {
75         Image(systemName: systemName)
76         .font(.system(size: 15, weight: .semibold))
77         .foregroundColor(.secondary)
78         .frame(width: 30, height: 30)
79         .contentShape(Rectangle())
80     }
81     .buttonStyle(.plain)
82     .accessibilityLabel(label)
83 }
84
85 private func candidateTitle(_ word: String, at index: Int) -> String {
86     if expanded {
87         return index / 6 == highlighted / 6 ? "\(index % 6 + 1) \((word)" : "
\((word)"
88     }
89     return index < 9 ? "\(index + 1) \((word)" : "    \((word)"
90 }
91 }
92
93 @MainActor

```

Listing 7.1: SwiftUI 候选列表视图

7.1 候选窗口不应成为主窗口

如果候选窗成为 key window，原应用可能失去键盘焦点，下一次按键不再进入原输入会话。窗口通常使用无标题、不可激活、较高层级，并在组合结束后立即隐藏。课堂测试不能只看“窗口出现”，还要验证光标仍在 Notes 文档中。

7.2 定位坐标

客户端提供插入点附近的矩形，候选窗应放在其下方；若接近屏幕底部，则翻到上方；横向位置也需限制在可见屏幕内。设光标矩形为 R_c ，窗口大小为 (w, h) ，屏幕可见区域为 R_s ，基础位置可写成：

$$x = \min(\max(R_c.x, R_s.x), R_s.x + R_s.w - w),$$

$$y = \begin{cases} R_c.y - h - g, & \text{下方空间足够,} \\ R_c.y + R_c.h + g, & \text{否则放在上方.} \end{cases}$$

其中 g 是视觉间距。真正容易出错的并不是这个夹取公式，而是把属于不同空间的矩形直接相加。

7.2.1 macOS 中的三层坐标空间

候选窗定位至少涉及三种坐标：

1. **全局屏幕坐标**：NSScreen.frame、visibleFrame 和 NSWindow.frame 使用同一全局桌面空间，主显示器通常以左下角为原点；显示器排列在主屏左侧或下方时，次屏原点可能为负数。
2. **窗口/视图坐标**：AppKit 视图默认从左下角计量，但 isFlipped=true 的视图从左上角向下增长；SwiftUI 布局有自己的局部空间。局部坐标必须通过窗口转换后才能与屏幕矩形比较。
3. **输入客户端给出的行框**：attributes(forCharacterIndex:lineHeightRect angle:) 返回插入点所在文本行的屏幕矩形。MyIME 将该 lineRect 直接与 NSScreen.frame 比较，因此不能再调用一次 convertToScreen，否则会发生二次偏移。

在 MyIME 中，anchor.minY 是行框下边缘。优先位置为 $y_b = anchor.minY -$

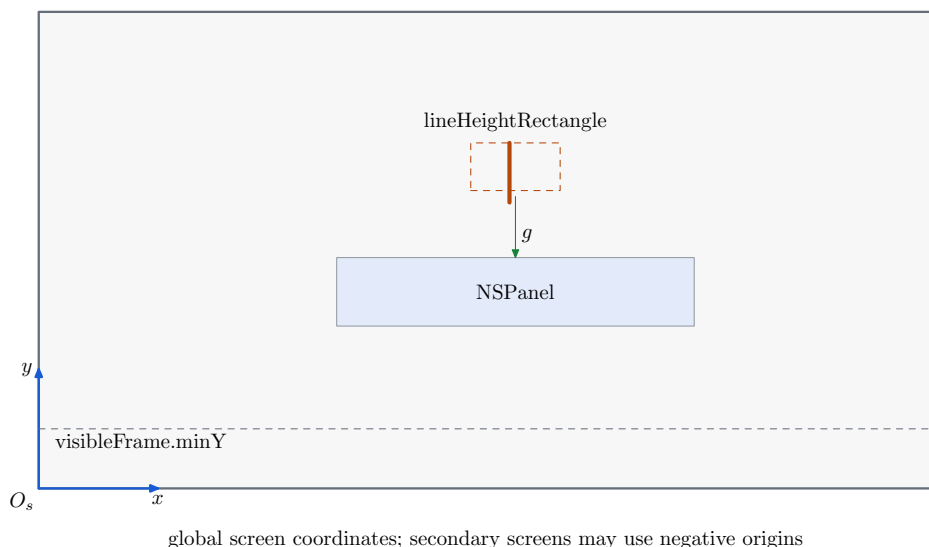


图 7.1: AppKit 全局屏幕坐标、客户端行框与候选 NSPanel 的关系

$h-g$; 如果 $y_b < visibleFrame.minY$, 候选窗翻到文本行上方: $y_a = anchor.maxY + g$ 。最后再同时夹取 x 与 y , 避免菜单栏、Dock、刘海安全区和多显示器边缘造成越界。

工程警告: lineHeightRectangle 并非永远可信

某些 WebKit/Electron 客户端会短暂返回零矩形、过期矩形或与当前 selectedRange 不一致的结果。MyIME 在空矩形时退回鼠标位置; 教学版还应记录一次脱敏诊断并隐藏窗口, 不能让错误坐标导致崩溃。测试时务必覆盖主屏左侧/下方的次显示器, 因为只测主屏会掩盖负坐标问题。

```

1 final class CandidateWindowController {
2     var onSelect: ((Int) -> Void)?
3     var onExpand: (() -> Void)?
4     var onCollapse: (() -> Void)?
5     private let panel: NSPanel
6     private var candidates: [Candidate] = []
7     private var highlighted = 0
8     private var preferences = EnginePrefs()
9     private var expanded = false
10
11     init() {
12         panel = NSPanel(

```

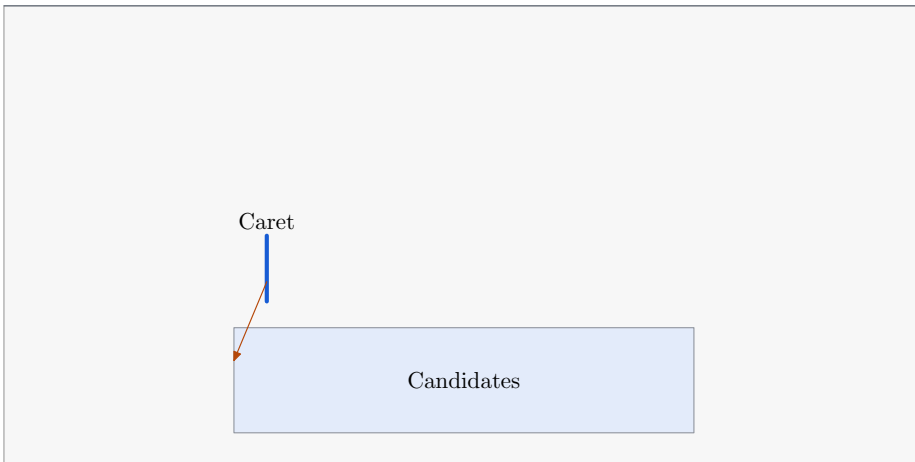


图 7.2: 候选窗口相对于插入点和可见屏幕的定位

```

13         contentRect: NSRect(x: 0, y: 0, width: 300, height: 48),
14         styleMask: [.borderless, .nonactivatingPanel],
15         backing: .buffered,
16         defer: true
17     )
18     panel.isOpaque = false
19     panel.backgroundColor = .clear
20     panel.hasShadow = true
21     panel.becomesKeyOnlyIfNeeded = true
22     panel.ignoresMouseEvents = false
23     panel.acceptsMouseMovedEvents = true
24     panel.level = .popUpMenu
25     panel.collectionBehavior = [.canJoinAllSpaces, .fullScreenAuxiliary]
26 }
27
28 func update(candidates: [Candidate], highlighted: Int, prefs: EnginePrefs,
29 expanded: Bool, client: IMKTextInput) {
30     updateCandidates(candidates, highlighted: highlighted, prefs: prefs,
31 expanded: expanded)
32     guard !candidates.isEmpty else { hide(); return }
33     var lineRect = NSRect.zero
34     let selectedLocation = client.selectedRange().location
35     let characterIndex = selectedLocation == NSNotFound ? 0 : selectedLocation
36     _ = client.attributes(forCharacterIndex: characterIndex,
lineHeightRectangle: &lineRect)
37     let fallback = NSEvent.mouseLocation
38     let anchor = lineRect.isEmpty ? NSRect(origin: fallback, size: .zero) :

```

```

lineRect
37     let screen = NSScreen.screens.first(where: { $0.frame.intersects(anchor) })
38     ?? NSScreen.screens.first(where: { $0.frame.contains(fallback) })
39     ?? NSScreen.main
40     let visibleFrame = screen?.visibleFrame ?? NSRect(origin: .zero, size:
panel.frame.size)
41     let belowY = anchor.minY - panel.frame.height - 4
42     let preferredY = belowY >= visibleFrame.minY ? belowY : anchor.maxY + 4
43     let origin = NSPoint(
44         x: min(max(anchor.minX, visibleFrame.minX), max(visibleFrame.minX,
visibleFrame.maxX - panel.frame.width)),
45         y: min(max(preferredY, visibleFrame.minY), max(visibleFrame.minY,
visibleFrame.maxY - panel.frame.height))
46     )
47     panel setFrameOrigin(origin)
48     panel.orderFrontRegardless()
49 }

50
51 func updateCandidates(_ candidates: [Candidate], highlighted: Int, prefs:
EnginePrefs, expanded: Bool) {
52     self.candidates = candidates
53     self.highlighted = highlighted
54     preferences = prefs
55     self.expanded = expanded
56     let hosting = NSHostingView(rootView: CandidateListView(
57         candidates: candidates,
58         highlighted: highlighted,
59         prefs: prefs,
60         expanded: expanded,
61         onSelect: { [weak self] in self?.onSelect?($0) },
62         onExpand: { [weak self] in self?.onExpand?() },
63         onCollapse: { [weak self] in self?.onCollapse?() }
64     ))
65     let fittingSize = hosting.fittingSize
66     let minimumHeight: CGFloat = expanded ? 70 : 50
67     let contentSize = NSSize(width: max(320, fittingSize.width), height: max(
minimumHeight, fittingSize.height))
68     hosting.frame.size = contentSize
69     panel.contentView = hosting
70     panel.setContentSize(contentSize)
71 }
72
73 func updateHighlight(_ highlighted: Int) {

```

```

74     updateCandidates(candidates, highlighted: highlighted, prefs: preferences,
75     expanded: expanded)
76 }
77 func hide() { panel.orderOut(nil) }
78 }
79
80 @MainActor
81 final class InputModeIndicatorController {
82     private let panel: NSPanel
83     private var displayGeneration = 0
84
85     init() {
86         panel = NSPanel(
87             contentRect: NSRect(x: 0, y: 0, width: 132, height: 40),
88             styleMask: [.borderless, .nonactivatingPanel],
89             backing: .buffered,
90             defer: true
91         )
92         panel.isOpaque = false
93         panel.backgroundColor = .clear
94         panel.hasShadow = true
95         panel.level = .popUpMenu

```

Listing 7.2: 候选窗口控制器的数据更新与展示

7.3 分页与数字映射

若每页大小为 p ，当前高亮索引为 i ，页首为

$$b = \left\lfloor \frac{i}{p} \right\rfloor p.$$

数字键 $d \in \{1, \dots, 9\}$ 映射到候选索引 $b + d - 1$ 。所有映射都必须在候选数组范围内检查。横向紧凑模式与展开模式可以使用不同页宽，但导航状态应保持一致。

7.4 VoiceOver 与无障碍高亮

颜色高亮只服务视觉用户。候选按钮还应向辅助技术提供“序号、候选文本、是否当前选中、可执行动作”。SwiftUI 可以为每行增加如下语义：

```

1 .accessibilityElement(children: .ignore)
2 .accessibilityLabel("候选 \((index + 1), \((candidate.word)")
3 .accessibilityValue(index == highlighted ? "当前候选" : "")
4 .accessibilityAddTraits(index == highlighted ? [.isSelected] : [])
5 .accessibilityHint("按回车或对应数字键上屏")

```

Listing 7.3: 候选按钮的 VoiceOver 语义规范

高亮由方向键改变时，可在节流后发布 `announcementRequested` 通知，朗读“第 2 项，天气”；连续按键时不要为每个中间状态堆积播报。Apple 将 `.isSelected` 定义为“当前被选中的无障碍元素”，公告通知需要提供 `announcement` 与 `priority`。¹

非激活 `NSPanel` 不应强制抢占 VoiceOver 焦点。验收时同时检查：鼠标点击候选、键盘导航、VoiceOver 朗读顺序、放大字体、提高对比度和“减少透明度”。辅助语义不能依赖颜色或仅显示在视觉标题中的空格对齐。

动手实验：边缘定位

在屏幕四角各放置一个文本框，输入拼音并截图候选窗。检查是否越界、遮挡插入点或跳到另一块显示器。把次显示器依次摆在主屏左侧和下方，记录负坐标。再改变系统缩放与候选字体大小重复实验，并用 VoiceOver 朗读前三个候选。

思考与练习

1. 为什么候选窗口不能通过普通 `NSApp.activate` 获得焦点？
2. 当候选不足一页时，按数字 9 应消费事件还是放行？
3. 设计一个对 VoiceOver 友好的候选行语义。

本章小结

候选窗口既是 UI，也是输入会话的一部分。正确性包括不抢焦点、坐标不越界、数字映射稳定以及组合结束后立即消失。

¹ 参见 Apple Developer Documentation: <https://developer.apple.com/documentation/swiftui/accessibilitytraits/isselected> 与 <https://developer.apple.com/documentation/appkit/nsaccessibilityannouncementrequestednotification>。

偏好设置、繁简输出与可观察性

学习目标

理解运行时偏好如何从 UI 传播到引擎；区分词库语言与输出转换；建立最小日志与状态菜单，使用户能够判断输入源是否真的连接。

输入法设置不是独立 App 的静态配置。候选数量、方向、字体、模糊音和繁体输出都可能在下一次按键立即影响结果。MyIME 把偏好编码为 `EnginePrefs`，写入共享 `UserDefaults` suite，并发布本进程通知。

```
1 import AppKit
2 import IMKit
3 import SwiftUI
4 import UniformTypeIdentifiers
5
6 struct SettingsView: View {
7     @State private var preferences = PreferencesStore.load()
8     @State private var message = ""
9
10    var body: some View {
11        TabView {
12            general
13                .tabItem { Label("通用", systemImage: "gearshape") }
14            fuzzy
15                .tabItem { Label("模糊音", systemImage: "waveform") }
16            appearance
17                .tabItem { Label("皮肤", systemImage: "paintpalette") }
18            dictionaries
19                .tabItem { Label("词库", systemImage: "books.vertical") }
20        }
21        .padding(16)
22        .onChange(of: preferences) { value in PreferencesStore.save(value) }
23    }
```

```

24
25 private var general: some View {
26     Form {
27         Picker("每页候选数", selection: $preferences.pageSize) {
28             ForEach(3...9, id: \.self) { Text("\( $0)").tag( $0) }
29         }
30         Toggle("输出繁体（繁简共享 MyIME 词库）", isOn: $preferences.
outputTraditional)
31         LabeledContent("默认上屏键", value: "空格")
32         LabeledContent("默认翻页键", value: "- / = 或 , / .")
33         LabeledContent("中英文切换", value: "单击 Shift")
34     }
35     .formStyle(.grouped)
36 }
37
38 private var fuzzy: some View {
39     Form {
40         fuzzyToggle("z = zh", keyPath: \.zZh)
41         fuzzyToggle("c = ch", keyPath: \.cCh)
42         fuzzyToggle("s = sh", keyPath: \.sSh)
43         fuzzyToggle("n = l", keyPath: \.nL)
44         fuzzyToggle("f = h", keyPath: \.fH)
45         fuzzyToggle("r = l", keyPath: \.rL)
46         fuzzyToggle("an = ang", keyPath: \.anAng)
47         fuzzyToggle("en = eng", keyPath: \.enEng)
48         fuzzyToggle("in = ing", keyPath: \.inIng)
49         fuzzyToggle("ian = iang", keyPath: \.ianIang)
50         fuzzyToggle("uan = uang", keyPath: \.uanUang)
51     }
52     .formStyle(.grouped)
53 }
54
55 private var appearance: some View {
56     Form {
57         Slider(value: $preferences.fontSize, in: 13...28, step: 1) { Text("候选
字号") }
58         LabeledContent("字号", value: "\(Int(preferences.fontSize)) pt")
59         Toggle("跟随系统深浅色", isOn: $preferences.followsSystemAppearance)
60         Picker("候选排列", selection: $preferences.orientation) {
61             Text("横向").tag(CandidateOrientation.horizontal)
62             Text("纵向").tag(CandidateOrientation.vertical)
63         }
64         .pickerStyle(.segmented)

```

```

65     }
66     .formStyle(.grouped)
67 }
68
69 private var dictionaries: some View {
70     Form {
71         Section("系统词库") {
72             let metadata = IMEnvironment.shared.systemStore?.metadata ?? [:]
73             LabeledContent("Schema", value: metadata["schema_version"] ?? "不可
74             用")
75             LabeledContent("词条数", value: metadata["entry_count"] ?? "0")
76             LabeledContent("Build hash", value: metadata["build_hash"] ?? "—")
77             sourceToggle("雾凇拼音", bit: 0)
78             unavailableSource("CustomPinyin (等待再分发授权)")
79             sourceToggle("中文维基", bit: 2)
80             sourceToggle("景行词库", bit: 3)
81             unavailableSource("专业词库 (等待再分发授权)")
82             sourceToggle("搜狗分类整理", bit: 5)
83             sourceToggle("清华开放中文词库 (THUOCL)", bit: 10)
84             unavailableSource("程序员术语 (等待再分发授权)")
85             Button("校验并替换 system.sqlite...", action: replaceDictionary)
86         }
87         Section("用户词组与学习") {
88             Button("导入 TSV...", action: importUserWords)
89             Button("导出 TSV...", action: exportUserWords)
90             Button("清空使用频率", role: .destructive) { IMEnvironment.shared.
91             userStore?.clearLearning() }
92             if !message.isEmpty { Text(message).foregroundColor(.secondary) }
93         }
94     }
95     .formStyle(.grouped)
96 }
97
98 private func fuzzyToggle(_ title: String, keyPath: WritableKeyPath<FuzzyRules,
99 Bool>) -> some View {
100     Toggle(title, isOn: Binding(
101         get: { preferences.fuzzy[keyPath: keyPath] },
102         set: { preferences.fuzzy[keyPath: keyPath] = $0 }
103     ))
104 }
105
106 private func sourceToggle(_ title: String, bit: Int) -> some View {
107     let mask = 1 << bit

```



```

105     return Toggle(title, isOn: Binding(
106         get: { preferences.disabledSourceMask & mask == 0 },
107         set: { enabled in
108             if enabled { preferences.disabledSourceMask &= ~mask }
109             else { preferences.disabledSourceMask |= mask }
110         }
111     ))
112 }
113
114 private func unavailableSource(_ title: String) -> some View {
115     Toggle(title, isOn: .constant(false))
116         .disabled(true)
117 }
118
119 private func importUserWords() {
120     let panel = NSOpenPanel()

```

Listing 8.1: 设置界面中的候选、外观与模糊音选项

8.1 设置传播的三个问题

每个选项都应回答：谁写入、谁读取、何时生效。若设置窗口与输入法服务最终运行在不同进程，仅使用进程内 NotificationCenter 就不够；如果它们在同一进程，共享通知可以即时刷新。课堂设计文档必须明确进程边界。

8.2 繁体输出是映射，不是另一套词库

MyIME 默认使用简体词库和学习记录，繁体选项在显示与提交阶段转换文本。转换必须优先匹配长词，否则逐字转换会把“头发”的“发”和“开发”的“发”混为一谈。

```

1  import Foundation
2
3  public struct TraditionalConverter: Sendable {
4      public static let empty = TraditionalConverter(mappings: [:])
5
6      private let mappings: [String: String]
7      private let maximumKeyLength: Int
8
9      public init(mappings: [String: String]) {
10         self.mappings = mappings

```

```

11     maximumKeyLength = mappings.keys.map(\.count).max() ?? 1
12 }
13
14 public var isAvailable: Bool { !mappings.isEmpty }
15
16 public func convert(_ text: String) -> String {
17     guard !text.isEmpty, !mappings.isEmpty else { return text }
18     let characters = Array(text)
19     var result = ""
20     var index = 0
21     while index < characters.count {
22         let maxLength = min(maximumKeyLength, characters.count - index)
23         var matched = false
24         for length in stride(from: maxLength, through: 1, by: -1) {
25             let key = String(characters[index.. $(index + length)$ ])
26             guard let replacement = mappings[key] else { continue }
27             result += replacement
28             index += length
29             matched = true
30             break
31         }
32         if !matched {
33             result.append(characters[index])
34             index += 1
35         }
36     }
37     return result
38 }
39 }

```

Listing 8.2: 最长优先的繁体转换器

若映射集合为 M ，从位置 i 开始选择最长可匹配键 k ，然后输出 $M(k)$ 并前进 $|k|$ 。该算法是贪心的；成立前提是映射表以词组覆盖字符歧义。教学时可用“头发后台开发”验证短键不会提前吞掉长词。

8.3 状态菜单是诊断界面

菜单栏显示“中”不应是装饰，而应反映系统真实输入源状态。MyIME 在输入源变化通知到来时重新查询 TIS 状态，并显示“中”或“A”。这使普通用户能区分“应用启动了”和“MyIME 已选中”。

```

1 private let logger = Logger(subsystem: "fudan.miniS.MyIME", category: "
bootstrap")
2
3 init(server: IMKServer?) {
4     self.server = server
5 }
6
7 func applicationWillFinishLaunching(_ notification: Notification) {
8     shouldStartServer = SelfInstaller.prepareInstalledCopy(logger: logger)
9 }
10
11 func applicationDidFinishLaunching(_ notification: Notification) {
12     guard shouldStartServer else { return }
13     guard server != nil else {
14         logger.error("已安装输入法未能建立 IMK 服务")
15         return
16     }
17     _ = IMEnvironment.shared
18     let registered = SelfInstaller.ensureRegistered(logger: logger)
19     SelfInstaller.removeLegacyLoginRegistration(logger: logger)
20     installStatusMenu()
21     DistributedNotificationCenter.default().addObserver(
22         self,
23         selector: #selector(inputSourceChanged(_:)),
24         name: Notification.Name(kTISNotifySelectedKeyboardInputSourceChanged as
String),
25         object: nil,
26         suspensionBehavior: .deliverImmediately
27     )
28     updateStatusMenu()
29     if SelfInstaller.isPostInstallLaunch {
30         let selected = registered && SelfInstaller.isCurrentInputSource
31         updateStatusMenu()
32         presentPostInstallResult(selected: selected)
33     }
34     if let message = IMEnvironment.shared.resourceError {
35         logger.error("\(message, privacy: .public)")
36     }
37 }
38
39 func applicationShouldTerminateAfterLastWindowClosed(_ sender: NSApplication)
-> Bool { false }

```

```

40
41 func applicationWillTerminate(_ notification: Notification) {
42     DistributedNotificationCenter.default().removeObserver(self)
43 }
44
45 @objc private func inputSourceChanged(_ notification: Notification) {
46     updateStatusMenu()
47     MyIMEInputController.finalizeStrandedCompositionIfNeeded()
48 }
49
50 private func updateStatusMenu() {
51     // Keep this recovery entry visible even when the system input menu has
52     // lost MyIME.
53     statusItem?.isVisible = true
54     let isCurrent = SelfInstaller.isCurrentInputSource
55     // Show real TIS state — a permanent "中" misleads when Notes has fallen
56     // back to ABC.
57     statusItem?.button?.title = isCurrent ? "中" : "A"
58     statusItem?.button?.toolTip = isCurrent ? "MyIME 已选中" : "MyIME 未选中 (当前为其他输入源)"
59     switchInputSourceItem?.title = isCurrent
60         ? "MyIME 已连接"

```

Listing 8.3: 输入源变化观察与状态菜单更新

动手实验：设置即时生效

组合 ci 时切换 c/ch 模糊音，观察候选变化；调整字体和横纵方向；启用繁体后输入“头发后台开发”。记录哪些设置立即生效，哪些需要重新建立输入会话。

思考与练习

1. 为什么繁简转换应该在候选显示和最终上屏两个位置保持一致？
2. 如果用户在组合过程中改变候选页大小，高亮索引应如何处理？
3. 设计一个完全本地的诊断面板，列出五项最有价值的信息。

本章小结

偏好设置必须有清晰的数据流和生效时机。繁体输出是显示层转换，状态菜单则是帮助用户理解真实系统状态的可观察性工具。

第一次端到端验收

学习目标

把编译、安装、注册、选择、输入和恢复组成一份可重复验收脚本；学会记录预期、实际和证据；完成初级篇里程碑。

到本章为止，学生已经具备最小输入服务、组合状态、词库候选、候选窗和偏好设置。现在必须把它们放回真实系统。端到端验收不是“随便打几句话”，而是固定环境、固定步骤和固定判定标准。

9.1 基础验收矩阵

场景	操作	通过标准
空闲英文	切换 ABC，输入 hello	原样出现，无候选窗
中文整词	切换 MyIME，输入 niha	上屏“你好”，组合清空
退格	输入 niha 后退格	raw 变为 nih ，候选刷新
取消	输入拼音后按 Esc	文档无新增，候选关闭
数字选词	输入同音拼音后按 2	第二候选上屏一次
应用切换	组合中切到另一应用	无旧组合串入新应用
进程恢复	结 束 进 程， ABC→MyIME	服务重新启动并可输入

9.2 目标应用分层

第一层使用 Notes 或 TextEdit，代表系统原生文本控件。第二层使用 Safari 或 Chrome 的网页输入框；第三层使用 Electron 编辑器；第四层使用 WPS、微信等大

型应用。若第一层失败，不要继续用更多应用制造噪声。若只有第三层失败，问题更可能在组合协议兼容，而不是词库。

9.3 记录证据

每次失败至少记录：系统版本、芯片、应用与版本、当前输入源、输入序列、控制器日志、文档最终文本、是否显示候选窗。对于“偶尔失效”，再增加重复次数和触发前操作。没有输入序列的“不能用”无法形成回归测试。

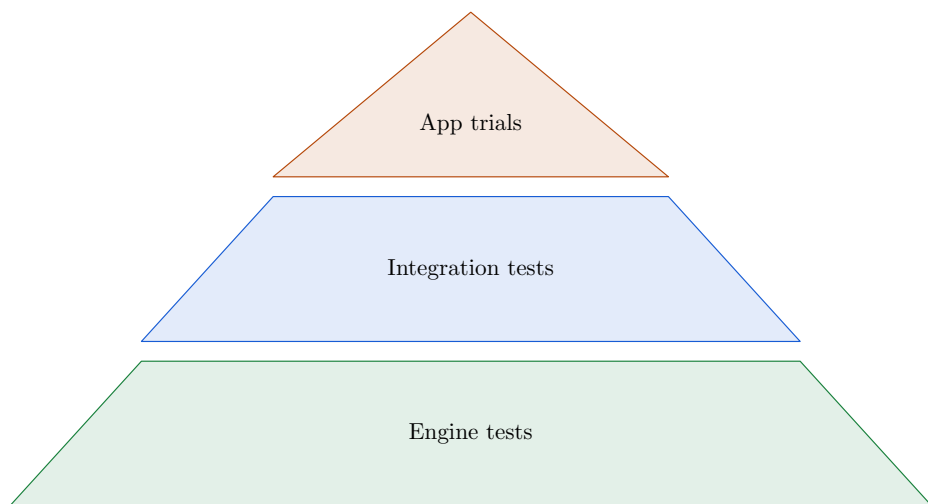


图 9.1: 输入法测试金字塔：大量纯逻辑测试、适量集成测试、少量跨应用人工验收

动手实验：初级篇里程碑

在 Notes 与一个浏览器中完成上述矩阵。提交一段不超过 3 分钟的演示录像，以及一份包含失败尝试的测试记录。允许存在缺陷，但不允许隐去缺陷。

思考与练习

1. 为什么录像不能替代结构化测试记录？
2. Notes 成功而 Chrome 失败时，应如何缩小范围？
3. 设计一条覆盖“部分候选消费 + 继续组合”的端到端用例。

本章小结

初级篇的终点不是候选窗出现，而是形成一套可重复的端到端证据。之后的高级算法改动都必须重新通过这条基线。

第二部分

高级篇：从候选词到语言模型

拼音切分：把字符串变成图

学习目标

能够把无分隔拼音建模为有向无环图；实现全拼、撇号硬边界和多路径切分；理解消费长度、路径排序与截断策略。

词库需要音节边界，用户却通常输入连续字母。**xian** 既可能是 **xian** (先)，也可能是 **xi'an** (西安)；**fang'an** 中撇号明确禁止跨界。切分器的任务不是立即决定汉字，而是产生若干可能的音节路径。

令归一化字符串为 $x = x_0x_1\cdots x_{n-1}$ 。建立顶点集合 $V = \{0, 1, \dots, n\}$ 。如果子串 $x[i:j]$ 是合法拼音音节，就添加边 (i, j) 。从 0 到 n 的每一条路径对应一种完整切分。

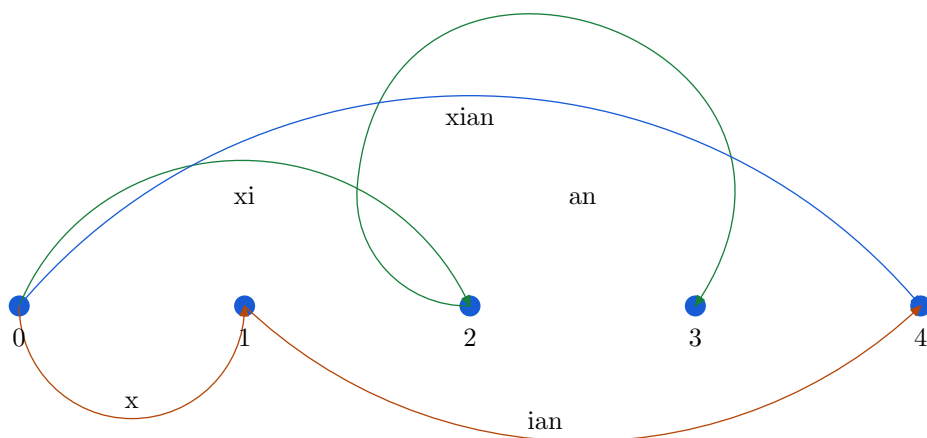


图 10.1: **xian** 的若干切分路径；顶点表示字母位置，边表示合法音节

10.1 路径不仅包含音节

MyIME 的 `SegmentationPath` 还记录消费长度、模糊匹配数、简拼数量和错拼数量。这样排序器无需重新分析路径来源，就能对非精确路径施加惩罚。

```

1 import Foundation
2
3 public struct SegmentationPath: Equatable, Sendable {
4     public let syllables: [String]
5     public let consumedLength: Int
6     public let fuzzyMatches: Int
7     public let partialCount: Int
8     public let typoCount: Int
9
10    public init(syllables: [String], consumedLength: Int, fuzzyMatches: Int,
11               partialCount: Int, typoCount: Int = 0) {
12        self.syllables = syllables
13        self.consumedLength = consumedLength
14        self.fuzzyMatches = fuzzyMatches
15        self.partialCount = partialCount
16        self.typoCount = typoCount
17    }
18
19    public var preedit: String { syllables.joined(separator: "'") }
20 }
21
22 public struct Segmenter: Sendable {
23     private let syllables: Set<String>
24     private let initials = Set("bpmfdtnlgkhjqxzcsryw".map(String.init))
25     /// Formal full finals users occasionally type; normalized penalty-free because
26     /// the raw spelling is never a valid syllable on its own.
27     private static let fullFormFinals = [("uen", "un"), ("uei", "ui"), ("iou", "iu"
28     )]
29
30    public init(syllables: Set<String> = PinyinTable.syllables) {
31        self.syllables = syllables
32    }

```

Listing 10.1: 拼音路径的数据结构

10.2 搜索与限制

完整枚举可能指数增长。教学版可以用深度优先搜索理解所有路径，工程版必须限制：单音节最大长度、每个输入返回路径数、搜索中间结果数和总耗时。MyIME 对每个片段递归尝试最长到最短的 token，并在结果过多时提前停止。

```

1
2 public func segment(_ raw: String, fuzzy: FuzzyRules = FuzzyRules(), limit: Int
   = 8) -> [SegmentationPath] {
3     let normalized = raw.lowercased().filter { $0.isASCII && ($0.isLetter || $0
   == "'')}
4     guard !normalized.isEmpty else { return [] }
5     let pieces = normalized.split(separator: "'", omittingEmptySubsequences:
   false).map(String.init)
6     var combined = [SegmentationPath(syllables: [], consumedLength: 0,
   fuzzyMatches: 0, partialCount: 0)]
7
8     for piece in pieces where !piece.isEmpty {
9         let paths = segmentPiece(piece, fuzzy: fuzzy, limit: limit)
10        guard !paths.isEmpty else { continue }
11        combined = combined.flatMap { prefix in
12            paths.map { path in
13                SegmentationPath(
14                    syllables: prefix.syllables + path.syllables,
15                    consumedLength: prefix.consumedLength + path.consumedLength
16                    ,
17                    fuzzyMatches: prefix.fuzzyMatches + path.fuzzyMatches,
18                    partialCount: prefix.partialCount + path.partialCount,
19                    typoCount: prefix.typoCount + path.typoCount
20                )
21            }
22        }.sorted(by: Self.precedes)
23        .prefix(limit)
24        .map { $0 }
25    }
26    return combined.filter { !$0.syllables.isEmpty }.sorted(by: Self.precedes)
27 }
28
29 private func segmentPiece(_ text: String, fuzzy: FuzzyRules, limit: Int) -> [
   SegmentationPath] {
30     let chars = Array(text)

```

```

31     var results: [SegmentationPath] = []
32
33     func walk(_ index: Int, _ path: [String], _ fuzzyCount: Int, _ partialCount
: Int, _ typoCount: Int) {
34         if results.count >= limit * 8 { return }
35         if index == chars.count {
36             results.append(SegmentationPath(
37                 syllables: path,
38                 consumedLength: index,
39                 fuzzyMatches: fuzzyCount,
40                 partialCount: partialCount,
41                 typoCount: typoCount
42             ))
43             return
44         }
45         var found = false
46         let maxEnd = min(chars.count, index + 6)
47         if index < maxEnd {
48             for end in stride(from: maxEnd, through: index + 1, by: -1) {
49                 let token = String(chars[index..

```

```

67         }
68     }
69     let initial = String(chars[index])
70     if initials.contains(initial) {
71         found = true
72         walk(index + 1, path + [initial], fuzzyCount, partialCount + 1,
typoCount)
73     }
74     if !found, !path.isEmpty {
75         results.append(SegmentationPath(
76             syllables: path,
77             consumedLength: index,
78             fuzzyMatches: fuzzyCount,
79             partialCount: partialCount + 1,
80             typoCount: typoCount
81         ))
82     }
83 }
84
85 walk(0, [], 0, 0, 0)
86 return Array(results.sorted(by: Self.precedes).prefix(limit))

```

Listing 10.2: 切分主流程与受限递归搜索

如果某条路径只消费了前缀，它可以用于“用户还在输入”的候选，但绝不能压过完整消费路径。因此路径排序首先比较 `consumedLength`，再比较简拼、错拼、模糊音和音节数。

可以把路径代价写成：

$$C(P) = a(n - \ell_P) + bN_{\text{partial}} + cN_{\text{typo}} + dN_{\text{fuzzy}},$$

其中 ℓ_P 是消费长度，第一项应具有最高优先级，或者在比较器中直接先比较消费长度。

动手实验：枚举歧义

对 `xian`、`fangan`、`chang'an` 和 `jintian` 输出全部切分路径。分别启用和禁用撇号硬边界，说明哪些边被删除。

思考与练习

1. 用邻接表写出 `xian` 的切分图。
2. 为什么“总是取最长音节”不能正确处理所有中文拼音？
3. 给出一个输入，使路径数量迅速增加，并提出两个剪枝办法。
4. 将递归搜索改写为动态规划，比较两种实现的可读性。

本章小结

拼音切分是路径生成，不是最终选词。用 DAG 表示合法音节，用路径元数据保存容错来源，再以消费完整性和代价控制候选空间。

简拼、混拼、模糊音与错拼容忍

学习目标

区分缩写与模糊匹配；理解容错召回和排序惩罚必须同时设计；能为相邻字母颠倒、正式韵母写法和模糊音建立测试。

容错不是“把更多东西都当成正确”。它包含两个阶段：**召回**阶段生成更多可能路径，**排序**阶段让精确路径在通常情况下领先。只有召回没有惩罚，候选会被噪声淹没；只有惩罚没有召回，用户错误输入时根本看不到正确词。

11.1 简拼与混拼

如果单个声母字母被允许作为占位音节，bj 可表示 b'j，匹配“北京”；beij 可以切为 bei'j，形成尾部简拼。路径记录 `partialCount`，评分时按数量扣分。

纯简拼查询通常需要专门的首字母索引。MyIME 只在路径中的每个音节都是单字母时生成 `initials` 查询键，防止完整拼音 `xiexie` 被错误拿去匹配首字母 `xx` 的“学习”等词。

```
1 public func update(_ raw: String, prefs: EnginePrefs) -> EngineOutput {
2     let normalized = raw.lowercased().filter { $0.isASCII && ($0.isLetter || $0
    == "'')} }
3     guard !normalized.isEmpty else { return EngineOutput(preedit: "",
    candidates: [], hasMore: false, raw: "") }
4     let paths = segmenter.segment(normalized, fuzzy: prefs.fuzzy, limit: 4)
5     guard let firstPath = paths.first else {
6         return EngineOutput(preedit: normalized, candidates: [], hasMore: false
    , raw: normalized)
7     }
8
9     let history = stateLock.withLock { committedHistory }
10    let anchors = contextAnchors(last: history.last, beforeLast: history.
```



```

beforeLast)
11     let inputLength = max(1, normalized.filter { $0 != "" }.count)
12     let userScale = prefs.userWeight / 0.35
13     let fuzzyScale = prefs.fuzzyPenalty / 0.08
14     let partialScale = prefs.partialPenalty / 0.1
15     var best: [String: Candidate] = [:]
16     let wordDeadline = ContinuousClock.now.advanced(by: .milliseconds(40))
17     for path in paths {
18         if ContinuousClock.now >= wordDeadline { break }
19         let pinyinKey = path.syllables.joined()
20         // Initials matching is only meaningful when every typed letter stands
for
21         // one syllable (pure abbreviation). Full-pinyin input must not recall
22         // abbreviation homonyms (e.g. "xiexie" → initials "xx" → 学习).
23         let isAbbreviation = path.syllables.allSatisfy { $0.count == 1 }
24         let initials = isAbbreviation ? path.syllables.joined() : ""
25         var matches = (store.lookup(pinyinKey: pinyinKey, initials: initials,
disabledSourceMask: prefs.disabledSourceMask, limit: 100)
26             + (userStore?.lookup(pinyinKey: pinyinKey, initials: initials,
limit: 100) ?? []))
27         .map { ($0, path.consumedLength, 0) }
28         if path.syllables.count > 1, let first = path.syllables.first {
29             let firstInitials = first.count == 1 ? first : ""
30             let fallback = (store.lookup(pinyinKey: first, initials:
firstInitials, disabledSourceMask: prefs.disabledSourceMask, limit: 30)
31                 + (userStore?.lookup(pinyinKey: first, initials: firstInitials,
limit: 30) ?? []))
32                 .filter { $0.pinyin.split(separator: "").count == 1 }
33                 .map { ($0, first.count, 1) }
34             matches += fallback
35         }
36         let pathPenalty = Scale.fuzzyPenalty * fuzzyScale * Double(path.
fuzzyMatches)
37         + Scale.typoPenalty * Double(path.typoCount)
38         + Scale.partialPenalty * partialScale * Double(path.partialCount)
39         for (entry, consumedLength, fallbackPenalty) in matches {
40             let pinyin = entry.pinyin.split(separator: "").map(String.init)

```

Listing 11.1: 完整拼音与纯简拼查询的区分

11.2 模糊音是可配置关系

模糊音可以表示为一组双向替换关系，例如 $z \leftrightarrow zh$ 、 $n \leftrightarrow l$ 、 $an \leftrightarrow ang$ 。不同用户需要的规则不同，因此默认全部关闭，并把命中次数写入路径。

```

1      }
2
3      private func fuzzyEquivalents(of token: String, rules: FuzzyRules) -> Set<
String> {
4          let pairs: [(Bool, String, String)] = [
5              (rules.zZh, "z", "zh"), (rules.cCh, "c", "ch"), (rules.sSh, "s", "sh"),
6              (rules.nL, "n", "l"), (rules.fH, "f", "h"), (rules.rL, "r", "l"),
7              (rules.anAng, "an", "ang"), (rules.enEng, "en", "eng"),
8              (rules.inIng, "in", "ing"), (rules.ianIang, "ian", "iang"),
9              (rules.uanUang, "uan", "uang"),
10         ]
11         var values = Set<String>()
12         for (enabled, lhs, rhs) in pairs where enabled {
13             if token == lhs { values.insert(rhs) }
14             if token == rhs { values.insert(lhs) }
15             if token.hasPrefix(lhs) { values.insert(rhs + token.dropFirst(lhs.count
)) }
16             if token.hasPrefix(rhs) { values.insert(lhs + token.dropFirst(rhs.count
)) }
17             if token.hasSuffix(lhs) { values.insert(String(token.dropLast(lhs.count
)) + rhs) }
18             if token.hasSuffix(rhs) { values.insert(String(token.dropLast(rhs.count
)) + lhs) }
19         }
20         values.remove(token)
21         return values
22     }
23
24     private static func precedes(_ lhs: SegmentationPath, _ rhs: SegmentationPath)
-> Bool {
25         // Consuming more of the input always wins: a truncated parse must never
26         // shadow a complete one (e.g. [wo,xian,za]+junk vs [wo,xian,zai,hen,m]).
27         if lhs.consumedLength != rhs.consumedLength { return lhs.consumedLength >
rhs.consumedLength }
28         if lhs.partialCount != rhs.partialCount { return lhs.partialCount < rhs.
partialCount }
29         if lhs.typoCount != rhs.typoCount { return lhs.typoCount < rhs.typoCount }
30         if lhs.fuzzyMatches != rhs.fuzzyMatches { return lhs.fuzzyMatches < rhs.

```

```

fuzzyMatches }
31     if lhs.syllables.count != rhs.syllables.count { return lhs.syllables.count
    < rhs.syllables.count }
32     return lhs.preedit < rhs.preedit
33 }
34 }

```

Listing 11.2: 模糊音等价集合的生成

11.3 相邻字母颠倒

用户可能把 hao 输入为 hoa, 把 zhong 输入为 zhogn。MyIME 枚举 token 中相邻字符的一次交换, 只接受交换后成为合法音节的结果, 并限制整条路径最多使用一次错拼修正。若无限允许交换, 搜索空间和误召回都会失控。

```

1
2 private func normalizedFullForm(of token: String) -> String? {
3     for (fullForm, short) in Self.fullFormFinals where token.hasSuffix(fullForm
4     ) {
5         let normalized = String(token.dropLast(fullForm.count)) + short
6         if syllables.contains(normalized) { return normalized }
7     }
8     return nil
9 }
10
11 private func transposedVariants(of token: String) -> [String] {
12     var variants: [String] = []
13     var chars = Array(token)
14     for index in 0.. $(chars.count - 1)$  where chars[index] != chars[index + 1] {
15         chars.swapAt(index, index + 1)
16         let variant = String(chars)
17         if syllables.contains(variant), !variants.contains(variant) { variants.
18         append(variant) }
19         chars.swapAt(index, index + 1)
20     }
21     return variants
22 }

```

Listing 11.3: 一次相邻字符颠倒修正

设精确候选基础分为 S_0 , 一种简单容错分数为

$$S = S_0 - \lambda_f N_f - \lambda_p N_p - \lambda_t N_t,$$

其中 N_f, N_p, N_t 分别是模糊音、简拼和错拼次数。通常应令 $\lambda_t > \lambda_f > \lambda_p$ ，但最终顺序要由验证集决定。

动手实验：容错边界

为 bj、beij、ci/chi、hoa/hao、zhogn/zhong 建立表格，记录关闭与开启规则后的 top-3。检查精确输入的首选是否被容错候选改变。

思考与练习

1. 为什么模糊音不适合直接修改用户 raw?
2. 设计一种允许两次错拼、但不导致组合爆炸的策略。
3. 简拼路径应该比不完整全拼路径优先吗？给出产品场景后作答。

本章小结

容错是一项“召回 + 惩罚”的成对设计。每条非精确路径都应携带来源和次数，使排序可解释、测试可回归。

SQLite 词库：模式、索引与只读运行时

学习目标

理解系统词库与用户词库分离的原因；能根据查询模式设计 SQLite 索引；掌握完整性检查、只读打开和故障降级。

百万级词库不适合每次启动全部载入内存。SQLite 提供紧凑存储、索引查询、事务和跨架构一致性。MyIME 把系统词库打包在应用资源中，以只读方式使用；用户学习写入用户目录的独立数据库。更新应用时可以替换系统库，而不覆盖个人数据。

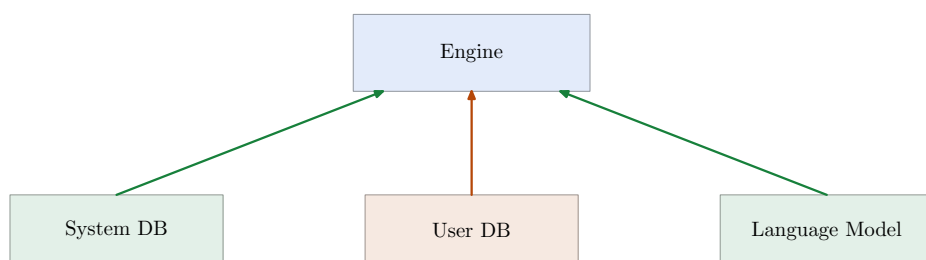


图 12.1: 只读系统词库、可写用户词库和统计语言模型共同服务引擎

12.1 查询模式决定索引

典型查询包括：

- 根据无分隔拼音键精确或前缀查词；
- 根据纯简拼 initials 查词；
- 查询词语 unigram 权重；

- 根据前词查询 bigram;
- 根据音节区间查询词格边。

索引应围绕这些访问路径，而不是“所有列都建索引”。前缀查询只有在模式与排序规则允许使用 B-tree 时才会命中索引；课堂实验应使用 `EXPLAINQUERYPLAN` 验证，而不是凭直觉。

```

1 public final class SQLiteStore: CandidateLookup, @unchecked Sendable {
2     private var database: OpaquePointer?
3     private let lock = NSLock()
4     private var languageModel: [UInt64: Float] = [:]
5     private var wordUnigrams: [UInt64: Float] = [:]
6     private var wordBigrams: [UInt64: Float] = [:]
7     private var predictions: [UInt64: [StoreEntry]] = [:]
8     private var exactLookupCache: [String: [StoreEntry]] = [:]
9     public let metadata: [String: String]
10    public let traditionalConverter: TraditionalConverter
11
12    public init?(path: String) {
13        var handle: OpaquePointer?
14        guard sqlite3_open_v2(path, &handle, SQLITE_OPEN_READONLY |
15        SQLITE_OPEN_NOMUTEX, nil) == SQLITE_OK,
16            let handle else {
17                if let handle { sqlite3_close(handle) }
18                return nil
19            }
20            database = handle
21            sqlite3_exec(handle, "PRAGMA query_only=ON; PRAGMA busy_timeout=100;", nil,
22            nil, nil)
23            metadata = Self.readMetadata(handle)
24            guard ["1", "2", "3", "4"].contains(metadata["schema_version"]) else {
25                sqlite3_close(handle)
26                database = nil
27                return nil
28            }
29            languageModel = Self.readLanguageModel(handle)
30            let wordModel = Self.readWordModel(handle)
31            wordUnigrams = wordModel.unigrams
32            wordBigrams = wordModel.bigrams
33            predictions = wordModel.predictions
34            traditionalConverter = Self.readTraditionalConverter(handle)
35    }

```

```

34
35 deinit {
36     if let database { sqlite3_close(database) }
37 }
38
39 public func integrityCheck() -> Bool {
40     lock.withLock {
41         guard let database else { return false }
42         var statement: OpaquePointer?
43         defer { sqlite3_finalize(statement) }
44         guard sqlite3_prepare_v2(database, "PRAGMA integrity_check", -1, &
statement, nil) == SQLITE_OK,
45             sqlite3_step(statement) == SQLITE_ROW,
46             let text = sqlite3_column_text(statement, 0) else { return false
}
47         return String(cString: text) == "ok"
48     }
49 }
50
51 public func lookup(pinyinKey: String, initials: String, disabledSourceMask: Int
, limit: Int) -> [StoreEntry] {
52     lock.withLock {
53         guard let database else { return [] }
54         let sql = """
55 SELECT id, word, pinyin, weight FROM entries
56 WHERE (source_mask & ?1) != 0
57     AND (py_key = ?2 OR py_key GLOB ?3 OR initials = ?4)
58 ORDER BY CASE WHEN py_key = ?2 THEN 0 WHEN initials = ?4 THEN 1 ELSE 2
END,
59             weight DESC, word ASC, id ASC
60 LIMIT ?5
61 """
62         var statement: OpaquePointer?
63         defer { sqlite3_finalize(statement) }
64         guard sqlite3_prepare_v2(database, sql, -1, &statement, nil) ==
SQLITE_OK else { return [] }
65         sqlite3_bind_int64(statement, 1, Int64(~disabledSourceMask))
66         bind(pinyinKey, to: statement, index: 2)
67         bind(pinyinKey + "*", to: statement, index: 3)
68         bind(initials, to: statement, index: 4)
69         sqlite3_bind_int(statement, 5, Int32(limit))
70         var values: [StoreEntry] = []
71         while sqlite3_step(statement) == SQLITE_ROW {

```

```

72         guard let wordText = sqlite3_column_text(statement, 1),
73             let pinyinText = sqlite3_column_text(statement, 2) else {
74             continue }
75         values.append(StoreEntry(
76             id: sqlite3_column_int64(statement, 0),
77             word: String(cString: wordText),
78             pinyin: String(cString: pinyinText),
79             weight: Int(sqlite3_column_int(statement, 3))
80         ))
81     }
82     return values
83 }
84
85 public func lookupExact(pinyinKey: String, disabledSourceMask: Int, limit: Int)
86     -> [StoreEntry] {
87     lock.withLock {
88         guard let database else { return [] }
89         let cacheKey = "\(disabledSourceMask)\u{0}\(limit)\u{0}\(pinyinKey)"

```

Listing 12.1: SQLiteStore 的打开、语句准备与查询入口

12.2 只读与完整性

系统库来自签名应用包，运行时不应修改。只读打开能减少锁和误写风险。启动时完整性检查失败，MyIME 会降级为空系统词库但保留英文直通，而不是让输入法进程持续崩溃。对输入法而言，“中文候选暂时不可用但键盘仍可输入”通常优于“所有按键失效”。

12.3 用户库写入

用户选择候选后更新词频、最近使用时间和上下文关系。写入不能阻塞每次按键的主路径。可以使用串行队列异步写入，同时提供测试用的 `waitForPendingWrites()` 以建立确定的断言。

```

1 public final class UserStore: @unchecked Sendable {
2     private let queue = DispatchQueue(label: "fudan.miniS.MyIME.user-store", qos: .utility)
3     private let lock = NSLock()
4     private var database: OpaquePointer?

```



```

5
6 public init?(path: String) {
7     let directory = URL(fileURLWithPath: path).deletingLastPathComponent()
8     try? FileManager.default.createDirectory(at: directory,
9 withIntermediateDirectories: true)
10    var handle: OpaquePointer?
11    guard sqlite3_open_v2(path, &handle, SQLITE_OPEN_READWRITE |
12 SQLITE_OPEN_CREATE | SQLITE_OPEN_FULLMUTEX, nil) == SQLITE_OK,
13        let handle else {
14        if let handle { sqlite3_close(handle) }
15        return nil
16    }
17    database = handle
18    let schema = """
19    PRAGMA journal_mode=WAL; PRAGMA busy_timeout=1000;
20    CREATE TABLE IF NOT EXISTS meta(key TEXT PRIMARY KEY, value TEXT);
21    INSERT OR IGNORE INTO meta VALUES('schema_version','1');
22    CREATE TABLE IF NOT EXISTS user_freq(word TEXT, pinyin TEXT, count INTEGER,
23 last_used INTEGER, PRIMARY KEY(word,pinyin));
24    CREATE TABLE IF NOT EXISTS user_phrase(word TEXT, pinyin TEXT, weight
25 INTEGER, created INTEGER, PRIMARY KEY(word,pinyin));
26    CREATE TABLE IF NOT EXISTS bigram(prev TEXT, word TEXT, count INTEGER,
27 PRIMARY KEY(prev,word));
28    """
29    guard sqlite3_exec(handle, schema, nil, nil, nil) == SQLITE_OK else {
30        sqlite3_close(handle)
31        database = nil
32        return nil
33    }
34 }
35
36 deinit {
37     if let database { sqlite3_close(database) }
38 }
39
40 public func learn(word: String, pinyin: String, previous: String?) {
41     queue.async { [weak self] in
42         self?.lock.withLock {
43             guard let database = self?.database else { return }
44             sqlite3_exec(database, "BEGIN IMMEDIATE", nil, nil, nil)
45             var statement: OpaquePointer?
46             let frequencySQL = """
47             INSERT INTO user_freq(word,pinyin,count,last_used) VALUES

```

```

    (?1,?2,1,?3)
43         ON CONFLICT(word,pinyin) DO UPDATE SET count=count+1,last_used=
excluded.last_used
44         """
45         if sqlite3_prepare_v2(database, frequencySQL, -1, &statement, nil)
== SQLITE_OK {
46             bind(word, to: statement, index: 1)
47             bind(pinyin, to: statement, index: 2)
48             sqlite3_bind_int64(statement, 3, Int64(Date().
timeIntervalSince1970))
49             sqlite3_step(statement)
50         }
51         sqlite3_finalize(statement)
52         if let previous, !previous.isEmpty {
53             statement = nil
54             let bigramSQL = """
55             INSERT INTO bigram(prev,word,count) VALUES(?1,?2,1)
56             ON CONFLICT(prev,word) DO UPDATE SET count=count+1
57             """
58             if sqlite3_prepare_v2(database, bigramSQL, -1, &statement, nil)
== SQLITE_OK {
59                 bind(previous, to: statement, index: 1)
60                 bind(word, to: statement, index: 2)
61                 sqlite3_step(statement)
62             }
63             sqlite3_finalize(statement)
64         }
65         sqlite3_exec(database, "COMMIT", nil, nil, nil)
66     }
67 }
68 }
69
70 public func learnPhrase(word: String, pinyin: String) {
71     guard word.count > 1, !pinyin.isEmpty else { return }
72     lock.withLock {
73         guard let database else { return }
74         var statement: OpaquePointer?
75         let sql = """
76         INSERT INTO user_phrase(word,pinyin,weight,created) VALUES
(?1,?2,4096,?3)
77         ON CONFLICT(word,pinyin) DO UPDATE SET
78             weight=min(65535,weight+4096),created=excluded.created
79         """

```

```

80         if sqlite3_prepare_v2(database, sql, -1, &statement, nil) == SQLITE_OK
81         {
82             bind(word, to: statement, index: 1)
83             bind(pinyin, to: statement, index: 2)
84             sqlite3_bind_int64(statement, 3, Int64(Date().timeIntervalSince1970
85         ))
86             sqlite3_step(statement)
87         }
88         sqlite3_finalize(statement)
89     }
90
91     public func lookup(pinyinKey: String, initials: String, limit: Int) -> [
92         StoreEntry] {
93         lock.withLock {
94             guard let database else { return [] }
95             let now = Int64(Date().timeIntervalSince1970)
96             let sql = """
97             SELECT rowid,word,pinyin,weight,created FROM user_phrase

```

Listing 12.2: UserStore 的本地持久化结构

工程警告：数据库失败必须静默降级

用户库损坏、目录只读或磁盘空间不足时，学习可以停止，但基本输入不能停止。系统词库损坏时也要避免在每次按键重复执行昂贵的失败操作。

动手实验：查询计划

复制一个小型 system.sqlite，分别在有无 pinyin key 索引时执行 1000 次查询。记录平均值与 P95，并使用 EXPLAINQUERYPLAN 保存查询计划截图。

思考与练习

1. 为什么系统词库和用户词库不应放在同一个可写文件中？
2. 前缀查询 LIKE 'bei%' 在什么条件下可以使用索引？
3. 设计用户库迁移版本号 and 原子升级流程。

本章小结

SQLite 在这里不是“方便保存数据”，而是运行时查询结构。只读系统库、独立用户库、针对性索引和失败降级共同保证输入路径稳定。

候选评分：从词频到对数域

学习目标

能够解释每个评分项；理解为什么多词句子要在对数域累加；掌握频率、用户偏好、上下文、完整度和容错惩罚的组合。

候选排序不是给每个词拍脑袋加一个权重。它要比较单个完整词与多个词组成的句子，还要容纳来自不同量纲的词频、用户次数和上下文统计。概率乘积在句子变长时迅速变小，因此通常转入对数域：

$$\log P(w_1, \dots, w_m) = \sum_{i=1}^m \log P(w_i \mid w_{i-1}, h).$$

MyIME 把归一化频率映射到伪对数尺度，每增加一个词产生插入成本，bigram/trigram 证据可以补偿该成本。

```

1  private enum Scale {
2      static let logRange = 16.0          // dynamic range of the pseudo log-
      probability
3      static let wordPenalty = 3.2        // per-word insertion cost
4      static let bigramReward = 6.0       // system word-bigram transition reward
5      static let userBigramReward = 1.4   // user bigram reward (log1p count, 0..4)
6      static let trigramReward = 1.6     // char-trigram boundary reward
7      static let userReward = 8.0        // user frequency/recency reward
8      static let fuzzyPenalty = 2.2      // per fuzzy syllable substitution
9      static let typoPenalty = 4.5       // per corrected transposition typo
10     static let partialPenalty = 1.1     // per abbreviated (initial-only)
      syllable
11     static let prefixPenalty = 0.8      // speculative pinyin-prefix completion
12     static let completionPenalty = 0.8  // per syllable the completion adds
      beyond typed tail
13     static let incompletePenalty = 5.0  // scaled by unconsumed input ratio
14 }

```

```

15
16 public init(store: any CandidateLookup, userStore: UserStore? = nil, segmenter:
    Segmenter = Segmenter()) {
17     self.store = store
18     self.userStore = userStore
19     self.segmenter = segmenter
20 }

```

Listing 13.1: 引擎共享状态与对数尺度常量

13.1 一个可解释的总分

可将候选 c 的总分抽象为：

$$\begin{aligned}
 S(c) = & \alpha F(c) + \beta U(c) + \gamma C(c, h) \\
 & - \lambda_f N_f - \lambda_t N_t - \lambda_p N_p \\
 & - \lambda_i R_{\text{inc}} - \lambda_c R_{\text{comp}}.
 \end{aligned}$$

其中 F 是系统频率， U 是用户偏好， C 是上下文；后两行分别惩罚模糊、错拼、简拼、未消费输入和超前补全。

初学者最容易把这些字母误读成“概率”，或把配置项的原始值直接与内部缩放常量比较。表 13.1 把数学符号、物理意义和 MyIME 1.0.10 的典型量级放在一起。典型值只是当前实现的默认值或内部尺度，不是普适常数；调参时仍须以封存验证集为准。

```

1         let pinyin = entry.pinyin.split(separator: "").map(String.init)
2         let entryKey = entry.pinyin.replacingOccurrences(of: "", with: "")
3         let base = wordLogProb(word: entry.word, weight: entry.weight,
isUserEntry: entry.id < 0) - Scale.wordPenalty
4         let user = (userStore?.boost(word: entry.word, pinyin: entry.pinyin
) ?? 0) + store.userBoost(word: entry.word, pinyin: entry.pinyin)
5         let context = contextBonus(anchors: anchors, word: entry.word)
6         let prefixCompletion = entryKey != pinyinKey && entryKey.hasPrefix(
pinyinKey) ? Scale.prefixPenalty : 0
7         let incompleteRatio = 1 - Double(consumedLength) / Double(
inputLength)
8         let score = prefs.frequencyWeight * base
9         + Scale.userReward * userScale * user

```

表 13.1: 候选评分符号与源码映射

符号	物理含义	典型取值	源码对应
α	系统词频奖励的权重	1.0	<code>frequencyWeight</code>
β	本地用户选择偏好的权重	0.35	<code>userWeight</code> ; 再乘归一化用户尺度
γ	bigram/trigram 上下文奖励的权重	0.20	<code>contextWeight</code>
λ_f	每次模糊音匹配的惩罚	0.08 (配置)	<code>fuzzyPenalty</code> , 内部按 <code>Scale</code> 缩放
λ_t	每次容错拼写边的惩罚	4.5 (内部尺度)	<code>Scale.typoPenalty</code>
λ_p	简拼或部分匹配的惩罚	0.10 (配置)	<code>partialPenalty</code> , 内部按 <code>Scale</code> 缩放
λ_i	未完整消费输入的惩罚	5.0 (内部尺度)	<code>Scale.incompletePenalty</code>
λ_c	超前补全一个音节的成本	0.8/音节	<code>Scale.completionPenalty</code>
N_f, N_t, N_p	对应边或匹配发生次数	非负整数	切分路径上的计数
R_{inc}	未消费输入量	字符或音节数	完整度检查
R_{comp}	候选超出当前输入的音节量	非负整数	前缀补全长度差

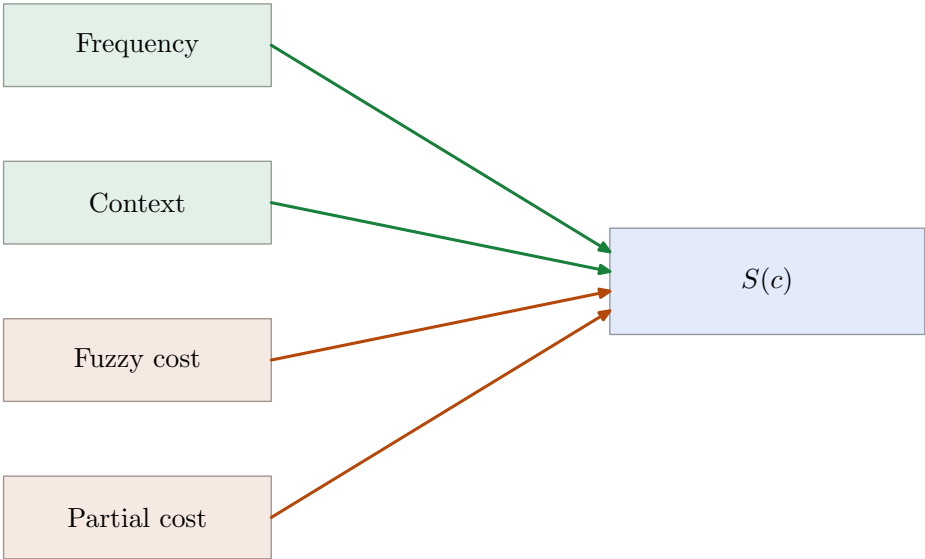


图 13.1: 候选总分由奖励项和惩罚项共同组成

```

10         + (prefs.contextWeight / 0.2) * context
11         - pathPenalty
12         - Scale.wordPenalty * Double(fallbackPenalty)
13         - prefixCompletion
14         - Scale.incompletePenalty * max(0, incompleteRatio)
15         let candidate = Candidate(id: entry.id, word: entry.word,
pinyinPath: pinyin, score: score, consumedLength: consumedLength)
16         // Key on the separator-free pinyin so identical readings collapse
across paths.
17         let key = entry.word + "\u{0}" + entryKey
18         if best[key].map({ candidate.score > $0.score }) ?? true { best[key
] = candidate }
19     }
20 }
21 // The lattice gets its own budget from this point so a slow cold word
lookup
22 // can never starve sentence composition; the first path always completes.
23 let latticeStart = ContinuousClock.now
24 let latticeDeadline = latticeStart.advanced(by: .milliseconds(60))
25 for (pathIndex, path) in paths.prefix(2).enumerated() {
26     if pathIndex > 0, ContinuousClock.now >= latticeDeadline { break }
27     let pathDeadline = pathIndex == 0 ? latticeStart.advanced(by: .
milliseconds(150)) : latticeDeadline
28     for sentence in latticeCandidates(for: path, anchors: anchors, prefs:
prefs, deadline: pathDeadline, limit: 6) {
29         let key = sentence.word + "\u{0}" + sentence.pinyinPath.joined()
30         if best[key].map({ sentence.score > $0.score }) ?? true { best[key
= sentence }
31     }
32 }
33 let candidates = best.values.sorted {
34     let lhsFull = $0.consumedLength >= inputLength
35     let rhsFull = $1.consumedLength >= inputLength
36     if lhsFull != rhsFull { return lhsFull }
37     if $0.score != $1.score { return $0.score > $1.score }
38     if $0.word != $1.word { return $0.word < $1.word }
39     return $0.id < $1.id
40 }
41 let candidatesWithEmoji = EmojiProvider.shared.addingEmojiCandidates(to:
candidates, limit: 100)
42 return EngineOutput(
43     preedit: firstPath.preedit,
44     candidates: candidatesWithEmoji,

```



```

45         hasMore: candidates.count > 100,
46         raw: normalized
47     )
48 }
49
50 // MARK: - Log-domain scoring
51
52 /// Blend dictionary weight with the corpus unigram model. Entries the corpus
53   has
54   /// never seen (merged sources inflate proper nouns like 江泽民同志) are
55   strongly
56   /// discounted so they cannot outrank corpus-supported words.
57 private func frequencyBlend(word: String, weight: Int, isUserEntry: Bool) ->
58   Double {
59     let dictionary = Double(weight) / 65_535.0

```

Listing 13.2: 单词候选的召回、加权、去重与稳定排序

13.2 长度偏差

直接累加词频会偏爱由很多高频单字组成的句子，或者反过来偏爱一个低质量长词。词插入成本是控制分词粒度的一种方法。另一些系统使用长度归一化、语言模型概率或学习排序。教学实现应先保证评分可解释，再追求更复杂模型。

13.3 稳定排序

当分数相等时，MyIME 继续比较是否完整消费、词文本和 ID。稳定 tie-breaker 对测试和肌肉记忆都重要。若候选在相同条件下随机交换，用户每次按数字键的结果都可能不同。

动手实验：权重消融

选择 20 条多义输入，依次把 userWeight、contextWeight、fuzzyPenalty 设
为零。记录首选变化，并为每条变化写出分数分解。禁止只报告“感觉更准”。

思考与练习

1. 为什么不能把原始词频直接与用户选择次数相加？

2. 证明概率乘积取对数后可转化为和，且不改变候选次序。
3. 设计一项避免超长候选在短输入中过早出现的惩罚。

本章小结

候选评分应在统一尺度上组合奖励与惩罚，并为每项保留解释。对数域让句子评分可加，稳定排序让结果可测试、可学习。

词格与 Beam Search：生成整句候选

学习目标

能够把音节序列转换成词格；理解 Viterbi/Beam Search 的状态、转移和剪枝；分析完整词、组合句与尾部简拼之间的竞争。

词库中未必存在“我是中国人”这个完整词条，但可能分别存在“我”“是”“中国”“人”。若音节路径包含 m 个音节，就在边界 $0 \dots m$ 上建立词格：词条覆盖音节区间 $[i, j]$ 时添加一条边。任何从 0 到 m 的路径都是一种句子组合。

完整枚举仍可能指数增长。Viterbi 在满足马尔可夫状态假设时保存每个位置的最优路径；为了输出多个候选并保留不同末词上下文，MyIME 使用宽度受限的 Beam Search。每扩展一层，只保留得分最高的若干假设。

14.1 宽度为 2 的动态推演

下图展示一次故意缩小的搜索。分数越大越好，Beam Width $B = 2$ 。开始只有空假设；第一步产生 A、B、C，只保留 A 与 B；第二步只能从幸存者继续展开，因此被剪掉的 C 即使后来存在一条非常好的后继边，也永远无法回来。

时刻	展开后的假设（累计分数）	排序与保留
0	$\epsilon : 0$	保留起点
1	A: -1.2, B: -1.7, C: -2.9	保留 A、B；剪掉 C
2	AD: -2.0, AE: -2.4, BF: -2.2, BG: -3.1	保留 AD、BF；剪掉 AE、BG

这张图还揭示一个重要边界：Beam Search 只承诺“当前层前 B 名”，不承诺全局最优。若 C 的下一条边奖励极高，完整枚举的最佳路径可能以 C 开头，但宽度

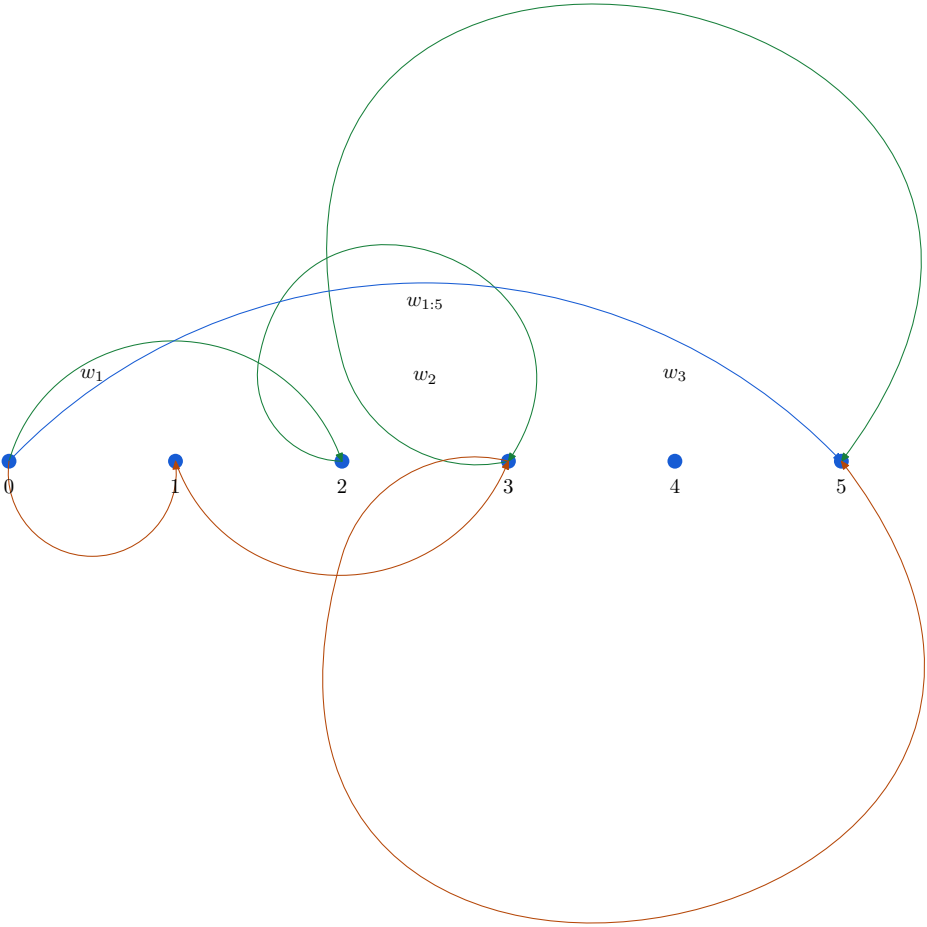


图 14.1: 词格中的完整词边与多词组合路径

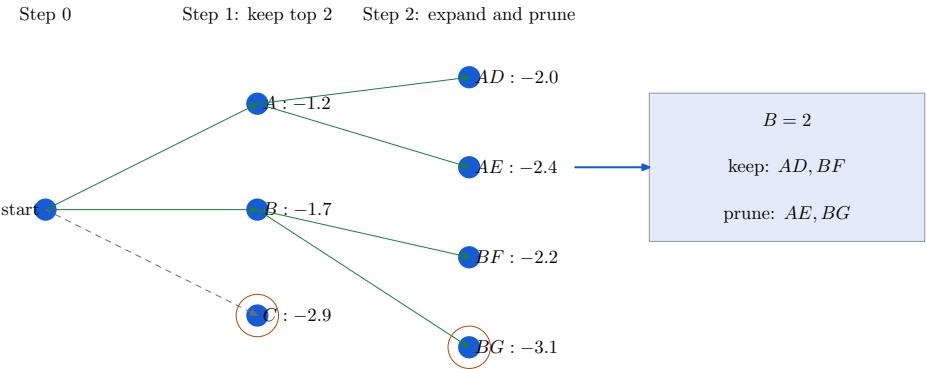


图 14.2: Beam Width 为 2 时, 假设的展开、排序与剪枝

2 已经把它永久删除。调试时应记录“每层入选阈值、被剪假设及其分数”，这样准确率回退才能定位到具体剪枝层。

14.2 假设状态

一个假设至少包含已生成文本、累计分数、词数和最后一个词。最后一个词用于计算下一条边的 bigram 转移。比较器依次看得分、词数和文本，保持确定性。

```

1  private struct Hypothesis {
2      let text: String
3      let score: Double
4      let pieces: Int
5      let lastWord: String
6  }
7
8  private static func hypothesisPrecedes(_ lhs: Hypothesis, _ rhs: Hypothesis) ->
    Bool {
9      if lhs.score != rhs.score { return lhs.score > rhs.score }
10     if lhs.pieces != rhs.pieces { return lhs.pieces < rhs.pieces }
11     return lhs.text < rhs.text
12 }
13
14 /// Composes sentence candidates over the word lattice with a beam search that
15 /// maximizes the summed log-domain score. Runs on every multi-syllable input;
16 /// a trailing abbreviated segment is completed through prefix lookup so the
17 /// sentence keeps forming while the user is still typing.
18 private func latticeCandidates(
19     for path: SegmentationPath,
20     anchors: [String],
21     prefs: EnginePrefs,
22     deadline: ContinuousClock.Instant,
23     limit: Int
24 ) -> [Candidate] {
25     let syllables = path.syllables
26     guard syllables.count > 1, syllables.count <= 16 else { return [] }
27     let isPartialSyllable: (String) -> Bool = { $0.count == 1 && !"aoe".
contains($0) }
28     let firstPartial = syllables.firstIndex(where: isPartialSyllable) ??
syllables.count
29     guard firstPartial >= 2 else { return [] }
30
31     let userScale = prefs.userWeight / 0.35

```

```

32     let fuzzyScale = prefs.fuzzyPenalty / 0.08
33     let partialScale = prefs.partialPenalty / 0.1
34     let beamWidth = 12
35
36     func nodeScore(_ entry: StoreEntry) -> Double {
37         let user = (userStore?.boost(word: entry.word, pinyin: entry.pinyin) ??
38 0)
39         + store.userBoost(word: entry.word, pinyin: entry.pinyin)
39         return wordLogProb(word: entry.word, weight: entry.weight, isUserEntry:
entry.id < 0)
40         - Scale.wordPenalty
41         + Scale.userReward * userScale * user
42     }
43     func linkScore(from hypothesis: Hypothesis, to word: String) -> Double {
44         guard hypothesis.pieces > 0 else {
45             return contextBonus(anchors: anchors, word: word) * 0.8
46         }
47         return transitionBonus(previousWord: hypothesis.lastWord, word: word)
48     }
49     func extend(_ beam: inout [Hypothesis], from sources: [Hypothesis], entries
: some Collection<StoreEntry>, extra: Double = 0) {
50         let scored = entries.map { ($0, nodeScore($0)) }
51         for hypothesis in sources {

```

Listing 14.1: 词格假设与句子组合入口

14.3 转移分数

从假设末词 u 转移到词 v 时, 可写成

$$S_{\text{new}} = S_{\text{old}} + F(v) - \lambda_{\text{word}} + B(u, v) + T(u, v) + U(v),$$

其中 B 是系统 bigram, T 是字符序列证据, U 是用户偏好。若尾部只有简拼, 查询可以允许前缀补全, 但必须扣除额外音节成本。

```

1         for hypothesis in sources {
2             for (entry, node) in scored {
3                 beam.append(Hypothesis(
4                     text: hypothesis.text + entry.word,
5                     score: hypothesis.score + node + linkScore(from: hypothesis
, to: entry.word) + extra,

```

```

6         pieces: hypothesis.pieces + 1,
7         lastWord: entry.word
8     ))
9 }
10 }
11 if beam.count > beamWidth {
12     beam.sort(by: Self.hypothesisPrecedes)
13     beam.removeSubrange(beamWidth...)
14 }
15 }
16
17 var beams = Array(repeating: [Hypothesis](), count: firstPartial + 1)
18 beams[0] = [Hypothesis(text: "", score: 0, pieces: 0, lastWord: "")]
19 for cursor in 0..

```

```

45         let tail = syllables[cursor...]
46         guard tail.count <= 6 else { continue }
47         let key = tail.joined()
48         let tailInitials = tail.allSatisfy { $0.count == 1 } ? tail.joined
    () : ""
49         let entries = (store.lookup(pinyinKey: key, initials: tailInitials,
    disabledSourceMask: prefs.disabledSourceMask, limit: 20)
50             + (userStore?.lookup(pinyinKey: key, initials: tailInitials,
    limit: 10) ?? []))
51         .prefix(10)
52         guard !entries.isEmpty else { continue }
53         // Prefer completions close to what was typed: 现在很m → 很忙/很美
54         // over 很明显/很迷茫 (each speculative extra syllable costs).
55         for entry in entries {
56             let entrySyllables = entry.pinyin.isEmpty
57                 ? entry.word.count
58                 : entry.pinyin.split(separator: "").count
59             let excess = Double(max(0, entrySyllables - tail.count))
60             extend(
61                 &finals,
62                 from: beams[cursor],
63                 entries: CollectionOfOne(entry),
64                 extra: -Scale.prefixPenalty - Scale.completionPenalty *
    excess
65             )
66         }
67     }
68 }
69
70 let pathPenalty = Scale.fuzzyPenalty * fuzzyScale * Double(path.
    fuzzyMatches)
71     + Scale.typoPenalty * Double(path.typoCount)
72     + Scale.partialPenalty * partialScale * Double(path.partialCount)
73 var seen = Set<String>()
74 return finals
75     .sorted(by: Self.hypothesisPrecedes)
76     .filter { $0.pieces > 1 && seen.insert($0.text).inserted }
77     .prefix(limit)
78     .enumerated()
79     .map { index, hypothesis in
80         Candidate(
81             id: Int64.min + Int64(index),
82             word: hypothesis.text,

```



```

83         pinyinPath: syllables,
84         score: hypothesis.score - pathPenalty,
85         consumedLength: path.consumedLength
86     )
87 }
88 }
89
90 // MARK: - Suggestions
91
92 public func suggestions(limit: Int = 9) -> [Candidate] {

```

Listing 14.2: 词格扩展、Beam 剪枝与句子候选生成

14.4 时间预算

输入法每个按键都会重新求候选，不能让某个歧义长串无限搜索。MyIME 为普通词查找和词格组合设置独立 deadline，使冷启动查询不会完全挤占整句预算。时间预算是一种工程剪枝：结果可能不是数学上全局最优，但必须在交互期限内返回。

核心概念：正确性与实时性的交换

桌面输入法的目标不是离线求出所有句子，而是在几十毫秒内给出足够好的前几项。Beam 宽度、候选边上限和 deadline 都属于产品质量参数，应通过准确率—延迟曲线选择。

动手实验：Beam 宽度实验

把 Beam 宽度分别设为 2、4、8、12、24，在固定 30 句验证集上测 top-1 与 P95 延迟。画出横轴为宽度、纵轴为准确率和延迟的双曲线，解释拐点。

思考与练习

1. 画出 wo'shi'zhong'guo'ren 的一个词格。
2. 为什么仅按“当前位置”合并假设可能丢失正确上下文？
3. Beam Search 在什么情况下会剪掉最终最优路径？
4. 为长于 16 音节的输入设计分段策略。

本章小结

整句生成是词格上的受限搜索。候选质量取决于边召回、转移分数与剪枝；交互系统必须同时给准确率和延迟设预算。

上下文与本地用户学习

学习目标

理解短期历史、用户词频和 bigram 的不同作用；设计异步、可衰减、可删除的本地学习；避免把偶然选择永久固化。

静态词库无法知道某位用户经常输入“课程设计”还是“产品设计”。本地学习通过重复选择提高候选权重，通过相邻上屏词建立上下文关系，还可以把短时间内分开选择的字词组合成新词。

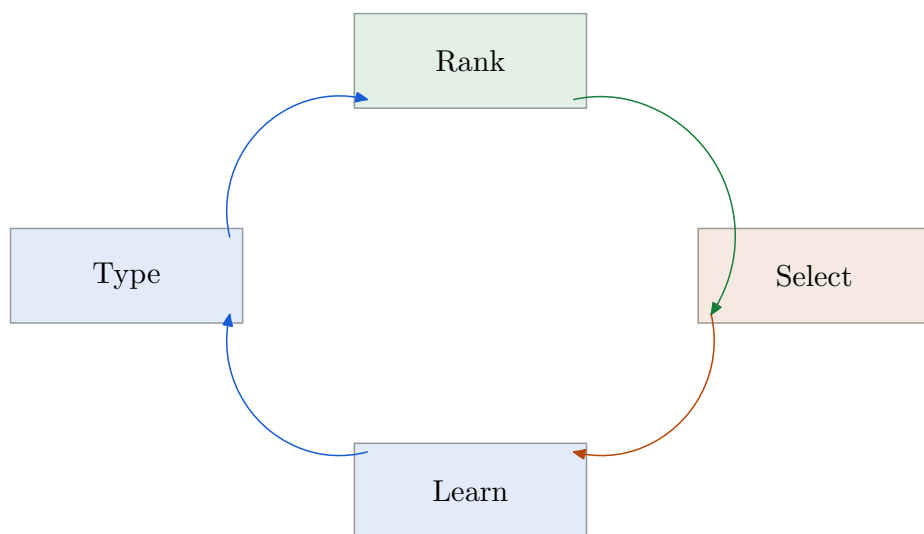


图 15.1: 输入、排序、选择和学习形成的本地反馈闭环

15.1 三种学习信号

用户 **unigram** 同一词与读音被选择的次数和最近时间；

用户 **bigram** 在前词 u 后选择 v 的次数；

近期组词 两次相近选择在短时间窗口内组合为新短语。

用户权重不宜线性无限增长。常用形式是 $\log(1 + c)$ 或带上限的归一化，并加入时间衰减：

$$U(w) = \log(1 + c_w) \exp\left(-\frac{t - t_w}{\tau}\right).$$

这里 c_w 是选择次数， t_w 是最近使用时间， τ 控制遗忘速度。

```

1 public func suggestions(limit: Int = 9) -> [Candidate] {
2     let previous = stateLock.withLock { committedHistory.last }
3     guard let previous, !previous.isEmpty else { return [] }
4
5     // Prefer the exact committed phrase. Single-character suffix anchors pull
6     // in web junk
7     // and drown useful continuations, so only keep a two-character suffix as
8     // fallback.
9     var anchors = [previous]
10    if previous.count > 2 { anchors.append(String(previous.suffix(2))) }
11
12    var seen = Set<String>()
13    var ranked: [(word: String, score: Double)] = []
14    func consider(word: String, anchor: String, base: Double, exactAnchor: Bool
15    ) {
16        guard word.count >= 2, !Self.isLowQualityAssociation(word), seen.insert
17        (word).inserted else { return }
18        let user = userStore?.bigram(previous: previous, word: word) ?? 0
19        let system = store.bigramBoost(previous: anchor, word: word)
20        let sequence = store.sequenceBoost(prefix: previous, word: word)
21        let unigram = store.unigramBoost(word: word)
22        // User intent and exact-phrase bigrams dominate; raw unigram often
23        rewards content-farm heads.
24        let score = base
25            + user * 2.8
26            + system * (exactAnchor ? 3.2 : 1.1)
27            + sequence * 0.9
28            + unigram * 0.12
29        ranked.append((word, score))
30    }
31
32    for entry in userStore?.predictNext(after: previous, limit: limit * 2) ??

```

```

[] {
28     consider(word: entry.word, anchor: previous, base: Double(entry.weight)
/ 65_535.0 + 1.5, exactAnchor: true)
29 }
30 for word in AssociationSeeds.continuations(after: previous) {
31     consider(word: word, anchor: previous, base: 1.8, exactAnchor: true)
32 }
33 for (index, anchor) in anchors.enumerated() {
34     let exactAnchor = index == 0
35     for entry in store.predictNext(after: anchor, limit: limit * 4) {
36         consider(
37             word: entry.word,
38             anchor: anchor,
39             base: Double(entry.weight) / 65_535.0 * (exactAnchor ? 0.45 :
0.2),
40             exactAnchor: exactAnchor
41         )
42     }
43     if exactAnchor, ranked.count >= limit { break }
44 }
45
46 return ranked
47     .sorted {
48         if $0.score != $1.score { return $0.score > $1.score }
49         return $0.word < $1.word
50     }
51     .prefix(limit)
52     .enumerated()
53     .map { index, item in
54         Candidate(
55             id: Int64.min / 4 + Int64(index),
56             word: item.word,
57             pinyinPath: [],
58             score: item.score,
59             consumedLength: 0
60         )
61     }
62 }
63
64 private static func isLowQualityAssociation(_ word: String) -> Bool {
65     let bannedFragments = [
66         "小编", "给大家", "为大家", "特价", "大盘", "点击", "点赞", "关注", "转发",
        "

```

```

67         "头条", "链接", "阅读原文", "扫码", "微信号",
68     ]
69     return bannedFragments.contains { word.contains($0) }
70 }
71
72 // MARK: - Selection & learning
73
74 public func select(_ index: Int, from output: EngineOutput) -> SelectResult {
75     guard output.candidates.indices.contains(index) else { return SelectResult(
76         commitText: "", remainingRaw: output.raw) }
77     let candidate = output.candidates[index]
78     let plainRaw = output.raw.filter { $0 != " " }
79     let remaining = String(plainRaw.dropFirst(min(candidate.consumedLength,
80         plainRaw.count)))
81     return SelectResult(commitText: candidate.word, remainingRaw: remaining)
82 }
83
84 public func commitLearning(word: String, pinyin: [String]) {
85     let previous = stateLock.withLock { () -> String? in
86         defer { committedHistory = (last: word, beforeLast: committedHistory.
87             last) }
88         return committedHistory.last
89     }
90     userStore?.learn(word: word, pinyin: pinyin.joined(separator: " "),
91         previous: previous)
92 }
93
94 public func commitUserPhrase(word: String, pinyin: [String], at time:
95     TimeInterval? = nil) {
96     guard !word.isEmpty, !pinyin.isEmpty else { return }
97     let timestamp = time ?? Date().timeIntervalSinceReferenceDate
98     let combined = stateLock.withLock { () -> (String, [String])? in
99         guard let recentPhrase,
100             timestamp >= recentPhrase.timestamp,
101             timestamp - recentPhrase.timestamp <= 3 else {
102             self.recentPhrase = (word, pinyin, timestamp)
103             return nil
104         }
105         let combinedWord = recentPhrase.word + word
106         let combinedPinyin = recentPhrase.pinyin + pinyin
107         guard combinedWord.count <= 12, combinedPinyin.count <= 12 else {
108             self.recentPhrase = (word, pinyin, timestamp)
109             return nil
110         }

```

```

105         }
106         self.recentPhrase = (combinedWord, combinedPinyin, timestamp)
107         return (combinedWord, combinedPinyin)
108     }
109     if word.count > 1 {
110         userStore?.learnPhrase(word: word, pinyin: pinyin.joined(separator: "'"))
111     })

```

Listing 15.1: 续写候选、选择结果和提交学习

15.2 近期组词的边界

如果用户在 3 秒内先选择“课程”再选择“设计”，可以学习“课程设计”；时间距离过大则不组合。时间窗口只是启发式，还需要限制总长度、字符类型和低质量网页短语。MyIME 的测试分别验证近距离选择会组词、远距离选择不会组词。

```

1      let userStore = try #require(UserStore(path: directory.appending(path: "
user.sqlite").path))
2      var weights: [Int] = []
3      for _ in 0..<16 {
4          userStore.learnPhrase(word: "新词", pinyin: "xin'ci")
5          userStore.waitForPendingWrites()
6          weights.append(try #require(userStore.lookup(pinyinKey: "xinci",
initials: "xc", limit: 10).first).weight)
7      }
8      #expect(weights == weights.sorted())
9      #expect(weights.last == 65_535)
10     let competingStore = MemoryStore(entries: [StoreEntry(id: 20, word: "心词",
pinyin: "xin'ci", weight: 65_535)])
11     let output = Engine(store: competingStore, userStore: userStore).update("
xinci", prefs: EnginePrefs())
12     #expect(output.candidates.first?.word == "新词")
13 }
14
15 @Test func repeatedWordPairGainsContextWeight() throws {
16     let directory = FileManager.default.temporaryDirectory.appending(path: "
MyIME-context-tests-\(UUID().uuidString)")
17     defer { try? FileManager.default.removeItem(at: directory) }
18     let userStore = try #require(UserStore(path: directory.appending(path: "
user.sqlite").path))
19     userStore.learn(word: "天气", pinyin: "tian'qi", previous: "今天")

```

```

20     userStore.waitForPendingWrites()
21     let firstBoost = userStore.bigram(previous: "今天", word: "天气")
22     for _ in 0..<8 {
23         userStore.learn(word: "天气", pinyin: "tian'qi", previous: "今天")
24     }
25     userStore.waitForPendingWrites()
26     #expect(firstBoost > 0)
27     #expect(userStore.bigram(previous: "今天", word: "天气") > firstBoost)
28 }
29
30 @Test func nearbySeparateSelectionsBecomeARecentPhrase() throws {
31     let directory = FileManager.default.temporaryDirectory.appending(path: "
MyIME-nearby-tests-\(UUID().uuidString)")
32     defer { try? FileManager.default.removeItem(at: directory) }
33     let userStore = try #require(UserStore(path: directory.appending(path: "
user.sqlite").path))
34     let engine = Engine(store: MemoryStore(entries: []), userStore: userStore)
35
36     engine.commitUserPhrase(word: "输", pinyin: ["shu"], at: 100)
37     engine.commitUserPhrase(word: "入", pinyin: ["ru"], at: 102)
38
39     let output = engine.update("shuru", prefs: EnginePrefs())
40     #expect(output.candidates.first?.word == "输入")
41 }
42
43 @Test func distantSelectionsDoNotBecomeAPhrase() throws {
44     let directory = FileManager.default.temporaryDirectory.appending(path: "
MyIME-distant-tests-\(UUID().uuidString)")
45     defer { try? FileManager.default.removeItem(at: directory) }
46     let userStore = try #require(UserStore(path: directory.appending(path: "
user.sqlite").path))
47     let engine = Engine(store: MemoryStore(entries: []), userStore: userStore)
48
49     engine.commitUserPhrase(word: "输", pinyin: ["shu"], at: 100)
50     engine.commitUserPhrase(word: "入", pinyin: ["ru"], at: 104)
51
52     #expect(engine.update("shuru", prefs: EnginePrefs()).candidates.isEmpty)
53 }
54
55 @Test func updateP95StaysWithinBudget() {

```

Listing 15.2: 用户词、上下文和近期组词的行为测试

15.3 隐私与控制权

本地学习的优势不仅是离线，还包括可解释和可删除。用户应能导出、清空或禁用学习。调试日志不能记录完整敏感输入；课堂项目可以在 Debug 构建中记录哈希、长度和候选 ID，在 Release 中关闭原文日志。

工程警告：不要学习所有东西

密码框、安全输入、一次性验证码、包含异常控制字符的文本和用户取消的组合都不应进入学习库。输入法必须尊重客户端的安全输入状态。

动手实验：个性化收敛

构造两个同音候选 A、B，使 A 初始领先。连续选择 B 1、3、5、10 次，记录其排名和得分；重启引擎后复测。再等待或模拟时间衰减，观察偏好是否下降。

思考与练习

1. 为什么最近一次选择不应无条件永久置顶？
2. 如何在完全离线前提下跨机器迁移用户词库？
3. 设计“撤销一次错误学习”的数据结构。

本章小结

用户学习是受控反馈系统，不是简单计数。次数、时间、上下文、组词边界、异步写入和用户删除权必须一起设计。

Emoji、英文辅助与繁体转换的插件化

学习目标

理解附加候选不应破坏中文主排序；能够设计候选后处理管线；区分英文辅助、中文输入源和真正的双语输入法。

候选生成完成后，还可以附加 Emoji、英文拼写建议或繁体显示。最安全的做法是把这些功能当作后处理插件：输入基础候选，输出扩展候选，同时保持首选稳定和来源可追踪。

16.1 Emoji 紧随语义候选

如果“医院”命中对应的医院 Emoji，表情符号可以插在该文本候选之后，而不是统一堆在列表尾部或抢到首位。MyIME 使用映射表并保持去重。

```
1 import Foundation
2
3 struct EmojiProvider: Sendable {
4     static let shared = EmojiProvider()
5
6     private let emojisByWord: [String: [String]]
7
8     init() {
9         guard let url = Bundle.module.url(forResource: "emoji-map", withExtension:
10             "txt"),
11             let contents = try? String(contentsOf: url, encoding: .utf8) else {
12             emojisByWord = [:]
13             return
14         }
15     }
16 }
```

```

14
15     var result: [String: [String]] = [:]
16     for line in contents.split(whereSeparator: \.isNewline) {
17         let fields = line.split(whereSeparator: \.isWhitespace).map(String.init
18     )
19         guard fields.count > 1, !fields[0].hasPrefix("#") else { continue }
20         let emoji = fields[0]
21         for word in fields.dropFirst() where !result[word, default: []].
contains(emoji) {
22             result[word, default: []].append(emoji)
23         }
24     }
25     emojisByWord = result
26
27 func addingEmojiCandidates(to candidates: [Candidate], limit: Int) -> [
Candidate] {
28     var result: [Candidate] = []
29     var usedEmoji = Set<String>()
30     result.reserveCapacity(min(limit, candidates.count * 2))
31
32     for candidate in candidates {
33         guard result.count < limit else { break }
34         result.append(candidate)
35         for emoji in emojisByWord[candidate.word] ?? [] where result.count <
limit && usedEmoji.insert(emoji).inserted {
36             result.append(Candidate(
37                 id: stableID(for: emoji),
38                 word: emoji,
39                 pinyinPath: candidate.pinyinPath,
40                 score: candidate.score.nextDown,
41                 consumedLength: candidate.consumedLength
42             ))
43         }
44     }
45     return result
46 }
47
48 private func stableID(for value: String) -> Int64 {
49     var hash: UInt64 = 14_695_981_039_346_656_037
50     for byte in value.utf8 {
51         hash ^= UInt64(byte)
52         hash &*= 1_099_511_628_211

```

```

53     }
54     return Int64(bitPattern: hash | (1 << 63))
55 }
56 }

```

Listing 16.1: Emoji 候选的映射、插入与去重

插件需要明确优先级。例如基础候选索引为 i ，附加候选应获得小于主候选、但大于远端无关候选的显示位置。相比重新混合所有分数，“紧随来源候选”更容易解释。

16.2 英文辅助不是英文模式

MyIME 在中文组合中识别可能的英文 raw，允许回车原样提交并给出基础建议；持续英文输入仍推荐切换 ABC。这避免输入法内部维护第二套隐形语言状态。商业双语输入法可以选择更复杂策略，但教材应让学生明确三种不同需求：

1. 中文句子里临时输入英文单词；
2. 持续输入完整英文段落；
3. 拼音串恰好也是英文单词时判断用户意图。

```

1  private func addingEnglishCandidates(to base: EngineOutput) -> EngineOutput {
2      guard raw.count >= 2, raw.allSatisfy({ $0.isASCII && $0.isLetter }) else {
3          return base }
4      let checker = NSSpellChecker.shared
5      let fullRange = NSRange(location: 0, length: raw.utf16.count)
6      let misspelled = checker.checkSpelling(
7          of: raw,
8          startingAt: 0,
9          language: "en_US",
10         wrap: false,
11         inSpellDocumentWithTag: 0,
12         wordCount: nil
13     )
14     let words: [String]
15     if misspelled.location == NSNotFound {
16         words = [raw]
17     } else {
18         words = Array((checker.guesses(

```

```

19         in: raw,
20         language: "en_US",
21         inSpellDocumentWithTag: 0
22     ) ?? []).prefix(2))
23 }
24 let uniqueWords = words.reduce(into: [String]()) { result, word in
25     if !result.contains(where: { $0.caseInsensitiveCompare(word) == .
orderedSame }) {
26         result.append(word)
27     }
28 }
29 guard !uniqueWords.isEmpty else { return base }
30
31 var candidates = base.candidates
32 let insertionIndex = min(3, candidates.count)
33 let englishCandidates = uniqueWords.enumerated().map { index, word in
34     Candidate(
35         id: Int64.min / 2 + Int64(index),
36         word: word,
37         pinyinPath: [],
38         score: 0,
39         consumedLength: raw.count
40     )
41 }
42 candidates.insert(contentsOf: englishCandidates, at: insertionIndex)
43 return EngineOutput(
44     preedit: base.preedit,
45     candidates: Array(candidates.prefix(100)),
46     hasMore: base.hasMore || candidates.count > 100,
47     raw: base.raw
48 )
49 }
50
51 private func localizedPunctuation(for character: Character) -> String {
52     let values: [Character: String] = [",": " ", ".": "。", "?": "? ", " ": "!",
53     ";": "; ", ":": ":", "(": " (", ")": ") "]
54     return values[character] ?? String(character)

```

Listing 16.2: 控制器附加基础英文候选

16.3 后处理次序

若先繁体转换再 Emoji 映射，映射表可能只认识简体；若先插 Emoji 再去重，要避免把相同符号重复添加。一个清晰管线是：基础引擎候选 → 英文辅助 → 繁体显示映射 → 候选窗；学习时仍保存规范化简体词和原拼音，提交时再输出用户选择的显示形式。

动手实验：插件稳定性

为“医院”“咖啡”“good”设计候选快照测试。依次启用 Emoji、繁体和英文辅助，确认中文首选不变、Emoji 紧邻语义词、繁体只改变显示与提交。

思考与练习

1. 如果用户选择 Emoji，是否应该学习其对应中文词？说明方案。
2. 设计一个不会把 `shi` 误判为英文的判定规则。
3. 后处理插件应否直接修改 Candidate 的 score？比较两种方案。

本章小结

附加能力应通过可组合后处理扩展核心候选，并保持中文首选、学习语义和显示转换之间的边界。英文辅助不等于完整英文输入模式。

准确率、延迟与科学评测

学习目标

能设计训练集、开发集和测试集；理解 top-k、KSPC、P50/P95、确定性和冷启动；避免把小型回归集的 100% 当作商业准确率。

输入法“感觉不错”无法指导迭代。评测至少包含候选准确率、击键效率、交互延迟、崩溃与确定性。不同指标可能冲突：扩大 Beam 会提高召回，也可能增加 P95；激进用户学习会提高个人语料准确率，也可能污染新场景。

17.1 候选指标

给定测试集 $D = \{(x_i, y_i)\}_{i=1}^N$ ，top-k 准确率为

$$A_k = \frac{1}{N} \sum_{i=1}^N \mathbf{1}\{y_i \in C_k(x_i)\}.$$

top-1 衡量空格直接上屏，top-5 衡量用户是否能在首屏找到。**KSPC**（每字符击键数）进一步计入翻页、数字选词和纠错动作，更接近真实效率。

17.2 延迟指标

平均延迟会隐藏少数卡顿，应报告 P50、P95、P99 和最大值。分开测冷启动首键、热词查询、长句、简拼和错拼。输入法每次按键都更新候选，因此 100ms 的尾延迟已经明显影响跟手感。

17.3 MyIME 内部回归集

项目提供整句、常用词、多义词、简拼、错拼和尾部简拼测试。这些用例适合防止已知行为退化，但规模较小且由项目开发者维护，不能据此声称与微信或搜狗达到相同准确率。

```

1 import Foundation
2 import Testing
3 @testable import IMKit
4
5 /// Offline quality benchmark against the bundled system dictionary.
6 /// Each category asserts a minimum accuracy so regressions in ranking,
7 /// sentence composition, abbreviation or typo tolerance fail loudly,
8 /// while individual flaky words don't break the build.
9 @Suite("AccuracyEval", .serialized) struct AccuracyEvalTests {
10     private static let sentencesTop1: [(String, String)] = [
11         ("jintiantianqihenhao", "今天天气很好"),
12         ("womenmingtianyiqichifan", "我们明天一起吃饭"),
13         ("xianzaijidianle", "现在几点了"),
14         ("woyijingdaojiale", "我已经到家了"),
15         ("zhegewentihenzhongyao", "这个问题很重要"),
16         ("xiexienidebangzhu", "谢谢你的帮助"),
17         ("zhunishengrikuaile", "祝你生日快乐"),
18         ("mingtianzaoshangkaihui", "明天早上开会"),
19         ("woxianzaihenmang", "我现在很忙"),
20         ("nichifanlema", "你吃饭了吗"),
21         ("wanshangyiqikandianying", "晚上一起看电影"),
22         ("jintianwanshangchishenme", "今天晚上吃什么"),
23         ("womenshenmeshihouchufa", "我们什么时候出发"),
24         ("nimgtianyounkongma", "你明天有空吗"),
25         ("woganggangxiaban", "我刚刚下班"),
26         ("huochezhanzenmezou", "火车站怎么走"),
27         ("mashangjiudao", "马上就到"),
28         ("wozhengzaikaihui", "我正在开会"),
29         ("nizaiganshenme", "你在干什么"),
30         ("haojiubujian", "好久不见"),
31         ("zhegezhoumoyoushenmeanpai", "这个周末有什么安排"),
32         ("mingtianhuixiayuma", "明天会下雨吗"),
33         ("wodeshoujimeidianle", "我的手机没电了"),
34         ("nixiangchishenme", "你想吃什么"),
35         ("womenzaishangbandelushang", "我们在上班的路上"),
36         ("zhejiacantinghenhaochi", "这家餐厅很好吃"),

```



```

37     ("wojintianyoudianlei", "我今天有点累"),
38     ("nishenmeshihouxiaban", "你什么时候下班"),
39     ("zhouyiwoyaochuchai", "周一我要出差"),
40     ("bangwokanyixiazhegewenjian", "帮我看一下这个文件"),
41 ]
42
43 private static let wordsTop1: [(String, String)] = [
44     ("nihao", "你好"), ("xiexie", "谢谢"), ("zaijian", "再见"),
45     ("pengyou", "朋友"), ("shijian", "时间"), ("gongzuo", "工作"),
46     ("xuexi", "学习"), ("kaixin", "开心"), ("shouji", "手机"),
47     ("dianna", "电脑"), ("went", "问题"), ("bangzhu", "帮助"),
48     ("xihuan", "喜欢"), ("zhidao", "知道"), ("juede", "觉得"),
49     ("keyi", "可以"), ("xianzai", "现在"), ("mingtian", "明天"),
50     ("zuotian", "昨天"), ("weishenme", "为什么"), ("zenmeyang", "怎么样"),
51     ("dianying", "电影"), ("yinyue", "音乐"), ("yiyuan", "医院"),
52     ("xuexiao", "学校"), ("laoshi", "老师"), ("tongshi", "同时"),
53     ("huiyi", "会议"), ("lvxing", "旅行"), ("chifan", "吃饭"),
54     ("shuijiao", "睡觉"), ("xiawu", "下午"), ("shangban", "上班"),
55     ("xiaban", "下班"), ("kafei", "咖啡"), ("pijiu", "啤酒"),
56     ("dianhua", "电话"), ("youxi", "游戏"), ("shengri", "生日"),
57     ("zhoumo", "周末"),
58 ]
59
60 private static let wordsTop3: [(String, String)] = [
61     ("yisi", "意思"), ("shiji", "实际"), ("jiaoyu", "教育"),
62     ("xiaoguo", "效果"), ("yijian", "意见"), ("gongsi", "公司"),
63     ("kaishi", "开始"), ("xingqi", "星期"), ("banfa", "办法"),
64     ("qingkuang", "情况"), ("jihua", "计划"), ("fangbian", "方便"),
65     ("anpai", "安排"), ("zhunbei", "准备"), ("liaojie", "了解"),
66     ("jieshu", "结束"), ("canjia", "参加"), ("tongyi", "同意"),
67     ("queren", "确认"), ("fasong", "发送"), ("tongshi", "同事"),
68 ]
69
70 private static let abbreviationsTop5: [(String, String)] = [
71     ("bj", "北京"), ("sh", "上海"), ("nh", "你好"),
72     ("sj", "手机"), ("xx", "学习"), ("gz", "工作"),
73     ("wt", "问题"), ("pengy", "朋友"), ("xiex", "谢谢"),
74     ("mingt", "明天"),
75 ]
76
77 private static let typosTop3: [(String, String)] = [
78     ("nihao", "你好"),
79     ("zhongguo", "中国"),

```

```

80     ("jintain", "今天"),
81     ("shijei", "世界"),
82     ("xeixie", "谢谢"),
83     ("migtian", "明天"),
84 ]
85
86 private static let trailingPartialTop3: [(String, String)] = [
87     ("jintiantianq", "今天天气"),
88     ("mingtianzaosh", "明天早上"),
89     ("woxianzaihenm", "我现在很忙"),
90     ("xiexienidebangz", "谢谢你的帮助"),
91 ]
92
93 private static func bundledStore() throws -> SQLiteStore {
94     let packageDirectory = URL(fileURLWithPath: #filePath)
95         .deletingLastPathComponent()
96         .deletingLastPathComponent()
97         .deletingLastPathComponent()
98     let databasePath = ProcessInfo.processInfo.environment["
99 MYIME_TEST_SYSTEM_DATABASE"]
100     ?? packageDirectory.appending(path: "../MyIME/Resources/system.sqlite")
101     .standardizedFileURL.path
102     return try #require(SQLiteStore(path: databasePath))
103 }
104
105 private static func measure(
106     _ cases: [(String, String)],
107     topK: Int,
108     engine: Engine,
109     label: String
110 ) -> Double {
111     var hits = 0
112     var misses: [String] = []
113     for (input, expected) in cases {
114         let words = engine.update(input, prefs: EnginePrefs()).candidates.
115         prefix(topK).map(\.word)
116         if words.contains(expected) {
117             hits += 1
118         } else {
119             misses.append("\(input) → 期望 \(expected), 实得 \(words.prefix(3).
120             joined(separator: "/"))")
121         }
122     }
123 }

```

```

119     let accuracy = Double(hits) / Double(max(1, cases.count))
120     print("EVAL[\\(label)] top\\(topK): \\(hits)/\\(cases.count) = \\(String(format:
121     "%.1f%%", accuracy * 100))")
122     for miss in misses { print("  MISS \\(miss)") }
123     return accuracy
124 }
125
126 @Test func benchmarkAgainstBundledDictionary() throws {
127     let engine = Engine(store: try Self.bundledStore())
128     let sentence = Self.measure(Self.sentencesTop1, topK: 1, engine: engine,
129     label: "整句")
130     let word1 = Self.measure(Self.wordsTop1, topK: 1, engine: engine, label: "
131     常用词")
132     let word3 = Self.measure(Self.wordsTop3, topK: 3, engine: engine, label: "
133     多义词")
134     let abbreviation = Self.measure(Self.abbreviationsTop5, topK: 5, engine:
135     engine, label: "简拼")
136     let typo = Self.measure(Self.typosTop3, topK: 3, engine: engine, label: "错
137     拼")
138     let partial = Self.measure(Self.trailingPartialTop3, topK: 3, engine:
139     engine, label: "尾部简拼")
140
141     #expect(sentence >= 0.96)
142     #expect(word1 >= 0.95)
143     #expect(word3 >= 0.95)
144     #expect(abbreviation >= 0.90)
145     #expect(typo >= 0.80)
146     #expect(partial >= 0.75)
147 }
148 }

```

Listing 17.1: 准确率评测的用例结构与 top-k 断言

17.4 避免数据泄漏

如果看到错误就直接把测试句加入词库，再用同一测试集报告提升，得到的是记忆而非泛化。课程项目至少分为：训练/词库构建数据、调参开发集、最终封存测试集。最终测试集在截止前不公开标签。

动手实验：建立班级基准

全班共同设计 500 条匿名句子,按日常、专业、新词、多义、简拼和错拼分层。每组只能在 300 条开发集上调参,教师用 200 条封存集验收。报告 top-1/top-5、KSPC 和 P95。

思考与练习

1. top-5 提高而 KSPC 变差,可能是什么原因?
2. 为什么应同时测试冷启动和热启动?
3. 设计一个衡量候选稳定性的指标。
4. 如何在不收集用户击键的情况下构建公开评测集?

本章小结

评测必须回答“在哪类数据上、用什么指标、在什么环境下”。小型回归集证明已知行为没有退化,不证明真实世界全面领先。

第三部分

工程篇：把实验变成可发布的软件

安装与输入源注册：为什么复制应用还不够

学习目标

理解输入法安装位置、输入源注册和系统缓存；区分开发目录副本、DMG 副本与已安装副本；能够设计幂等、自愈且不破坏用户数据的安装流程。

普通 macOS 应用可以从 Applications 目录启动，输入法还要被文本输入系统发现。MyIME 采用当前用户安装，把自身复制到 `~/Library/InputMethods/MyIME.app`，注册输入源，再从安装位置重新启动。这样不需要管理员权限，也避免开发构建与正式输入服务并存。

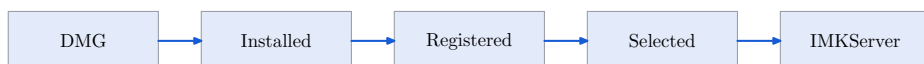


图 18.1: 从 DMG 到 IMKServer 的输入源生命周期

18.1 幂等安装

理想安装器重复运行不会制造多个副本或多个登录项。它应先解析自身位置和目标位置，比较版本，安全替换目标，注册输入源，移除历史机制，再启动目标副本。任何删除都必须使用已经解析并验证的具体路径。

```
1 import AppKit
2 import Carbon
3 import Darwin
4 import os
5
6 enum SelfInstaller {
7     static var installedURL: URL {
```

```

8     FileManager.default.homeDirectoryForCurrentUser
9         .appendingPathComponent("Library/Input Methods/MyIME.app", isDirectory:
10     true)
11 }
12 private static let legacySystemURL = URL(
13     fileURLWithPath: "/Library/Input Methods/MyIME.app",
14     isDirectory: true
15 )
16 static let inputModeID = "fudan.miniS.inputmethod.MyIME.Chinese"
17
18 static var isRunningInstalledCopy: Bool {
19     Bundle.main.bundleURL.resolvingSymlinksInPath().standardizedFileURL
20     == installedURL.resolvingSymlinksInPath().standardizedFileURL
21 }
22
23 static var isCurrentInputSource: Bool {
24     currentInputSourceID() == inputModeID
25 }
26
27 static var isPostInstallLaunch: Bool {
28     ProcessInfo.processInfo.arguments.contains("--post-install")
29 }
30
31 static func performCommandIfRequested(logger: Logger) -> Bool {
32     guard let command = ProcessInfo.processInfo.arguments.dropFirst().first
33     else { return false }
34     switch command {
35     case "--register-input-source", "--install":
36         _ = ensureRegistered(logger: logger)
37     case "--enable-input-source":
38         _ = setInputSourceEnabled(true, logger: logger)
39     case "--disable-input-source":
40         _ = setInputSourceEnabled(false, logger: logger)
41     case "--select-input-source":
42         // Programmatic TIS selection can leave a visually selected but
43         // disconnected IMK
44         // session on recent macOS releases. Keep this legacy command
45         registration-only;
46         // the user must select MyIME through the system input menu or keyboard
47         shortcut.
48         _ = ensureRegistered(logger: logger)
49     case "--quit":
50         let bundleID = Bundle.main.bundleIdentifier ?? "fudan.miniS.inputmethod

```

```

46     .MyIME"
47     NSRunningApplication.runningApplications(withBundleIdentifier: bundleID
48 )
49     .filter { $0.processIdentifier != ProcessInfo.processInfo.
50 processIdentifier }
51     .forEach { $0.terminate() }
52     default:
53     return false
54 }
55 // 命令模式是短命进程：TIS 的注册/启用变更经 XPC 异步提交，
56 // 立即退出会导致变更丢失，退出前稍作等待确保提交完成。
57 Thread.sleep(forTimeInterval: 1)
58 return true
59 }
60
61 static func prepareInstalledCopy(logger: Logger) -> Bool {
62     // Unit tests host the app from DerivedData. They must never invoke the
63     privileged
64     // self-installer or terminate the XCTest runner while it is bootstrapping.
65     if ProcessInfo.processInfo.environment["XCTestConfigurationFilePath"] !=
66     nil {
67         return true
68     }
69
70     let current = Bundle.main.bundleURL.resolvingSymlinksInPath().
71     standardizedFileURL
72     let installed = installedURL.resolvingSymlinksInPath().standardizedFileURL
73
74     if isRunningInstalledCopy {
75         return true
76     }
77
78     do {
79         try installForCurrentUser(from: current, to: installed)
80         try removeLegacySystemCopyIfPresent(logger: logger)
81         removeLegacyLoginRegistration(logger: logger)
82
83         // Register from the process the user has already approved through
84         Gatekeeper.
85         // A freshly copied development/local build may be rejected when
86         executed from
87         // /Library, which previously made the deferred registration silently
88         disappear.

```



```

80     guard register(installed, logger: logger) else {
81         presentRegistrationFailure()
82         DispatchQueue.main.async { NSApp.terminate(nil) }
83         return false
84     }
85
86     let process = Process()
87     process.executableURL = URL(fileURLWithPath: "/bin/sh")
88     process.arguments = [
89         "-c",
90         "sleep 1; /usr/bin/killall MyIME 2>/dev/null; sleep 1; "
91         + "/usr/bin/open \"(shellQuote(installed.path)) --args --post-
install\"
92     ]
93     try process.run()
94     DispatchQueue.main.async { NSApp.terminate(nil) }
95     return false
96 } catch {
97     logger.error("自动安装失败: \"(error.localizedDescription, privacy: .
public)")
98     presentInstallFailure(error)
99     DispatchQueue.main.async { NSApp.terminate(nil) }
100    return false
101 }
102 }
103
104 private static func presentInstallFailure(_ error: Error) {
105     NSApp.setActivationPolicy(.regular)
106     NSApp.activate(ignoringOtherApps: true)
107     let alert = UIAlertController()
108     alert.alertStyle = .critical
109     alert.messageText = "MyIME 自动安装失败"
110     alert.informativeText = "没有改动当前已安装的输入法。请重新打开 MyIME.app。若
正在从 1.0.8 或更早版本升级，系统会要求管理员密码以移除旧的系统级副本。\\n\\n错误：
\"(error.localizedDescription)"

```

Listing 18.1: SelfInstaller 的路径、命令和已安装副本判断

18.2 同一 Bundle ID 的多个副本

当 `/Library/InputMethods` 与 `~/Library/InputMethods` 同时存在相同 Bundle ID, `LaunchServices/TIS` 可能缓存错误副本。用户看到同名菜单项, 却不确定系统启动了哪个路径。升级时因此要检测和迁移旧的系统级副本。

18.3 App Translocation

从下载位置直接打开未正确处理的应用, macOS 可能通过随机只读路径运行它。输入法若把这个临时路径当作稳定安装位置, 重新启动和自更新会形成循环。工程代码必须只在确认已位于目标目录后建立服务器; 安装阶段完成复制后, 从确定路径启动。

```

1      alert.informativeText = "没有改动当前已安装的输入法。请重新打开 MyIME.app。若
    正在从 1.0.8 或更早版本升级, 系统会要求管理员密码以移除旧的系统级副本。\\n\\n错误:
    \\(error.localizedDescription)"
2      alert.addButton(withTitle: "好")
3      alert.runModal()
4  }
5
6  private static func presentRegistrationFailure() {
7      NSApp.setActivationPolicy(.regular)
8      NSApp.activate(ignoringOtherApps: true)
9      let alert = NSAlert()
10     alert.alertStyle = .critical
11     alert.messageText = "MyIME 已复制, 但输入法注册失败"
12     alert.informativeText = "系统没有识别 ~/Library/Input Methods/MyIME.app, 因
    此它不会出现在输入法列表中。请退出登录并重新登录一次; 如果仍未出现, 请重新运行安装
    包。"
13     alert.addButton(withTitle: "好")
14     alert.runModal()
15 }
16
17 static func ensureRegistered(logger: Logger) -> Bool {
18     register(installedURL, logger: logger)
19 }
20
21 /// 1.0.7 and earlier installed a login LaunchAgent for registration recovery.
    On recent
22 /// macOS versions it appears as an "App Background Activity" toggle. Disabling
    that item

```

```

23  /// can terminate the active IMK service and strand clients such as Notes.
    Registration is
24  /// now completed during installation; macOS owns the only process needed while
    selected.
25  static func removeLegacyLoginRegistration(logger: Logger) {
26      let label = "fudan.miniS.inputmethod.MyIME.register"
27      let agentDirectory = FileManager.default.homeDirectoryForCurrentUser
28          .appendingPathComponent("Library/LaunchAgents", isDirectory: true)
29      let agentURL = agentDirectory.appendingPathComponent("\(label).plist")
30      guard FileManager.default.fileExists(atPath: agentURL.path) else { return }
31
32      // Removing the file alone leaves the job loaded until the next login. Boot
    it out so
33      // System Settings also stops treating MyIME as a user-managed background
    activity.
34      let launchctl = Process()
35      launchctl.executableURL = URL(fileURLWithPath: "/bin/launchctl")
36      launchctl.arguments = ["bootout", "gui/\(getuid())/\(label)"]
37      try? launchctl.run()
38      launchctl.waitUntilExit()
39
40      do {
41          try FileManager.default.removeItem(at: agentURL)
42          logger.notice("已移除旧版登录注册 LaunchAgent")
43      } catch {
44          logger.error("移除旧版 LaunchAgent 失败: \(error.localizedDescription,
    privacy: .public)")
45      }
46  }
47
48  @discardableResult
49  private static func register(_ appURL: URL, logger: Logger) -> Bool {
50      guard let modes = Bundle(url: appURL)?.object(forKey: "
    ComponentInputModeDict") as? [String: Any],
51          let modeList = modes["tsInputModeListKey"] as? [String: Any] else {
    return false }
52
53      // 仅在系统尚未发现任一输入模式、或包版本变化 (Info.plist 元数据需刷新) 时才注
    册:
54      // 对已注册的输入源重复调用 TISRegisterInputSource 会清空其启用状态。
55      let versionKey = "LastRegisteredBundleVersion"
56      let bundleVersion = Bundle(url: appURL)?.infoDictionary?["CFBundleVersion"]
    as? String

```

```

57     let needsRegistration = modelList.keys.contains(where: { inputSource(withID:
    $0) == nil })
58     || UserDefaults.standard.string(forKey: versionKey) != bundleVersion
59     if needsRegistration {
60         let status = TISRegisterInputSource(appURL as CFURL)
61         if status != noErr {
62             logger.error("输入法注册失败, 状态码: \(status)")
63             return false
64         }
65         UserDefaults.standard.set(bundleVersion, forKey: versionKey)
66     }
67     // 必须先启用父输入法, 但切勿对已启用的源重复调用 TISEnableInputSource:
68     // macOS 26 会把每一次调用都视为新的敏感权限请求并再次弹出授权框。
69     if let bundleID = Bundle(url: appURL)?.bundleIdentifier {
70         let parentFilter = [kTISPropertyInputSourceID as String: bundleID] as
    CFDictionary
71         if let parents = TISCreateInputSourceList(parentFilter, true) {
72             for case let source as TISInputSource in parents.takeRetainedValue
    () as NSArray {
73                 if isEnabledInputSource(source) { continue }
74                 let parentStatus = TISEnableInputSource(source)
75                 if parentStatus != noErr {
76                     logger.error("父输入法启用失败, 状态码: \(parentStatus)")
77                 }
78             }
79         }
80     }
81     var registeredModeCount = 0
82     var enabledModeCount = 0
83     for identifier in modelList.keys {
84         let filter = [kTISPropertyInputSourceID as String: identifier] as
    CFDictionary
85         guard let result = TISCreateInputSourceList(filter, true) else {
    continue }
86         let sources = result.takeRetainedValue() as NSArray
87         guard sources.count > 0 else { continue }
88         registeredModeCount += 1
89         var enabled = false
90         for case let source as TISInputSource in sources {
91             if isEnabledInputSource(source) {
92                 enabled = true
93                 continue
94             }

```

```

95         let enableStatus = TISEnableInputSource(source)
96         if enableStatus != noErr {
97             logger.error("输入模式启用失败, 状态码: \(enableStatus)")
98         } else {
99             enabled = true
100         }
101     }
102     if enabled { enabledModeCount += 1 }
103 }
104 guard registeredModeCount == modeList.count else {
105     logger.error("输入模式注册不完整: \(registeredModeCount)/\(modeList.
count)")
106     return false
107 }
108 if enabledModeCount == modeList.count {
109     logger.notice("MyIME 输入源已经注册并启用")
110 } else {
111     // Registration controls whether MyIME appears in the add-input-source
sheet.
112     // Enabling is a separate user-authorized operation on recent macOS
versions.
113     logger.notice("MyIME 输入源已经注册, 等待用户启用: \(enabledModeCount)/\(
modeList.count)")
114 }
115 return true
116 }
117
118 private static func isEnabledInputSource(_ identifier: String) -> Bool {
119     let filter = [kTISPropertyInputSourceID as String: identifier] as
CFDictionary
120     guard let result = TISCreateInputSourceList(filter, true) else { return
false }
121     for case let source as TISInputSource in result.takeRetainedValue() as
NSArray
122         where isEnabledInputSource(source) { return true }
123     return false
124 }
125
126 private static func isEnabledInputSource(_ source: TISInputSource) -> Bool {
127     guard let pointer = TISGetInputSourceProperty(source,
kTISPropertyInputSourceIsEnabled),
128         let enabled = unsafeBitCast(pointer, to: CFBoolean?.self) else {
return false }

```

```

129         return CFBooleanGetValue(enabled)
130     }
131
132     @discardableResult
133     private static func setInputSourceEnabled(_ enabled: Bool, logger: Logger) ->
134     Bool {
135         if enabled {
136             // 走完整注册路径：父输入法未启用时单独启用子模式会静默失败。
137             return register(installedURL, logger: logger)
138         }
139         guard let source = inputSource(withID: inputModeID) else {
140             logger.error("找不到 MyIME 输入模式")
141             return false
142         }
143     }

```

Listing 18.2: 安装准备、复制替换与目标副本重启

工程警告：安装器是高风险代码

安装器会复制、替换甚至清理旧应用。必须验证目标是具体的 MyIME.app，禁止对宽泛目录递归删除；失败时保留原可用版本；不要因为“只在自己电脑测试”而忽略恢复路径。

动手实验：幂等性

连续运行安装器三次，记录输入源数量、安装路径、进程路径和用户数据库校验和。通过标准是没有重复输入源、进程始终来自目标路径、用户库不变。

思考与练习

1. 为什么 DMG 中双击应用后不应直接把该副本作为长期输入服务？
2. 设计一次失败替换后的回滚方案。
3. 如何证明系统当前启动的确实是新版本而非缓存旧副本？

本章小结

输入法安装是“复制 + 注册 + 选择 + 启动”的事务。路径稳定、单副本、幂等和失败恢复比漂亮的安装窗口更重要。

InputMethodKit 的多条事件路径

学习目标

识别 `handle`、`inputText` 与文本数据模式；理解不同客户端为何表现不同；能够在不复制业务逻辑的前提下统一事件处理。

真实输入法常见现象是 Notes 正常、某个 WebKit 页面失败，或者开发机正常、目标机无法输入中英文。根源未必在词库，而可能是 IMKServer 的输入模式、客户端协议实现和事件投递路径不同。

MyIME 1.0.10 对“text data only”投递做了适配：系统可能不调用常见的 `handle(_:client:)` 或 `inputText(_:client:)`，而通过包含 Unicode、硬件键码和修饰符的文本数据入口提供事件。

```
1  /// IMKServerInput "text data only" delivery mode. In that mode InputMethodKit
   does not
2  /// call handle(_:client:) or inputText(_:client:); it supplies Unicode,
   hardware key code,
3  /// and modifiers through this four-argument callback instead.
4  override func inputText(
5      _ string: String!,
6      key keyCode: Int,
7      modifiers flags: Int,
8      client sender: Any!
9  ) -> Bool {
10     guard let string,
11           let client = textClient(from: sender),
12           (0...Int(UInt16.max)).contains(keyCode) else { return false }
13     activeClient = client
14
15     let modifiers = NSEvent.ModifierFlags(rawValue: UInt(flags))
16     if let event = NSEvent.keyEvent(
17         with: .keyDown,
```

```

18         location: .zero,
19         modifierFlags: modifiers,
20         timestamp: 0,
21         windowNumber: 0,
22         context: nil,
23         characters: string,
24         charactersIgnoringModifiers: string.lowercased(),
25         isARepet: false,
26         keyCode: UInt16(keyCode)
27     ) {
28         logger.debug("已通过文本数据回退路径处理输入: keyCode=\(keyCode)")
29         return safelyHandle(event, client: client)
30     }
31     return inputText(string, client: sender)
32 }
33
34 override func didCommand(by aSelector: Selector!, client sender: Any!) -> Bool
35 {
36     guard let aSelector, let client = textClient(from: sender) else { return
37     false }
38     activeClient = client
39     switch NSStringFromSelector(aSelector) {
40     case "deleteBackward:":
41         guard !raw.isEmpty else {
42             if showingSuggestions {
43                 clearSuggestions()
44                 return true
45             }
46             return false
47         }
48     case "cancelOperation:":
49         guard !raw.isEmpty || showingSuggestions else { return false }
50         reset(client)
51         return true
52     case "insertNewline:", "insertLineBreak:":
53         guard !raw.isEmpty || showingSuggestions else { return false }
54         if showingSuggestions, raw.isEmpty {
55             commitSuggestion(highlighted, client: client)
56         } else if hasNavigatedCandidates {
57             commitHighlighted(client)
58         }
59     }
60 }

```



```

59         } else {
60             commit(composedPhrase + raw, pinyin: [], client: client, learn:
false)
61         }
62         return true
63     case "moveLeft:":
64         return moveHighlight(-1)
65     case "moveRight:":
66         return moveHighlight(1)
67     case "moveDown:":
68         return moveDown()
69     case "moveUp:":
70         return moveUp()
71     case "pageUp:":
72         return movePage(-1)
73     case "pageDown:":
74         return movePage(1)
75     default:
76         return false
77     }
78 }
79
80 override func deactivateServer(_ sender: Any!) {
81     engine.breakPhraseLearningContext()
82     inputModeIndicator.hide()
83     candidateWindow.hide()
84     shiftWasDown = false
85     shiftUsedWithKey = false
86     defer {
87         if Self.activatedController === self {
88             Self.activatedController = nil
89         }
90         activeClient = nil
91     }
92     guard let client = textClient(from: sender) else {
93         super.deactivateServer(sender)
94         return
95     }
96     if !raw.isEmpty || !composedPhrase.isEmpty {
97         commit(composedPhrase + raw, pinyin: [], client: client, learn: false)
98     }
99     super.deactivateServer(sender)
100 }

```

```

101
102     override func hidePalettes() {
103         inputModeIndicator.hide()
104         candidateWindow.hide()
105         super.hidePalettes()
106     }
107
108     static func finalizeStrandedCompositionIfNeeded() {
109         guard !SelfInstaller.isCurrentInputSource,
110             let controller = activatedController else { return }
111         if let client = controller.activeClient {
112             controller.deactivateServer(client)
113         } else {
114             controller.hidePalettes()
115             activatedController = nil
116         }
117     }

```

Listing 19.1: 文本数据投递路径与统一按钮处理

19.1 统一语义，不统一假设

三条入口最终应转换成相同的语义动作，但不能假设每条入口都提供完全相同的信息。NSEvent 可能包含键盘布局后的字符与 flags；文本数据可能只含特定字段；安全输入环境可能限制内容。适配层应明确哪些字段缺失，以及缺失时选择放行还是降级处理。

19.2 客户端协议差异

原生 NSTextView、浏览器网页输入框和 Electron 编辑器都声称支持文本输入协议，但对 replacementRange、marked range、selected range 和窗口坐标的实现细节可能不同。避免依赖“恰好在 Notes 可用”的未声明行为，例如：

- 不应假设 marked range 永远有效；
- replacementRange 未知时使用 NSNotFound 语义；
- 客户端切换后不复用旧对象；
- 坐标查询失败时隐藏候选窗而不是崩溃；

- 提交和清空顺序在所有路径保持一致。

动手实验：事件路径日志

为三个入口分别增加只记录方法名、键码和修饰符的 Debug 日志。在 Notes、Safari、Chrome 和 Electron 应用中输入同一序列，形成对照表。Release 构建不得记录完整文本。

思考与练习

1. 为什么“某方法没有被调用”不能直接解释为输入法进程没有运行？
2. 如何避免三条入口复制三份退格与选词逻辑？
3. 客户端不给有效插入点坐标时，候选窗口应怎样降级？

本章小结

InputMethodKit 适配的核心是把多条不完全等价的系统入口转换成统一动作，同时保留缺失信息和客户端差异的证据。

组合会话、失活恢复与幽灵状态

学习目标

理解输入源切换、应用切换和控制器失活对组合状态的影响；能够避免幽灵候选、跨应用误提交和旧客户端引用；设计清晰的恢复策略。

输入法状态不仅被按键改变，也被系统生命周期改变。用户可以在组合中点击别的窗口、切换输入源、锁屏、结束进程或打开设置。若控制器只在 `Space` 和 `Esc` 时清理状态，就会留下“幽灵组合”。

MyIME 对 `commitComposition`、失活和输入源变化分别处理，并保存弱引用以便最终清理滞留组合。

```
1  override func commitComposition(_ sender: Any!) {
2      guard let client = textClient(from: sender) else { return }
3      if !raw.isEmpty || !composedPhrase.isEmpty {
4          commit(composedPhrase + raw, pinyin: [], client: client, learn: false)
5      }
6  }
7
8  private func safelyHandle(_ event: NSEvent, client: IMKTextInput) -> Bool {
9      if event.type == .flagsChanged, event.keyCode == 56 || event.keyCode == 60
10     {
11         let shiftIsDown = event.modifierFlags.contains(.shift)
12         if shiftIsDown, !shiftWasDown {
13             shiftUsedWithKey = false
14         }
15         if shiftWasDown, !shiftIsDown {
16             if !shiftUsedWithKey {
17                 if !raw.isEmpty { commit(composedPhrase + raw, pinyin: [],
18 client: client, learn: false) }
19             }
20             shiftUsedWithKey = false
21         }
22     }
```

```

20         shiftWasDown = shiftIsDown
21         return false
22     }
23     guard event.type == .keyDown else { return false }
24     let modifiers = event.modifierFlags.intersection(.
deviceIndependentFlagsMask)
25     if modifiers.contains(.command) || modifiers.contains(.control) ||
modifiers.contains(.option) { return false }
26     if modifiers.contains(.shift) {
27         shiftUsedWithKey = true
28     }
29     switch event.keyCode {
30     case 49: // Space
31         if raw.isEmpty {
32             if showingSuggestions, !output.candidates.isEmpty {
33                 commitSuggestion(highlighted, client: client)
34                 return true
35             }
36             engine.breakPhraseLearningContext()
37             clearSuggestions()
38             return false
39         }
40         commitHighlighted(client)
41         return true
42     case 36, 76: // Return / keypad enter
43         if raw.isEmpty {
44             if showingSuggestions, !output.candidates.isEmpty {
45                 commitSuggestion(highlighted, client: client)
46                 return true
47             }
48             return false
49         }
50         if hasNavigatedCandidates {
51             commitHighlighted(client)
52         } else {
53             commit(composedPhrase + raw, pinyin: [], client: client, learn:
false)
54         }
55         return true
56     case 51: // Backspace
57         if raw.isEmpty {
58             if showingSuggestions {
59                 clearSuggestions()

```

```

60         return true
61     }
62     return false
63 }
64 raw.removeLast()
65 refresh(client)
66 return true
67 case 53: // Escape
68     if raw.isEmpty {
69         if showingSuggestions {
70             clearSuggestions()
71             return true
72         }
73         return false
74     }
75     reset(client)
```

Listing 20.1: 提交组合、控制器失活和会话清理

20.1 策略矩阵

事件	推荐默认	风险
按 Esc	取消	用户预期明确
按 Return	提交当前选择/原文	需区分导航状态
切换输入源	提交或取消，保持一致	旧 marked text 滞留
客户端失活	结束当前组合	不能提交到新客户端
进程异常结束	由系统清理	文档可能保留显示残影

没有一种策略适合所有产品，但策略必须一致、可测试并写入用户说明。尤其不能让“失活时提交首选”和“输入源切换时取消”在用户无感知的情况下互相矛盾。

20.2 弱引用与所有权

输入法不拥有客户端生命周期。长时间强引用旧客户端可能导致泄漏或错误调用。弱引用允许控制器在必要时清理，同时接受客户端已经消失。所有回调都应在

使用前重新确认客户端能力。

动手实验：幽灵状态测试

输入 `nihao` 但不上屏，依次执行：切换窗口、切换输入源、关闭文档、强制结束进程。每个场景重新激活 Notes，确认候选窗和 `raw` 不会复活，也不会把“你好”写入错误文档。

思考与练习

1. 失活时提交原始拼音和取消组合各有什么优缺点？
2. 为什么全局静态变量保存“当前客户端”风险很高？
3. 设计一个可用状态转移表自动生成的生命周期测试。

本章小结

输入法组合是一段与客户端绑定的会话。生命周期事件必须显式结束会话，旧状态和旧客户端都不能跨越新的焦点边界。

日志、诊断与证据链

学习目标

建立分层日志类别；掌握从输入源状态到候选生成的证据链；在保护隐私的前提下定位“无法输入”和“候选错误”。

输入法错误经常发生在用户看不到的后台进程。弹窗会抢焦点，`print` 输出又未必容易找到，因此应使用统一日志系统，并按 `bootstrap`、`installer`、`controller`、`engine`、`store` 分类。

21.1 用 Logger 建立可过滤日志

统一日志的首要价值不是“多打印一些文字”，而是把来源、严重级别和隐私属性变成结构化字段。所有模块使用相同 `subsystem`，再用 `category` 区分职责：

```
1 import OSLog
2
3 private let log = Logger(
4     subsystem: "fudan.miniS.MyIME",
5     category: "controller"
6 )
7
8 log.debug("keyDown keyCode=\(event.keyCode) session=\(sessionID, privacy: .public)"
9 )
10 log.info("candidates count=\(candidates.count) latencyMs=\(elapsedMs)")
11 log.error("store unavailable code=\(errorCode, privacy: .public)")
```

Listing 21.1: 结构化、可过滤且默认脱敏的统一日志

只有不含正文、不会识别个人的字段才标记 `.public`。原始拼音、候选文本和提交句子保持 `private` 或根本不写入；`Debug` 构建也不应形成“先泄露、发布前再删除”的习惯。

21.2 Console.app 过滤 SOP

打开“应用程序 → 实用工具 → 控制台”，选中当前 Mac 并点击“开始流式传输”。在搜索框输入 `fudan.miniS.MyIME`，把匹配项类型改成 **Subsystem**；需要缩小范围时，再加 **Category** 为 `controller`、`bootstrap` 或 `installer`。打开“包括信息信息”和“包括调试信息”，清空旧视图后复现一次，再立即暂停。图中的蓝色 SUBSYSTEM 令牌说明过滤条件已被 Console 识别，而不是普通全文搜索。

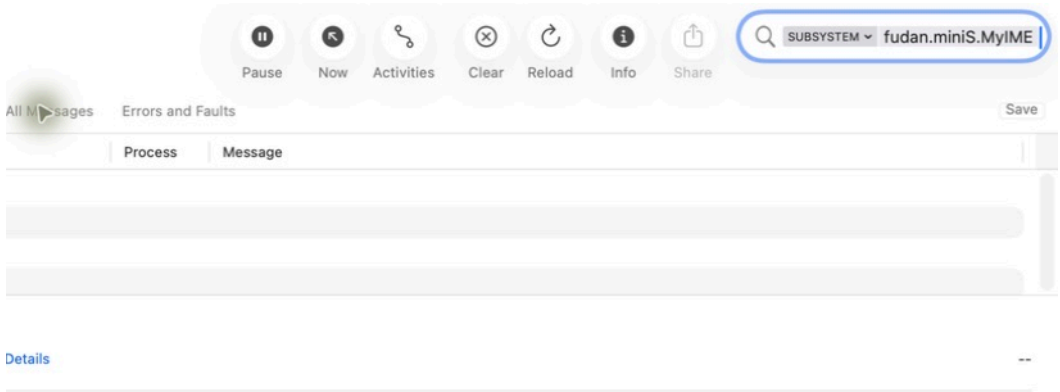


图 21.1: Console.app 以 subsystem 过滤 MyIME 日志；截图已裁去机器名称

一轮复现应形成以下最小证据链：`bootstrapserverready` → `controllerkeyDown` → `enginecandidates=N` → `clientmarked/insert`。若第一项都没有，查进程和 `IMKServer`；有第一项而没有第二项，查当前输入源与事件入口；有候选却没有客户端调用，查控制器状态和事件返回值。导出日志前必须再次搜索正文样例和用户名，确认脱敏。

21.3 诊断问题树

面对“没有 MyIME”时依次问：

1. 安装目录是否存在应用，版本和签名是什么？

2. TIS 是否枚举到目标输入源 ID?
3. 当前输入源 ID 是否为 MyIME?
4. 进程路径是否来自安装目录? IMKServer 是否建立?
5. 按键进入哪个回调? 控制器返回了什么?
6. 引擎输出候选是否为空? 资源库是否完整?
7. 客户端收到 marked text/insert text 了吗?

面对“候选不对”则从第 5 步开始：归一化 raw、切分路径、SQL 召回、分数分解、后处理和 UI 映射。

21.4 隐私友好的日志

Release 日志可记录：输入长度、路径数量、候选数量、耗时、错误码、应用 Bundle ID 和状态转移；不应默认记录用户完整 raw、候选文本、剪贴板或已提交句子。必要的用户诊断应采用明确同意、短时开启和本地导出。

核心概念：关联 ID

为一次输入会话生成短关联 ID，使 bootstrap、controller、engine 和 store 日志能够串联，而无需写出正文。例如：`session=7F2A,rawLength=8,paths=3,candidates=25,p95Bucket=20ms`。

21.5 死锁与卡顿调试 SOP

先分症状。崩溃、数据竞争、逻辑死锁和主线程长任务是四类问题，不能只靠同一个工具。复现前固定输入、客户端、系统版本和构建号，并保存正常基线。

1. 在 Xcode 的 Scheme→Run→Diagnostics 打开 Thread Sanitizer (TSan)，用 Debug 构建重放并发选择、异步学习和数据库故障。TSan 擅长发现未同步的共享内存访问，但不能证明所有逻辑死锁都不存在。命令行可用 `xcodebuild ...-enableThreadSanitizerYES`。TSan 会显著拖慢程序，只用于诊断构建。¹

¹Apple: <https://developer.apple.com/documentation/xcode/diagnosing-memory-thread-and-crash-issues-early>

2. 若界面已经卡住，在 Xcode 点击 Pause，展开所有线程。记录主线程调用栈、持锁线程和等待栈；若线程 A 等待由 A 自己持有的非递归 `NSLock`，就是重入；若 A 等 B、B 又等 A，则画出等待环。不要在锁内调用客户端、SQLite、日志插值闭包或未知回调。
3. 打开 Main Thread Checker，捕获从后台线程触碰 `NSPanel`/`SwiftUI` 的路径。它检查 UI 线程契约，和 TSan 的数据竞争检查互补。
4. 对“能动但很慢”选择 Product→Profile→Time Profiler。只录制 10–20 秒：先空闲 3 秒，再输入固定长句，最后空闲 3 秒。按进程过滤 MyIME，展开 Main Thread；重点查 `sqlite3_step`、锁等待、候选排序和 SwiftUI 布局是否占据热栈。Apple 也建议用 Time Profiler 分析无响应与挂起。²
5. 修改后用相同机器、输入和采样窗口复测，比较 P50/P95 与主线程最重栈，而不是只报告“感觉顺了”。再关闭 Sanitizer 运行一次 Release 配置，排除诊断开销造成的假象。

工程警告：NSLock 重入的典型陷阱

线程在持有 `NSLock` 时调用一个看似只读的辅助函数，而该函数为了读取同一状态再次加锁，会永久等待。把锁保护的不变量写在字段旁；锁内只复制/更新小状态，解锁后再调用数据库、客户端和 UI。

21.6 诊断工具

仓库提供输入源检查、注册和本地安装工具。工具输出应被视作辅助证据，而不是绕过正常系统路径的永久依赖。正式版不能靠一个登录守护进程不断强制恢复，因为这会隐藏生命周期设计缺陷。

```

1 import Carbon
2 import Foundation
3
4 guard CommandLine.arguments.count == 2 else {
5     fputs("usage: inspect_input_source.swift bundle.identifier\n", stderr)
6     exit(2)
7 }
8
9 let bundleID = CommandLine.arguments[1]
```

²Apple: <https://developer.apple.com/documentation/xcode/improving-your-app-s-performance/>

```

10 let filter = [kTISPropertyBundleID as String: bundleID] as CFDictionary
11 guard let result = TISCreateInputSourceList(filter, true) else { exit(1) }
12 let sources = result.takeRetainedValue() as NSArray
13
14 func property(_ source: TISInputSource, _ key: CFString) -> Any {
15     guard let pointer = TISGetInputSourceProperty(source, key) else { return "<nil>" }
16     return Unmanaged<AnyObject>.fromOpaque(pointer).takeUnretainedValue()
17 }
18
19 for case let source as TISInputSource in sources {
20     print("ID: \(property(source, kTISPropertyInputSourceID))")
21     print("Name: \(property(source, kTISPropertyLocalizedString))")
22     print("Category: \(property(source, kTISPropertyInputSourceCategory))")
23     print("Type: \(property(source, kTISPropertyInputSourceType))")
24     print("Mode: \(property(source, kTISPropertyInputModeID))")
25     print("Languages: \(property(source, kTISPropertyInputSourceLanguages))")
26     print("Enabled: \(property(source, kTISPropertyInputSourceIsEnabled))")
27     print("Select capable: \(property(source, kTISPropertyInputSourceIsSelectCapable))")
28     print("----")
29 }
30
31 if let current = TISCopyCurrentKeyboardInputSource()?.takeRetainedValue() {
32     print("Current: \(property(current, kTISPropertyInputSourceID))")
33 }

```

Listing 21.2: 输入源检查工具示例

动手实验：故障注入

分别制造词库缺失、用户库只读、错误输入源 ID 和候选坐标失败。为每个故障写出预期日志和用户可见降级，验证不会让所有按键失效。

思考与练习

1. 日志中记录 raw 的 SHA-256 是否绝对安全？为什么？
2. 设计一个一键生成的本地诊断报告目录结构。
3. “进程存在但没有回调日志”下一步应收集什么证据？

本章小结

好的诊断系统把模糊故障变成逐层证据链，并把可观察性与隐私同时纳入设计，而不是发生问题后临时加 `print`。

隐私、安全与词库许可证

学习目标

理解输入法的高敏感权限；能够建立完全离线的威胁模型；识别安全输入、日志、更新包、用户数据库和第三方词库中的风险。

输入法位于所有文本之前，理论上可能接触私人通信、搜索、地址和代码。即使产品承诺离线，也必须用架构和可验证事实支持承诺：没有网络依赖、没有隐蔽上传路径、日志不保存正文、更新包经过签名、用户库权限正确。

22.1 威胁模型

至少考虑以下资产与攻击面：

- 用户原始按键、候选选择和提交文本；
- 用户词库与上下文数据库；
- 应用内置系统词库和替换词库入口；
- 安装器的文件替换权限；
- 发布服务器、GitHub Release 和 DMG 供应链；
- 调试日志、崩溃报告与课堂演示录像。

完全离线可以显著缩小攻击面，但不能自动解决本地恶意软件、宽松文件权限、未签名更新和敏感日志。用户数据库应存放在当前用户目录，避免其他普通账户读取；导出时应明确提示内容敏感。

22.2 安全输入

密码字段可能启用 Secure Event Input 或拒绝第三方组合。输入法应尊重该边界，不尝试绕过；无法输入时回退系统安全键盘是正确行为。教学演示禁止在真实密码框记录事件日志。

22.3 开源与词库许可

输入法代码开源不等于所有聚合词库都可以任意再分发。MyIME 的流水线记录来源锁、许可证和派生步骤，应用内附第三方声明。

```
1 # Dictionary source licenses
2
3 The bundled `system.sqlite` is an adapted offline dictionary built on 2026-08-08
4   from the
5 following pinned sources. MyIME removes tone numbers, uses the simplified word form
6   , combines
7 frequency values, and compiles the result into SQLite.
8
9 ## CC-CEDICT
10
11 - Publisher: MDBG
12 - Source: https://www.mdbg.net/chinese/dictionary?page=cc-cedict
13 - Artifact date: 2026-08-07T08:04:28Z
14 - License: Creative Commons Attribution-ShareAlike 4.0 International
15 - License text: https://creativecommons.org/licenses/by-sa/4.0/
16 - Copyright referenced by upstream: CEDICT, Copyright 1997 - 1998 Paul Andrew
17   Denisowski
18
19 The adapted dictionary data is distributed under CC BY-SA 4.0. The attribution and
20   ShareAlike
21 terms apply to the dictionary data, not to independently written MyIME program code
22   .
23
24 ## jieba dictionary frequencies
25
26 - Project: jieba by Sun Junyi
27 - Source: https://github.com/fxsjy/jieba
28 - Revision: `67fa2e36e72f69d9134b8a1037b83fbb070b9775`
29 - License: MIT
```

```
26 Copyright (c) 2013 Sun Junyi
27
28 Permission is hereby granted, free of charge, to any person obtaining a copy of
   this software and
29 associated documentation files (the "Software"), to deal in the Software without
   restriction,
30 including without limitation the rights to use, copy, modify, merge, publish,
   distribute,
31 sublicense, and/or sell copies of the Software, and to permit persons to whom the
   Software is
32 furnished to do so, subject to the conditions in the upstream MIT license.
33
34 THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED.
35
36 ## Rime Ice dictionary
37
38 - Project: Rime Ice by iDvel and contributors
39 - Source: https://github.com/iDvel/rime-ice
40 - Revision: `569ff3bc65dd4aec0a26b33c49c8bbdfa8b5fd57`
41 - Included data: `cn_dicts/base`, `ext`, `others`, and `8105`
42 - License: GNU General Public License version 3
43
44 MyIME converts the selected Rime Ice entries to its SQLite schema and marks this as
   modified
45 dictionary data. The corresponding source revision, conversion script, and exact
   checksums are
46 listed in `tools/SOURCES.lock`. A copy of GPL-3.0 is bundled with the application.
47
48 ## fcitx5-pinyin-zhwiki
49
50 - Source: https://github.com/felixonmars/fcitx5-pinyin-zhwiki
51 - Revision: `cb1073ce2042ab431c872da7699d62ed8b857cff`
52 - Artifact: `zhwiki-20260416.dict.yaml`
53 - Repository license: Unlicense
54 - Generated dictionary terms follow the Wikimedia source terms described at
   https://dumps.wikimedia.org/legal.html
55
56
57 ## Jinghang Dictionary
58
59 - Source: https://github.com/kkhkl/Jinghang-Dictionary
60 - Revision: `4ee6f087f33ef5e6cede360277930e296881f11e`
61 - License: Jinghang custom license 2025 - 2026.5
62
```



```

63 MyIME includes a clearly identified derived data set, not an unmodified or official
    Jinghang
64 release. The upstream terms permit derivative data with that identification and
    attribution.
65
66 ## renfei/dict
67
68 - Copyright (c) 2022 Ren Fei
69 - Source: https://github.com/renfei/dict
70 - Revision: `f85efb9ffc788c7d79112448aed11b2225b3e0ac`
71 - License: MIT
72
73 ## THUOCL (Tsinghua Open Chinese Lexicon)
74
75 - Copyright (c) 2018 THUNLP
76 - Source: https://github.com/thunlp/THUOCL
77 - Revision: `a30ce79d895d01ab5132a5c74c29703ff7efb4cc`
78 - License: MIT
79
80 Permission is hereby granted, free of charge, to any person obtaining a copy of
    this software and
81 associated documentation files (the "Software"), to deal in the Software without
    restriction,
82 including without limitation the rights to use, copy, modify, merge, publish,
    distribute,
83 sublicense, and/or sell copies of the Software, and to permit persons to whom the
    Software is
84 furnished to do so, subject to inclusion of the copyright and permission notice.
85
86 THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED.
87
88 ## NLP Chinese Corpus
89
90 - Project: NLP Chinese Corpus by Bright Xu / brightmart
91 - Source: https://github.com/brightmart/nlp\_chinese\_corpus
92 - Revision: `5dc872154777d24c897f957f1ec848fc7fd59aa9`
93 - DOI: https://doi.org/10.5281/zenodo.3402023
94 - Included training sources: wiki2019zh, news2016zh, baike2018qa, webtext2019zh,
95 and the Chinese side of translation2019zh

```

Listing 22.1: 词库来源与许可清单的一部分

课堂项目应为每个来源记录：名称、URL、版本/提交、许可证、是否修改、输

出中保留什么以及再分发义务。无法确认许可的词表不得进入正式发布包。

动手实验：离线审计

在干净用户账户中运行输入法，使用系统网络观察工具检查进程连接；搜索源码中的 `URLSession`、`Network` 和 `socket` 调用；检查 Release 包中第三方声明与词库来源是否一致。

思考与练习

1. “不上传击键”与“完全没有网络请求”有什么区别？
2. 用户主动导出词库是否改变应用的隐私责任？
3. 为可下载的离线词库更新包设计签名验证流程。

本章小结

输入法的隐私承诺必须由网络边界、日志策略、文件权限、供应链和许可证记录共同支撑。离线是强优势，但不是安全证明的终点。

测试体系：从内存词库到跨机器验收

学习目标

建立单元、集成、性能和人工验收四层测试；掌握 golden、性质测试、故障注入和确定性；把历史故障转化为回归用例。

输入法核心适合大量自动化测试，系统集成则需要少量高价值人工测试。测试金字塔底层包含切分、排序、用户学习和数据库；中层构建应用并检查资源、签名和架构；顶层在真实客户端敲固定序列。

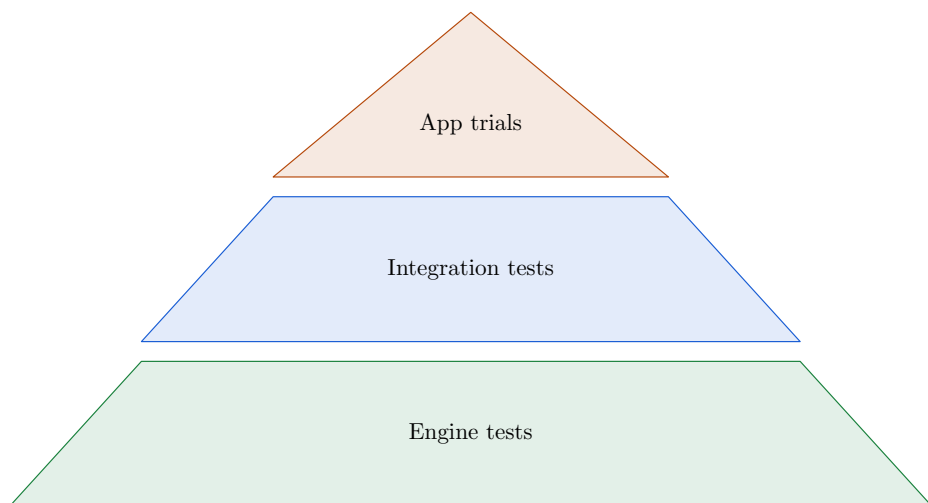


图 23.1: 自动化与人工测试的合理比例

23.1 行为测试

MyIME 的内存词库让测试只包含与行为有关的词。Golden 测试锁定 `nihao`→“你好”等核心首选；上下文测试故意让“田七”静态权重更高，再由“今天”上下文把“天

气”提升回来。

```

1      #expect(candidates.first?.word == "医院")
2  }
3
4  @Test func composesShortSentenceFromDictionaryPieces() {
5      let output = Engine(store: store).update("woshizhongguoren", prefs:
6      EnginePrefs())
7      #expect(output.candidates.first?.word == "我是中国人")
8      #expect(output.candidates.first?.consumedLength == "woshizhongguoren".count
9      )
10     }
11
12     @Test func completePhraseOutranksFrequentPartialCandidate() {
13         let phraseStore = MemoryStore(entries: [
14             StoreEntry(id: 40, word: "爱", pinyin: "ai", weight: 65_535),
15             StoreEntry(id: 41, word: "爱死你了", pinyin: "ai'si'ni'le", weight: 41
16             _247),
17         ])
18         let output = Engine(store: phraseStore).update("aisinile", prefs:
19         EnginePrefs())
20         #expect(output.candidates.first?.word == "爱死你了")
21     }
22
23     @Test func offersMultipleFullSentenceCandidates() {
24         let output = Engine(store: store).update("jintiantianqihenhao", prefs:
25         EnginePrefs())
26         #expect(output.candidates.prefix(2).map(\.word) == ["今天天气很好", "今天田
27         七很好"])
28         #expect(output.candidates.prefix(2).allSatisfy { $0.consumedLength == "
29         jintiantianqihenhao".count })
30     }
31
32     @Test func sentenceCompositionUsesWordContext() {
33         let contextStore = ContextStore(entries: [
34             StoreEntry(id: 30, word: "今天", pinyin: "jin'tian", weight: 65_535),
35             StoreEntry(id: 31, word: "天气", pinyin: "tian'qi", weight: 35_000),
36             StoreEntry(id: 32, word: "田七", pinyin: "tian'qi", weight: 65_535),
37             StoreEntry(id: 33, word: "很好", pinyin: "hen'hao", weight: 65_535),
38         ])
39         let output = Engine(store: contextStore).update("jintiantianqihenhao",
40         prefs: EnginePrefs())
41         #expect(output.candidates.first?.word == "今天天气很好")

```

```

34     }
35
36     @Test func suggestionsFollowCommittedWord() {
37         let predictingStore = PredictingStore(
38             entries: [
39                 StoreEntry(id: 50, word: "今天", pinyin: "jin'tian", weight: 65_535
40             ),
41                 StoreEntry(id: 51, word: "天气", pinyin: "tian'qi", weight: 65_535)
42             ],
43             predictions: ["今天": [StoreEntry(id: 52, word: "天气", pinyin: "",
44 weight: 60_000)]]
45         )
46         let engine = Engine(store: predictingStore)
47         engine.commitLearning(word: "今天", pinyin: ["jin", "tian"])
48         #expect(engine.suggestions(limit: 5).map(\.word).contains("天气"))
49     }
50
51     @Test func suggestionsSuppressContentFarmAndPreferSeeds() {
52         let predictingStore = PredictingStore(
53             entries: [
54                 StoreEntry(id: 50, word: "今天", pinyin: "jin'tian", weight: 65_535
55             ),
56                 ],
57             predictions: [
58                 "今天": [
59                     StoreEntry(id: 60, word: "小编", pinyin: "", weight: 65_535),
60                     StoreEntry(id: 61, word: "给大家", pinyin: "", weight: 60_000),
61                     StoreEntry(id: 62, word: "凌晨", pinyin: "", weight: 20_000),
62                 ],
63                 "天": [
64                     StoreEntry(id: 63, word: "的话", pinyin: "", weight: 65_535),
65                 ],
66             ]
67         )
68         let engine = Engine(store: predictingStore)
69         engine.commitLearning(word: "今天", pinyin: ["jin", "tian"])
70         let words = engine.suggestions(limit: 5).map(\.word)
71         #expect(words.first == "天气")
72         #expect(!words.contains("小编"))
73         #expect(!words.contains("给大家"))
74         #expect(!words.contains("的话"))
75     }

```

Listing 23.1: 整句、上下文与续写测试

23.2 性质与故障测试

固定样例覆盖已知场景，性质测试覆盖不变量。例如对随机 ASCII 字符串：引擎不崩溃、耗时不超过预算、消费长度不越界、同输入与同状态结果确定。对损坏数据库：引擎降级，不阻塞基本输入。

23.3 性能测试

性能断言应在可控环境执行，避免把 CI 机器偶发负载当作算法回归。更稳健的做法是记录分布并设置宽松硬门槛，同时在版本间比较相对变化。冷启动、热查询和长句需要分别统计。

23.4 跨机器门禁

发布前必须在不含开发证书、不含旧输入法副本的第二台 Mac 上，从 GitHub Release 下载 DMG，完成安装、注册和 Notes 输入。开发机成功不能替代跨机器验收，因为钥匙串、LaunchServices 缓存和旧构建都可能掩盖问题。

动手实验：把 bug 变成测试

选择一次历史故障，例如进程被结束后不能恢复、换机后输入源不出现或 Notes 无法组合。先写最小复现，再决定哪一部分能自动化，哪一部分保留为发布清单。

思考与练习

1. Golden 测试过多会带来什么维护问题？
2. 如何测试候选排序确定性？
3. 为什么跨机器验收属于发布门禁，而不是“有空再测”？

本章小结

测试体系要让算法快速回归、系统问题可复现、发布风险可阻断。每个真实故障都应留下自动化测试或明确人工门禁。

签名、公证、Universal 2 与 DMG 发布

学习目标

理解开发签名、Developer ID、公证和装订的区别；能够解释 Universal 2；掌握可重复发布流水线与校验和。

能在开发机运行的应用不等于可以跨机器分发。公开 DMG 应使用 Developer ID Application 证书签名，提交 Apple 公证，成功后把票据装订到应用或镜像，并验证 Gatekeeper 接受。



图 24.1: 从测试到公开 DMG 的发布门禁

24.1 签名身份

Apple Development 证书用于开发调试；Developer ID Application 用于 App Store 外分发。Ad-hoc 签名只证明代码页一致，不建立公开开发者信任。发布脚本必须主动拒绝把 development 或 ad-hoc 构建标为正式版。

24.2 Universal 2

Universal 2 应用同时包含 arm64 和 x86_64 代码。仅把文件名写成 universal 不够，需要用 `lipo-info` 或 `file` 检查主可执行文件和嵌入的本地库。SQLite 等系统动态库通常由系统提供，但自带二进制依赖也必须双架构。

24.3 发布脚本

MyIME 的脚本按顺序运行引擎测试、词库验证、构建、架构检查、签名、DMG、公证、装订和 SHA-256。顺序本身就是安全性质：任何前置门禁失败都不应继续发布。

```

1  #!/bin/sh
2  set -eu
3
4  SCRIPT_DIR=$(CDPATH= cd -- "$(dirname -- "$0")" && pwd)
5  ROOT_DIR=$(CDPATH= cd -- "$SCRIPT_DIR/.." && pwd)
6  PROJECT="$ROOT_DIR/MyIME/MyIME.xcodeproj"
7  PLIST="$ROOT_DIR/MyIME/MyIME/Resources/Info.plist"
8  DATABASE="$ROOT_DIR/MyIME/MyIME/Resources/system.sqlite"
9  OUTPUT_DIR=${MYIME_OUTPUT_DIR:-"$ROOT_DIR/dist"}
10 ALLOW_ADHOC=${MYIME_ALLOW_ADHOC:-0}
11 ALLOW_DEVELOPMENT=${MYIME_ALLOW_DEVELOPMENT:-0}
12 SIGNING_IDENTITY=${MYIME_SIGNING_IDENTITY:-}
13 NOTARY_PROFILE=${MYIME_NOTARY_PROFILE:-}
14
15 VERSION=$(/usr/libexec/PlistBuddy -c 'Print :CFBundleShortVersionString' "$PLIST")
16 BUILD=$(/usr/libexec/PlistBuddy -c 'Print :CFBundleVersion' "$PLIST")
17 WORK_DIR=$(mktemp -d "${TMPDIR:-/tmp}/MyIME-release.XXXXXX")
18 trap '/bin/rm -rf "$WORK_DIR"' EXIT HUP INT TERM
19
20 find_identity() {
21     identity=$(security find-identity -v -p codesigning 2>/dev/null \
22         | sed -n 's/.*"\(Developer ID Application:[^"]*\)".*/\1/p' \
23         | head -n 1)
24     printf '%s' "$identity"
25 }
26
27 if [ -z "$SIGNING_IDENTITY" ]; then
28     SIGNING_IDENTITY=$(find_identity)
29 fi
30
31 case "$SIGNING_IDENTITY" in
32     "Developer ID Application:*)" SIGNATURE_LABEL="developer-id" ;;
33     "Apple Development:*)"
34         if [ "$ALLOW_DEVELOPMENT" != "1" ]; then
35             echo "error: Apple Development signatures are not valid for public
distribution." >&2

```

```

36         echo "Use Developer ID Application, or set MYIME_ALLOW_DEVELOPMENT=1
for a registered test Mac only." >&2
37         exit 2
38     fi
39     SIGNATURE_LABEL="apple-development"
40     ;;
41     "") SIGNATURE_LABEL="local-adhoc" ;;
42     *) SIGNATURE_LABEL="signed" ;;
43 esac
44 ARTIFACT_SUFFIX=""
45 if [ -z "$SIGNING_IDENTITY" ]; then
46     if [ "$ALLOW_ADHOC" != "1" ]; then
47         echo "error: no usable Apple code-signing identity is available." >&2
48         echo "Install/import the certificate private key, or set MYIME_ALLOW_ADHOC
=1 for a local test build." >&2
49         exit 2
50     fi
51     ARTIFACT_SUFFIX="-local-adhoc"
52 elif [ "$SIGNATURE_LABEL" = "apple-development" ]; then
53     ARTIFACT_SUFFIX="-development"
54 elif [ "$SIGNATURE_LABEL" != "developer-id" ]; then
55     ARTIFACT_SUFFIX="-signed"
56 fi
57
58 if [ "$SIGNATURE_LABEL" = "developer-id" ] && [ -z "$NOTARY_PROFILE" ]; then
59     echo "error: public distribution requires Apple notarization." >&2
60     echo "Set MYIME_NOTARY_PROFILE to a notarytool keychain profile." >&2
61     exit 2
62 fi
63
64 echo "==> Testing IME engine"
65 # The bundled-dictionary performance checks share the same SQLite resource.
66 # Swift Testing otherwise runs them concurrently and turns host load into a flaky
P95 failure.
67 swift test --package-path "$ROOT_DIR/MyIME/IMEKit" --no-parallel
68
69 echo "==> Verifying bundled dictionary"
70 python3 "$SCRIPT_DIR/verify.py" "$DATABASE"
71
72 echo "==> Building MyIME $VERSION ($BUILD)"
73 xcodebuild \
74     -project "$PROJECT" \
75     -scheme MyIME \

```

```

76     -configuration Release \
77     -destination 'generic/platform=macOS' \
78     -derivedDataPath "$WORK_DIR/DerivedData" \
79     ARCHS="arm64 x86_64" \
80     ONLY_ACTIVE_ARCH=NO \
81     CODE_SIGNING_ALLOWED=NO \
82     build
83
84 BUILT_APP="$WORK_DIR/DerivedData/Build/Products/Release/MyIME.app"
85 if [ ! -d "$BUILT_APP" ]; then
86     echo "error: Xcode did not produce MyIME.app" >&2
87     exit 3
88 fi
89
90 BUILT_ARCHS=$(lipo -archs "$BUILT_APP/Contents/MacOS/MyIME")
91 case " $BUILT_ARCHS " in * " arm64 "*) ;; *)
92     echo "error: release is missing arm64: $BUILT_ARCHS" >&2
93     exit 3
94 esac
95 case " $BUILT_ARCHS " in * " x86_64 "*) ;; *)
96     echo "error: release is missing x86_64: $BUILT_ARCHS" >&2
97     exit 3
98 esac
99
100 if [ -n "$SIGNING_IDENTITY" ]; then
101     echo "==> Signing with $SIGNING_IDENTITY"
102     codesign --force --deep --options runtime --timestamp --sign "$SIGNING_IDENTITY"
103         " "$BUILT_APP"
104 else
105     echo "==> Applying ad-hoc signature (local testing only)"
106     codesign --force --deep --sign - "$BUILT_APP"
107 fi
108 codesign --verify --deep --strict --verbose=2 "$BUILT_APP"
109
110 MINIMUM_OS=$(/usr/libexec/PlistBuddy -c 'Print :LSMinimumSystemVersion' "$BUILT_APP
111     /Contents/Info.plist")
112 if [ "$MINIMUM_OS" != "13.0" ]; then
113     echo "error: unexpected minimum macOS version in release: $MINIMUM_OS" >&2
114     exit 3
115 fi
116
117 mkdir -p "$OUTPUT_DIR"
118 APP_NAME="MyIME-$VERSION$ARTIFACT_SUFFIX.app"

```

```

117 DMG_NAME="MyIME-$VERSION$ARTIFACT_SUFFIX.dmg"
118 APP_OUTPUT="$OUTPUT_DIR/$APP_NAME"
119 DMG_OUTPUT="$OUTPUT_DIR/$DMG_NAME"
120 STAGE_DIR="$WORK_DIR/dmg"
121 mkdir -p "$STAGE_DIR"
122
123 case "$APP_OUTPUT" in
124     "$OUTPUT_DIR"/MyIME-*.app) /bin/rm -rf "$APP_OUTPUT" ;;
125     *) echo "error: refusing unsafe output path: $APP_OUTPUT" >&2; exit 4 ;;
126 esac
127 /usr/bin/ditto "$BUILT_APP" "$APP_OUTPUT"
128 /usr/bin/ditto "$BUILT_APP" "$STAGE_DIR/MyIME.app"
129 /usr/bin/ditto "$ROOT_DIR/release/安装说明.txt" "$STAGE_DIR/安装说明.txt"
130
131 if [ -e "$DMG_OUTPUT" ]; then
132     case "$DMG_OUTPUT" in
133         "$OUTPUT_DIR"/MyIME-*.dmg) /bin/rm -f "$DMG_OUTPUT" ;;
134         *) echo "error: refusing unsafe output path: $DMG_OUTPUT" >&2; exit 4 ;;
135     esac
136 fi
137
138 echo "==> Creating $DMG_NAME"
139 hdiutil create \
140     -volname "MyIME $VERSION" \
141     -srcfolder "$STAGE_DIR" \
142     -format UDZO \
143     -ov \
144     "$DMG_OUTPUT"
145
146 if [ -n "$NOTARY_PROFILE" ]; then
147     if [ "$SIGNATURE_LABEL" != "developer-id" ]; then
148         echo "error: notarization requires a Developer ID Application signature"
149         >&2
150         exit 5
151     fi
152     echo "==> Notarizing and stapling DMG"
153     xcrun notarytool submit "$DMG_OUTPUT" --keychain-profile "$NOTARY_PROFILE" --
154     wait
155     xcrun stapler staple "$DMG_OUTPUT"
156     xcrun stapler validate "$DMG_OUTPUT"
157 fi
158
159 (cd "$OUTPUT_DIR" && shasum -a 256 "$DMG_NAME" > "$DMG_NAME.sha256")

```

```
158
159 echo ""
160 echo "Created:"
161 echo "  $APP_OUTPUT"
162 echo "  $DMG_OUTPUT"
163 echo "  $DMG_OUTPUT.sha256"
164 echo "Signature: $SIGNATURE_LABEL"
```

Listing 24.1: 正式发布脚本的环境检查与主要门禁

24.4 凭据管理

公证凭据应存入钥匙串 profile，而不是写进脚本或提交到仓库。应用专用密码一旦出现在截图或聊天记录中，应立即撤销并重新生成。课堂作业不要求学生共享教师证书；每组可生成 ad-hoc 演示包，正式公开发布由项目维护者在受控机器执行。

工程警告：公证成功不代表功能成功

公证只证明 Apple 的安全检查接受该二进制，不证明输入源能注册、候选准确或 Notes 可输入。最终门禁仍包含目标机功能验收。

动手实验：验收一个 Release

对 DMG 依次运行 `shasum-a256`、`spctl--assess`、`codesign--verify--deep--strict`、`xcrunstaplervalidate`。挂载后检查应用版本、架构与签名团队，再在干净 Mac 安装。

思考与练习

1. 签名、公证和装订分别解决什么问题？
2. 为什么发布凭据不能通过环境变量明文保存在 CI 日志？
3. 设计一份可由第三方复核的 Release 清单。

本章小结

正式发布是一条不可跳步的证据链。测试、双架构、Developer ID、Apple 公证、票据装订、校验和和跨机器输入共同定义“可下载版本”。

维护真实输入法：技术债与演进边界

学习目标

能够评审控制器和安装器中的复杂度；区分教学最小实现与生产兼容代码；设计版本迁移、文档同步和未来功能边界。

MyIME 作为教学案例的价值在于它既有清晰核心，也暴露真实复杂度。`Engine.swift`、`MyIMEInputController.swift` 和 `SelfInstaller.swift` 已经包含大量策略。继续增加双拼、云更新或移动平台前，应先检查模块边界，而不是把新分支推进控制器。

25.1 复杂度地图

模块	当前职责	演进建议
InputController	事件、组合、候选导航、客户端适配、英文辅助	抽出 <code>InputAction</code> 与 <code>CompositionState</code> ，降低分支耦合
Engine	切分协调、词查询、评分、词格、上下文、学习	保持纯逻辑；为评分解释和策略注入留接口
SQLiteStore	系统查询、统计模型、用户库	分离只读 <code>schema</code> 与用户迁移版本
SelfInstaller	复制、注册、旧版迁移、重启	用明确安装阶段和错误类型替代隐式布尔状态
候选窗口	SwiftUI 渲染与 AppKit 窗口定位	增加可访问性与多屏快照测试

25.2 规格、实现与测试同步

仓库中的 `SPEC.md` 描述目标，`FOOLPROOFING.md` 描述门禁，`README` 描述用户能力，代码是实际行为。它们可能随版本产生漂移。维护者应给每项功能标记 `implemented`、`planned` 或 `removed`，并把用户可见变化写入版本说明。

25.3 功能边界

双拼会改变切分与设置，但不必改变 IMK 生命周期；云词库会改变隐私承诺和更新安全；语音输入引入音频权限与服务端/本地模型；移动端输入法则不同平台架构。评审新功能时先问它改变哪一层契约，再决定是否仍属于同一个产品。

核心概念：教学版与完整版共存

最好的教材仓库不是把生产代码削成玩具，而是保留两个视图：章节化的教学版展示关键原理，完整版展示边界条件和发布现实。学生先能解释，再能维护。

25.4 把无障碍作为回归契约

候选列表的“能看见、能点击”不等于可访问。每个候选行应是独立无障碍元素，标签读出序号与候选词，当前高亮项增加 `.isSelected` trait；辅助说明放在 `hint`，不要把词频、调试分数等噪声塞进主标签。SwiftUI 示例见第 7 章。

当方向键改变高亮时，可按需发送 `NSAccessibility.announcementRequester.dNotification`，`userInfo` 中提供 `announcement` 与 `priority`。¹公告必须节流：按键长按时只读最终稳定项，避免 VoiceOver 被消息队列淹没。非激活 `NSPanel` 不应抢走宿主编辑器焦点；关闭候选窗后，VoiceOver 焦点仍应留在原文本控件。

每次改候选 UI 至少回归：VoiceOver 开/关、键盘全访问、提高对比度、减少透明度、浅色/深色、多屏与放大显示。自动化测试可验证 `label`、`selected trait` 和稳定候选索引，人工测试负责朗读顺序、焦点保持与语速下的可理解性。

¹Apple: <https://developer.apple.com/documentation/appkit/nsaccessibilityannouncementrequestednotification>

动手实验：架构评审

选择一个计划功能（双拼、词库导入、诊断面板或用户词导出），写一页变更契约：目标、涉及模块、不改变的行为、数据迁移、测试和发布风险。禁止先写代码。

思考与练习

1. 哪些控制器状态可以抽成纯 Swift 状态机？
2. 加入双拼时，Candidate 的 pinyinPath 是否仍足够？
3. 如何避免教材行号随源码变化失效？提出一种自动检查办法。

本章小结

真实输入法的维护重点不是持续增加功能，而是守住系统、算法、存储和发布契约。教学最小版解释原理，完整版本训练工程判断。

第四部分

课程实验与综合项目

实验一至四：从 Echo 到候选窗口

本章给出初级篇的可提交实验。每个实验都要求“源码、测试、观察记录”三件套。只提交能运行的 App 不足以证明理解；只提交报告也不足以证明实现。

26.1 从哪个分支开始

仓库提供 lab1-starter 至 lab12-starter 十二个起始分支；分支名表示“开始该实验前的状态”，不表示答案。学生只克隆教师指定的 starter，并从自己的作业分支提交：

```
git fetch origin --prune
git switch --create homework/lab1 origin/lab1-starter
swift test --package-path TeachingStarter
```

Starter	已提供的边界	学生完成的核心
lab1-starter	IMK 配置清单与 Echo 接口	最小服务器、事件回送
lab2-starter	纯状态机类型与表测框架	组合、退格、提交、取消
lab3-starter	小词库协议与 TSV fixture	候选召回、稳定排序
lab4-starter	候选 ViewModel 与坐标 fixture	不抢焦点窗口、多屏定位
lab5-starter	音节表和图搜索接口	切分图、撇号、简拼与容错
lab6-starter	ScoreBreakdown 数据型	分数解释与消融
lab7-starter	小词格 fixture	Beam Search 与完整枚举校验
lab8-starter	临时用户库 schema	异步学习、清空与故障降级
lab9-starter	客户端测试矩阵模板	跨客户端、VoiceOver 回归
lab10-starter	诊断报告字段协议	脱敏收集与证据链
lab11-starter	许可证/SBOM 清单模板	离线与供应链审计
lab12-starter	Release 验收脚本骨架	第二台 Mac 安装和恢复

完整 MyIME 1.0.10 用于课后对照，不是实验起点。公开仓库中的分支只能避

免“误入完整实现”，不能阻止刻意查看 `main`；正式评分课程应把对应 `starter` 导入独立的 GitHub Classroom 私有模板仓库，只向学生开放模板。教师的 `solution` 分支不要推送到学生可读仓库。

26.2 实验一：Echo IME

目标：建立最小 `IMKServer` 和控制器，在 Notes 中原样回送字母。

约束：不得引入数据库、候选窗口或自动安装器；核心代码不超过 150 行；必须记录真正进入的回调。

步骤：

1. 新建 macOS App target，配置输入源 plist；
2. 创建 `IMKServer`，并在入口日志记录 bundle path、bundle ID 和 connection name；
3. 控制器只处理可打印 ASCII，其他事件放行；
4. 安装到用户输入法目录，重新登录或注册；
5. 在 Notes 输入 `hello123`，验证无重复和丢字。

验收问题：返回 `true/false` 的区别是什么？进程存在为何不足以证明事件进入控制器？

26.3 实验二：组合状态

目标：实现 raw buffer、marked text、退格、Esc 和 Return。

建议先在纯 Swift 中实现：

```

1 enum InputAction: Equatable {
2     case append(Character), deleteBackward, commit, cancel
3 }
4
5 struct CompositionState: Equatable {
6     var raw = ""
7     mutating func reduce(_ action: InputAction) -> Effect { /* ... */ }
8 }
9
10 enum Effect: Equatable {

```

```
11 case updateMarked(String), insert(String), clearMarked, none
12 }
```

Listing 26.1: 教学状态机接口

用表驱动测试覆盖空闲态与组合态。随后写一个薄适配器，把 Effect 映射到 setMarkedText 或 insertText。

26.4 实验三：小词库候选

目标：使用不超过 100 条 TSV 词库完成全拼候选和数字选词。
必测输入：nihao、beijing、一个同音词、一个不存在的拼音、一个未完成前缀。候选排序必须有稳定 tie-breaker。报告中展示每个候选的权重和最终排序原因。

26.5 实验四：候选窗口

目标：实现不抢焦点的候选窗、方向键高亮、数字选择、翻页和屏幕边缘定位。
截图要求：屏幕中央、左下角、右下角、多显示器各一张。截图必须同时包含插入点和候选窗，证明没有越界。

26.6 评分量表

项目	分值	判定
输入源可注册、可选择	15	使用稳定 ID，无重复条目
事件所有权正确	15	无重复、无吞键，修饰键放行
组合状态正确	20	退格、取消、提交和失活一致
候选与部分输入	15	小词库结果稳定且可解释
窗口交互	15	不抢焦点、可导航、不越界
自动化测试	10	状态机与词库至少 12 个测试
实验报告	10	有失败证据、原因和修复验证

实验五至八：切分、排序、整句与学习

27.1 实验五：切分图

输入一组拼音，输出 DOT 或文本形式的切分图。至少覆盖全拼、多路径、撇号、简拼、混拼、模糊音和一次错拼。要求验证：完整消费优先、结果确定、非法字符不崩溃。

扩展任务：实现递归搜索与动态规划两种方法，对 1 万条随机输入比较输出与耗时。若结果不同，必须给出反例并解释。

27.2 实验六：评分解释器

为每个候选输出：

```
word=天气
frequency=-3.42
user=+0.00
context=+2.18
fuzzy=-0.00
partial=-0.00
incomplete=-0.00
total=-1.24
```

选择至少十组同音/近音候选，通过消融实验说明每项存在的必要性。调参结论必须来自开发集，不得根据封存测试集逐条修改。

27.3 实验七：词格整句

实现宽度可配置的 Beam Search。测试 woshizhongguoren、jintiantianqih enhao 和一条至少 10 音节句子。记录 Beam 宽度为 2、4、8、12、24 时的 top-1、top-5、P50 和 P95。

正确性检查：使用一个很小的词格完整枚举所有路径，与 Beam 宽度足够大时的结果比较。这是用第二种方法验证搜索实现，而不是只相信几个 golden 用例。

27.4 实验八：用户学习

实现用户 unigram、bigram 和最近组词之一。必须支持重启持久化和清空。为数据库故障注入只读、损坏文件和并发选择，证明基本输入仍可用。

27.5 实验报告模板

每个实验按以下结构提交：

1. 问题定义与不变量；
2. 两种实现或验证方法；
3. 数据集与环境；
4. 结果表和至少一幅图；
5. 失败案例与原因；
6. 对完整 MyIME 源码的对应关系；
7. 残余风险与下一步。

实验九至十二：兼容、诊断、安全与发布

28.1 实验九：跨客户端兼容矩阵

选择 Notes、TextEdit、Safari、Chrome、一个 Electron 应用和一个国产办公/通信应用。固定测试 12 条输入序列，包括中英文切换、长句、退格、候选导航、应用切换和进程恢复。

用例	Notes	TextEdit	Safari	Chrome	Electron	其他
首选上屏						
退格刷新						
候选定位						
应用切换						
进程恢复						

每个失败必须附最小输入序列和事件路径日志。禁止用“兼容性不好”作为结论。

候选窗口还须在 VoiceOver 下验证：方向键改变高亮时朗读序号与候选，selected 状态可识别，候选窗出现/消失不夺走宿主编辑器焦点。把该结果作为矩阵的一行，而不是只写进文字备注。

28.2 实验十：诊断包

实现一个不包含正文的本地诊断命令，输出：应用版本与路径、签名摘要、当前输入源 ID、TIS 枚举结果、系统版本、芯片架构、词库完整性、用户库 schema 版本和最近 200 行脱敏日志。

在报告中给出威胁分析：哪些字段仍可能间接识别用户？诊断包如何过期和删除？

28.3 实验十一：离线与供应链审计

检查源码依赖和运行时网络连接；确认 Release 包中的第三方许可；验证替换词库入口不能加载损坏或未经验证的文件。对下载 DMG 计算 SHA-256，并与发布页校验和比较。

28.4 实验十二：第二台 Mac 发布验收

目标机器必须满足：没有开发证书、没有源码、没有旧 MyIME 副本。只能从公开 Release 下载 DMG。验收步骤如下：

1. 验证 SHA-256、Developer ID 与公证票据；
2. 打开 DMG 并安装，不使用 Xcode 或本地脚本；
3. 在系统设置确认 MyIME 位于简体中文分类；
4. 在 Notes 输入规定句子并录屏；
5. 结束输入法进程，通过 ABC→MyIME 恢复；
6. 重启机器后再次验证；
7. 卸载并确认用户选择的数据保留/删除策略符合说明。

工程警告：凭据纪律

学生不得在报告、截图、录屏或 Git 历史中包含 Apple ID 密码、应用专用密码、证书私钥或钥匙串导出文件。发生泄露应立即撤销，而不是仅删除截图。

综合项目：实现一款可解释的离线输入法

学习目标

把课程知识转化为独立设计；形成可运行软件、可复现评测和可审查发布物；能够在答辩中用证据说明每个关键决策。

综合项目可以在 MyIME 上实现一个明确增量，也可以从教学版继续发展。推荐题目包括：双拼支持、用户词导入导出、评分解释面板、专业领域离线词库、可访问候选窗口、离线词库签名更新、组合状态机重构或跨客户端自动化测试工具。

29.1 项目契约

立项书不超过两页，必须包含：用户问题、范围、明确不做的功能、受影响模块、数据结构变化、隐私变化、测试指标和演示路径。若题目引入网络能力，必须单独通过隐私与安全评审。

29.2 最终交付物

1. 可构建源码与清晰提交历史；
2. 设计说明，包含架构图、状态机或算法图；
3. 自动化测试与完整命令输出；
4. 至少 200 条封存集评测和延迟分布；
5. 跨机器兼容矩阵；
6. 签名/公证发布物或明确标记的开发演示包；

7. 三分钟演示视频与十分钟技术答辩。

29.3 100 分三级评分量表

“卓越”通常获得该项 85–100% 分值,“合格”获得 60–84% ,“不合格”获得 0–59%。严重安全事故、无法构建或伪造数据可触发课程规则中的额外封顶;助教必须在评语中引用具体提交证据,不能只写主观印象。

维度	分	卓越	合格	不合格
问题与范围	10	用户问题、非目标、成功指标和变更契约均可验证;范围变化有记录	目标基本明确,能演示主要路径;少量非目标或指标模糊	只有功能口号;范围持续漂移,无法判定完成
架构	15	系统/算法/存储/UI 边界明确;主线程、锁和失败降级有图与不变量	能说明主要模块,依赖方向大体合理;存储在局部耦合但可维护	控制器包揽全部逻辑;共享状态无所有权,修改一处破坏多处
核心正确性	20	正常、边界、失败路径齐全;关键算法由完整枚举、模型或第二实现交叉验证	主流程和常见边界正确;测试覆盖主要不变量,但验证方法单一	吞键、重复上屏、崩溃或错误提交可稳定复现;只靠演示样例
准确率与性能	15	训练/开发/封存集隔离;报告 top-k、P95;结果可复现,P50/P95/P99、样本量及误差分析;优化有前后对照	有固定数据集与 P95;结果可复现,但分层/误差分析不完整	用挑选样例或“感觉更快”代替指标;测试集被用于逐例调参
系统兼容	10	多客户端、多屏、生命周期、VoiceOver 和二台 Mac 均有最小复现证据	Notes 加至少两类客户端通过;恢复和坐标边界有基本验证	仅开发机单一文本框可用;依赖守护进程或手工脚本维持
隐私与安全	10	数据流、日志脱敏、文件权限、许可、签名/公证和凭据撤销策略完整	无网络或网络用途清楚;无正文日志和已知明文凭据;许可基本齐全	上传/记录正文未告知;提交密码、私钥;来源或许可无法说明
测试工程	10	纯逻辑、存储、系统适配分层;历史故障有回归;失败输出可定位且无顺序依赖	自动化覆盖核心模块并能一键运行;少量系统步骤仍需人工	测试不能运行、依赖个人机器状态,或只有截图无断言

维度	分	卓越	合格	不合格
表达与答辩	10	能从按键到上屏逐层解释，并现场说明分数、剪枝、锁、SQL 和发布证据	能解释主要设计与已知限制；在提示下可定位源码和测试	无法区分 marked/commit、事件所有权或发布签名；报告与实现矛盾

29.4 答辩必问十题

- 1. 当前按键由哪个入口收到，为什么？
- 2. 返回 true/false 会怎样？
- 3. raw、preedit、marked text 和 commit 分别存在哪里？
- 4. 展示一个候选的完整分数分解。
- 5. 展示一个切分歧义和一个词格剪枝。
- 6. 用户词库损坏时怎样降级？
- 7. 哪些日志可能泄露隐私？
- 8. 为什么开发机成功不能证明 Release 成功？
- 9. 你的测试集如何避免泄漏？
- 10. 如果再给两周，你会优先消除哪个风险？

本章小结

综合项目的目标不是复制 MyIME 功能数量，而是证明学生能够用系统模型、算法、测试和发布证据完成一个可信的输入法增量。

Swift 与并发速查

本附录只收录理解 MyIME 必需的 Swift 概念。

A.1 值类型与引用类型

`struct` 复制值，适合 `Candidate`、`EngineOutput` 和偏好快照；`class` 具有对象身份，适合控制器、数据库连接和窗口控制器。包含数组的结构体仍享受写时复制，但跨线程安全要看元素与访问方式。

A.2 协议与 existential

`anyCandidateLookup` 表示运行时持有任意满足协议的对象；泛型 `Engine<Store: CandidateLookup>` 则在编译期特化。前者便于替换与保存，后者可能有性能优势。输入法查询开销主要在数据库与搜索，当前 existential 通常不是瓶颈。

A.3 可选值与 guard

系统 API 大量返回可选值。控制器入口宜用 `guard` 验证 `client`、字符和范围，把失败路径尽早放行或降级。不要为了减少代码而强制解包客户端对象。

A.4 Sendable、锁与队列

`Sendable` 表示值可以安全跨并发域传递；`@uncheckedSendable` 表示作者自行保证。常见保护方法包括：

- 不可变值快照；
- `NSLock.withLock` 保护小型同步状态；

- 串行 DispatchQueue 保护 SQLite 写入；
- Actor 隔离异步状态；
- @MainActor 限制 UI 和 App 生命周期。

工程警告

不要在持有状态锁时执行 SQL、调用客户端或刷新 UI。锁的临界区只读写必要字段，避免重入和主线程卡顿。

A.5 输入法线程模型：先画所有权，再写并发代码

输入法没有“所有工作都在后台越快越好”这一选项。IMK 事件回调、NSPanel 和 SwiftUI 更新属于 UI/会话层，MyIME 用 @MainActor 把它们限制在主执行器；用户数据库写入可以串行异步执行；引擎计算保持同步、短小，并以不可变 EngineOutput 快照返回 UI。

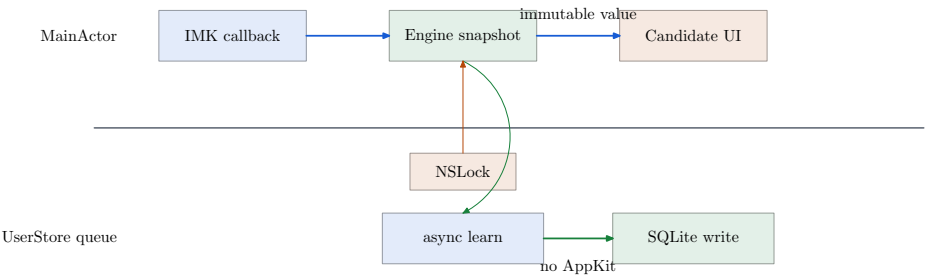


图 A.1: MyIME 的线程模型：主执行器处理会话与 UI，UserStore 队列串行持久化

对象/操作	推荐执行域	原因与约束
IMK 回调、客户端调用	MainActor	输入会话有严格顺序，客户端对象不可跨线程长期持有
CandidateWindow/ NSPanel	MainActor	AppKit UI 必须在主线程更新
Engine.update	调用线程，受时间预算限制	返回值快照；不得在其中等待异步写入

提交历史	NSLock 短临界区	只复制/更新少量值，禁止锁内 SQL 和 UI
UserStore.learn	私有串行队列	保证 SQLite 写入顺序，不阻塞按键路径
system.sqlite 查询	连接所有者线程/受控同步	只读；同一 statement 不得无保护并发复用

A.5.1 @MainActor、Sendable 与 @unchecked Sendable 分别保证什么

@MainActor 是隔离规则：编译器要求相关状态和方法在主执行器访问。Sendable 是跨并发边界的值安全承诺；像 Candidate 这样的不可变结构体适合自动满足。@uncheckedSendable 则表示“编译器不再证明，由作者负责”，必须在代码评审中附带同步策略。它绝不能成为消除警告的通用修饰符。

A.5.2 NSLock 的两条铁律

第一，锁保护的是一个写明的不变量，而不是“这一整个函数”；第二，不调用未知代码。客户端、闭包、日志格式化和 SQLite 都可能重入或阻塞。正确形式是锁内复制状态、立即解锁，再执行外部操作：

```
1 let history = stateLock.withLock { committedHistory }
2 // 锁已经释放；下面可以查词、评分和刷新 UI。
3 let anchors = contextAnchors(last: history.last,
4                               beforeLast: history.beforeLast)
```

Listing A.1: 缩短锁临界区

动手实验：画线程所有权图

从一次候选选择开始，标出回调线程、Engine、NSLock、UserStore 队列和 SQLite。对每条跨域箭头写出传递的数据类型；若传的是 client、NSWindow 或可变数组，说明为何危险并改成值快照。

A.6 String 索引

Swift String 按扩展字素簇索引，不支持任意整数下标。raw 被归一化为 ASCII 时可以安全按字符数量推导消费长度；汉字候选的 `count` 是字符数量，不等于 UTF-8 字节数。数据库和 C API 交界处必须明确编码单位。

InputMethodKit 与 TIS 速查

B.1 核心类型

类型/方法	教学意义
IMKServer	注册输入法服务连接，由系统发现控制器
IMKInputController	每个输入会话的事件与组合控制对象
IMKTextInput	客户端文本协议，提供 marked/insert/range/coordinate
handle(_:client:)	NSEvent 路径的事件入口之一
inputText(_:client:)	文本形式输入入口之一
setMarkedText	设置尚未最终提交的组合文本
insertText	提交普通文档文本
commitComposition	系统要求结束组合
activateServer	控制器会话被激活
deactivateServer	控制器会话失活，应处理残留状态
TIS API	枚举、注册、选择输入源和监听变化

B.2 常见按键语义

硬件键码与字符不是一回事。字符受到键盘布局与修饰键影响，键码更接近物理位置。处理普通字母优先使用字符；处理方向键、退格和功能键可使用键码或特殊字符，并在目标系统实测。

按键	组合态默认	必测边界
字母 a-z	追加 raw	Shift、Caps Lock、Option

Backspace	删除 raw 尾字符	raw 为空时放行
Space	选择首选	无候选、空闲态
Return	当前候选或原文上屏	导航前后行为
Esc	取消组合	空闲态不吞键
1-9	数字选词	页首偏移、越界
左右方向	移动高亮/光标策略	首尾边界
上下方向	翻页或移动高亮	横纵候选布局
Command 组	通常放行	复制、撤销、切应用
合		

B.3 AppKit 坐标转换专题

排查候选窗错位时，先给每个矩形标记所属空间，而不是立即加减常量：`R_screen` 表示 `NSScreen` 全局空间，`R_window` 表示窗口内容空间，`R_view` 表示视图局部空间，`R_line` 表示输入客户端返回的行框。

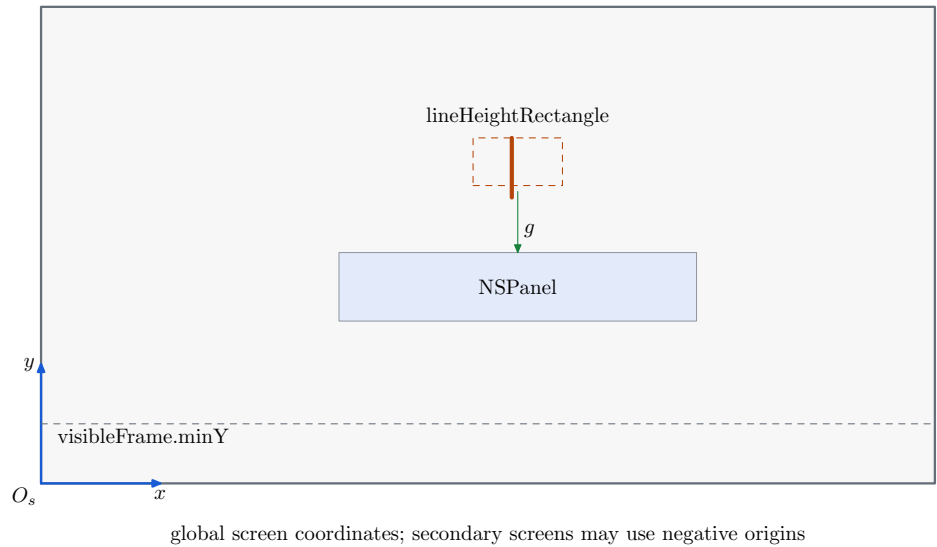


图 B.1: 坐标排错图：lineHeightRectangle 与 NSPanel 位于全局屏幕空间

若一个点来自普通 `NSView`，可按“view→window→screen”转换：先用 `view.convert(point,to:nil)` 得到窗口内容坐标，再用 `window.convertPoint(toScreen:)` 得到全局坐标。反向转换时顺序反过来。若 API 已明确返回屏幕矩形，则不要再次转换。

多显示器验收至少包含以下排列：右侧次屏、左侧次屏（负 x ）、下方次屏（负

y) 和不同缩放比例。永远用命中的 `screen.visibleFrame` 夹取候选窗；不要只用 `NSScreen.main`，因为 `main` 表示当前主屏，不一定是插入点所在屏。

算法推导与第二方法验证

C.1 切分的动态规划形式

令 $R[j]$ 表示到位置 j 的前若干最优路径。初始化 $R[0] = \{\epsilon\}$ 。对每个 $i < j$ ，若 $x[i:j]$ 为合法音节，则：

$$R[j] \leftarrow \text{TopK}\{p \oplus x[i:j] \mid p \in R[i]\}.$$

该方法与递归枚举的结果可以互相验证。递归更直观，动态规划更容易复用前缀结果。加入模糊音、简拼和错拼后，边携带代价，TopK 按消费完整性与总代价排序。

C.2 Viterbi 与 Beam 的关系

若每个位置只需保留一个与未来充分相关的状态，Viterbi 可保证最优。输入法需要 top-k，且转移依赖末词、用户上下文和多种特征，因此状态空间扩大。Beam Search 在每个位置保留有限假设，是可控近似。

为验证 Beam，教学中构造一个小词格，完整枚举路径集合 Π ：

$$\pi^* = \arg \max_{\pi \in \Pi} S(\pi).$$

再检查 Beam 宽度充分大时首选等于 π^* 。若宽度变小时结果改变，应展示被剪路径在哪一层落出前 B 名。

C.3 对数域数值示例

假设词“今天”“天气”“很好”的条件概率分别为 0.05、0.12、0.20，则句子概率为

$$0.05 \times 0.12 \times 0.20 = 0.0012.$$

对数分数为

$$\log 0.05 + \log 0.12 + \log 0.20 \approx -2.996 - 2.120 - 1.609 = -6.725.$$

比较两个候选时，对数单调性保证排序不变。每条边只需加法，且不会在长句上快速下溢。

C.4 平滑

未见 bigram 不能直接令整条路径概率为零。可以使用加性平滑、退避或插值：

$$P(w_i | w_{i-1}) = \lambda P_{\text{bi}}(w_i | w_{i-1}) + (1 - \lambda) P_{\text{uni}}(w_i).$$

MyIME 当前使用奖励式统计而非严格归一概率，教学者应明确这是工程评分函数。二者都可用，但不能把“分数”错误称为经过校准的概率。

词库流水线与可复现数据工程

运行时词库只是最终产物，教学还应理解来源下载、编码统一、拼音归一、去重、频率融合、噪声过滤、索引构建与许可清单。MyIME 的规范给出一条可复现流水线。

```

1  ## 2. CIF —— 归一中间格式
2
3  每来源先转成统一的 **CIF TSV** (UTF-8, `\n`) :
4  ---
5  word <TAB> pinyin <TAB> weight <TAB> source
6  ---
7  字段约束 (MUST) :
8  - `word`: 目标词, **简体**、Unicode **NFC**; `繁→简` 用 OpenCC `t2s`。长度 1-20 汉字。
9  - `pinyin`: 小写 `a-z` 音节, 以 `'` 连接, 无声调; 音节数 MUST == `word` 汉字数 (不等则丢弃并计入 `dropped_mismatch`)。
10 - `weight`: 原始频率/权重 (整数, 来源内部标度, 后续再全局归一)。缺失则置来源默认 (见各适配器)。
11 - `source`: 上表 `id`。
12
13 CIF 是**唯一进入合并器的入口**; 任何来源都 MUST 先落地为 `build/cif/<source>.tsv`, 便于抽检与回归。
14
15 ---
16
17 ## 3. 各来源适配器 (Adapters)
18
19 每个来源一个 `tools/adapters/<id>.py`, 输入原始文件、输出 `build/cif/<id>.tsv`。通用步骤: 解析 → 抽 `(word, pinyin?, weight?)` → 补拼音 → 校验 → 写 CIF。
20
21 ### 3.1 补拼音策略 (MUST)
22 - 来源**自带拼音** (rime/fcitx5/景行): 直接采用, 仅做小写化、去声调、`'` 归一、音节数校验。
23 - 来源**无拼音** (部分 thesaurus / renfei / lexicon 纯词表): 用 `pypinyin` (`Style.NORMAL`, `heteronym=False`, `errors='ignore'`) 生成; 含非汉字则该行进入 `non_han`

```

```

    分流。
24 - **多音字**: v1.0 只取来源给定读音; 自动生成时取 `pinyin` 默认读音, 不做多音字展开
    (避免噪声爆炸)。
25
26 ### 3.2 各来源要点
27 - **`rime_ice`**: 解析 `dict.yaml`, 跳过 YAML 头 (`---`...`...`), 数据行 `text\tcode\tweight`
    (weight 可缺)。`code` 空格→` `。作为基准, rank=1。
28 - **`custom_pinyin`**: Fcidx5/Rime 文本, 按其分隔符 (空格或 tab) 解析; 百万级, MUST
    流式处理, 勿全量载入内存。
29 - **`zhwiki`**: fcidx5 文本 `词 拼音 频率` (拼音以 ` ` 连); 繁体条目 OpenCC t2s; 长
    专名多, 长度上限放宽到 20。
30 - **`jingxing`**: Gboard 文本; **忽略双拼列** (v1.0 非目标), 只取全拼; 无频率则默认中
    位权重。
31 - **`skrik2_lexicon`**: JSON 走 `json`, 文本走通用解析; 带分类 tag 的 MAY 存入 `source`
    之外的辅助日志, 但**不进 `entries`** (v1.0 无分类字段)。
32 - **`renfei_dict`**: 搜狗整理版文本; 繁→简; 去分类表头行。
33 - **`thesaurus` (特殊)**: 程序员术语, **保留原大小写** (`Java`、`Markdown`)。这类词
    的 `word` 含 ASCII 字母:
34 - 「拼音」用其**小写 ASCII 本身**作为 `py_key`/`pinyin` (如 `java`→`java`), 使英文
    首字母/全拼都能召回; `initials` 取首字母。
35 - `source=thesaurus` 并打内部 `ascii=1` 标记, 合并时**与中文词隔离** (不同 `py_key`
    命名空间天然隔离, 无需额外列)。
36 - 音节数校验对 ASCII 词**豁免** (它不是汉字词)。
37
38 ---
39
40 ## 4. 校验 (每来源出 CIF 后, MUST)
41 逐行校验, 产出 `build/report/<id>.json`:
42 - 编码: 非法 UTF-8 → 丢弃并计数。
43 - `pinyin` 仅含 `a-z`; 否则丢弃 (ASCII 术语走豁免通道)。
44 - 音节数 == 汉字数 (汉字词); 不符丢弃并计入 `dropped_mismatch`。
45 - 空 `word` / 空 `pinyin` / 超长 → 丢弃。
46 - 统计: `total_in, kept, dropped_encoding, dropped_mismatch, dropped_nonhan, dedup_internal`。
47 > 任一来源 `kept/total_in < 0.6` MUST 报警并人工核对适配器 (防呆 G-DICT-YIELD)。
48
49 ---
50
51 ## 5. 合并与去重 (Merger)
52
53 输入全部 `build/cif/*.tsv`, 输出 `build/merged.tsv` (含 `source_mask`)。
54
55 ### 5.1 去重键

```

```
56 `dedupKey = (word, py_key)`(`py_key` = 去 `` 拼接)。同键即同一条目，跨来源合并。  
57  
58 ### 5.2 词形/拼音仲裁  
59 - 同 `dedupKey` 天然同 `word`+`pinyin`，无需仲裁。  
60 - 若不同来源对同一 `word` 给出**不同拼音**（多音差异）：各自成条（`py_key` 不同），都保留。
```

Listing D.1: 词库归一化和中间格式规范

D.1 建议的中间格式

所有来源先转换为统一 CIF：

```
word<TAB>pinyin_with_apostrophe<TAB>weight<TAB>source_mask  
北京<TAB>bei'jing<TAB>64000<TAB>4
```

归一化包括 Unicode NFC、空白清理、简繁策略、拼音小写、ü→v、音节合法性和权重范围。每条被丢弃记录应有原因计数，避免流水线静默吞掉大批数据。

D.2 去重与来源保留

相同词语和读音可能来自多个来源。可以合并权重并 OR 来源位掩码，使设置界面允许禁用某类词库。词语相同、读音不同不能简单去重；多音字需要保留独立记录。

D.3 可复现性

来源锁文件应固定 URL、版本、校验和和许可证。构建脚本不得悄悄抓取“最新版”覆盖输入。最终数据库记录 schema 版本、构建时间、流水线版本与来源摘要；Release 测试验证资源存在、完整且条数在合理区间。

动手实验：小型词库构建

选择两个许可明确的小词表，转换 CIF、去重、构建 SQLite，并输出每阶段统计。第二名同学在另一台机器运行同一脚本，比较数据库内容哈希或规范化导出哈希。

D.4 五分钟编译：从 CIF 到 SQLite

若已经准备好四列 CIF，学生不必先运行完整的网络下载与语言模型流水线。仓库现有的 `tools/compile.py` 可以直接把 CIF 编译为教学用 SQLite；下面的命令与脚本真实参数一致，可作为实验六和实验十一的最小起点：

```
1 set -eu
2
3 cif=build/cif/student.tsv
4 out=build/student/system.sqlite
5
6 mkdir -p "$(dirname "$out")"
7 SOURCE_DATE_EPOCH=0 python3 tools/compile.py "$cif" "$out"
8 sqlite3 "$out" \
9     'PRAGMA integrity_check; SELECT count(*) AS entries FROM entries;'
10 shasum -a 256 "$out"
```

Listing D.2: 从 CIF 编译并检查教学词库

`SOURCE_DATE_EPOCH=0` 固定数据库元数据时间，使同一 CIF 更容易做可复现性比较。若要构建发布版的完整词库、语言模型和简繁映射，则从仓库根目录设置 `PYTHON_BIN=/usr/bin/python3`，再执行 `tools/build_dict.sh`。该入口会下载锁定版本的来源并依次完成合并、模型构建、编译和严格验证，耗时与磁盘占用都明显高于上述课堂最小命令。

故障排查手册

E.1 先确认系统版本：版本防坑矩阵

输入法依赖 TIS、InputMethodKit、Gatekeeper 和宿主文本客户端，升级 macOS 后必须重新跑发布验收。表中的“现象”来自 MyIME 项目测试与应重点回归的风险，不应解释为 Apple 对所有版本作出的永久 API 保证。

系统	常见现象或风险	MyIME 的处理与验证
macOS 13	新安装后列表或当前输入源状态未立即刷新	核对安装路径、Bundle ID 与 TIS 枚举；切换 ABC→MyIME 或重新登录后复测，不让常驻守护进程掩盖问题
macOS 14	签名、公证、隔离属性与实际启动路径差异更容易被误判为算法故障	从公开 DMG 安装，分别检查 <code>code sign</code> 、 <code>spctl</code> 、 <code>stapler</code> 与进程真实路径；清除精确旧副本而非模糊批量删除
macOS 15	不同宿主对 <code>marked text</code> 、 <code>replacement range</code> 与生命周期回调仍可能表现不同	固定 <code>Notes</code> 、 <code>Safari</code> 、 <code>Chrome/Electron</code> 兼容矩阵；升级后重跑事件所有权和失活/恢复用例
macOS 26	项目实测中，对已经启用的输入源重复调用 <code>TISPropertyInputSource</code> 可能再次出现敏感授权提示	安装器先查 <code>kTISPropertyInputSourceIsEnabled</code> ；已启用则跳过；仅在输入源缺失或版本变化时重新注册，并在干净目标机验证
后续版本	安全 UI、缓存时机或客户端实现可能变化	把系统版本写进诊断包；发布门禁覆盖当前版与前一主版本；现象与规避方案带日期记录

尤其要区分“代码事实”和“平台推断”。当前 `SelfInstaller.swift` 的防重复策

略是可审查事实；“所有 macOS 26 机器必定弹窗”则不是。课程报告应写成“在系统版本、构建号、安装状态 X 下观察到 Y；采用 Z 后复测”，不要把单机经验写成 API 规范。

E.2 升级后的十分钟回归

从干净账户安装公证 DMG；确认简体中文列表出现 MyIME；在 Notes 完成中文、英文、退格和候选选择；在浏览器 `input/textarea/contenteditable` 重放同一序列；结束进程后通过 ABC→MyIME 恢复；开启 VoiceOver 检查候选朗读与焦点；最后导出带系统构建号的脱敏诊断包。任一步失败都先保留证据，再决定是否修改注册逻辑。

E.3 系统列表没有 MyIME

检查顺序：安装路径 → 应用包 Info.plist→Bundle ID/输入源 ID→ 签名可读性 →TIS 枚举 → 缓存刷新。不要先修改候选算法。若存在多个相同 Bundle ID 副本，先解析实际路径，再按安装器设计清理具体旧副本。

E.4 能看到 MyIME，但中英文都打不出来

先验证当前输入源确实选中；检查进程路径和 IMKServer 日志；确认事件进入哪条回调；检查控制器是否错误地对未处理事件返回 true；临时用 EchoEngine 排除词库；在 Notes 建立最小复现。

E.5 有拼音下划线，没有候选

检查 normalized raw、切分路径数量、词库路径、integrity check、SQL 查询键和 disabledSourceMask。用 MemoryStore 复测同一 raw；若内存词库正常，问题集中在资源或查询层。

E.6 候选出现但无法上屏

检查数字/空格是否映射到有效索引、select 返回的消费长度、insertText 的客户端与 replacementRange、提交后清理顺序。若出现双字，检查事件是否既被输入

法处理又被客户端处理。

E.7 Notes 正常，浏览器失败

比较事件入口、marked range、replacement range、坐标查询和失活顺序。用最小网页 `input`、`textarea` 和 `contenteditable` 分别测试，不要把整个复杂网站当作一个用例。

E.8 开发机正常，目标机失败

检查 Release 是否为 Developer ID 而非 Apple Development；验证公证和装订；确认目标机没有旧副本；检查 App Translocation 与进程实际路径；从公开下载链接重新获取，不要通过开发目录复制未公证包。

E.9 进程结束后无法恢复

确认 MyIME 仍是系统当前输入源；切换 ABC→MyIME 触发系统建立新会话；检查没有依赖登录守护进程；验证状态菜单反映真实 TIS 状态而不是固定显示“中”。

发布命令与验收清单

F.1 纯引擎

```
swift test --package-path MyIME/IMEKit
```

F.2 应用构建

```
xcodebuild build \  
-project MyIME/MyIME.xcodeproj \  
-scheme MyIME \  
-configuration Release \  
-destination 'platform=macOS'
```

F.3 签名、公证与架构检查

```
codesign --verify --deep --strict --verbose=2 MyIME.app  
codesign -dv --verbose=4 MyIME.app  
spctl --assess --type execute --verbose=4 MyIME.app  
lipo -info MyIME.app/Contents/MacOS/MyIME  
xcrun stapler validate MyIME.app  
shasum -a 256 MyIME-1.0.10.dmg
```

F.4 发布前人工清单

- ☐ 版本号、构建号、下载文件名和主页一致；
- ☐ 引擎测试、词库验证和 Release 构建通过；

- ☐ arm64 与 x86_64 均存在；
- ☐ Developer ID 签名正确，无 development/ad-hoc 混入；
- ☐ 公证 Accepted，票据装订并可验证；
- ☐ SHA-256 文件与 DMG 匹配；
- ☐ GitHub Release 下载链接匿名可访问；
- ☐ 第二台 Mac 无旧副本，安装成功；
- ☐ MyIME 出现在简体中文分类；
- ☐ Notes、浏览器完成固定输入序列；
- ☐ 结束进程后 ABC→MyIME 可恢复；
- ☐ 隐私说明、第三方许可和版本说明同步。

术语表

术语	定义
候选 Candidate	对当前输入可供用户选择的文本或符号
组合 Composition	尚未最终提交、仍由输入法管理的输入事务
Raw buffer	用户原始字母序列
Preedit	组合阶段向用户展示的格式化输入表示
Marked text	客户端中标记为正在组合的文本
Commit	将组合结果转为普通文档文本
消费长度	某候选对应并移除的 raw 字符数量
输入源 Input Source	macOS 文本输入系统可选择的语言模式
IMKServer	InputMethodKit 服务端连接对象
IMKInputController	处理一个输入会话的控制器
TIS	Text Input Source, 枚举与选择输入源的系统接口
切分路径	将连续拼音解释为音节序列的一条路径
硬边界	用户用撇号显式指定的不可跨越音节边界
简拼	用音节声母首字母表示完整音节
模糊音	用户可配置的近似声母或韵母关系
词格 Word Lattice	音节边界为顶点、词条为边的有向图
Beam Search	每层只保留有限高分假设的近似搜索
Unigram	单词自身统计
Bigram	相邻两个词的条件统计
Golden test	锁定特定输入与期望输出的回归测试
Top-k 准确率	正确答案位于前 k 个候选的样本比例
KSPC	每输出一个字符所需平均按键数
P95	95% 样本不通过的延迟分位数

Universal 2	同时包含 arm64 与 x86_64 的 macOS 二进制
Developer ID	App Store 外分发所用的 Apple 开发者签名身份
公证 Notarization	Apple 对提交软件进行自动安全检查并签发票据
装订 Stapling	将公证票据附着到应用或安装包
App Translocation	macOS 对部分下载应用采用的随机隔离运行路径

练习提示与参考方向

本附录不给出可以直接复制的完整作业答案，而给出验证方向。

H.1 初级篇提示

事件所有权问题优先构造最小 Echo 控制器，分别返回 true 和 false 比较最终文本。组合问题用纯 Swift reducer 列出状态，而不是直接在 IMK 回调里反复试。候选窗口问题先验证焦点，再验证坐标；窗口“看起来正确”不能证明输入会话没有被抢走。

H.2 切分提示

最长匹配失败的反例通常来自音节边界歧义。动态规划与递归两种方法应在小输入上完整枚举互相验证。容错规则每增加一种，都要检查精确输入首选不退化。

H.3 评分提示

先把每项映射到统一尺度，再讨论权重。调参时一次只关闭或改变一个特征，保存候选分解。若只看总分，很难判断改善来自正确上下文还是偶然长度偏差。

H.4 词格提示

在位置之外保留 lastWord，否则不同上下文假设会被错误合并。用完整枚举验证小图，用 Beam 验证大图性能；两种方法结果不一致时，先检查是否因 Beam 剪枝，而不是立即认定代码错误。

H.5 学习提示

用户选择次数使用对数或饱和函数，避免一次长期使用永久压倒系统词频。所有持久化测试都要重建 Engine 和 UserStore，防止只验证内存状态。

H.6 发布提示

开发机与目标机必须分开。签名检查读“Authority”和 Team ID，公证检查 Accepted 与 stapler validate，功能检查用固定输入序列。任何一步成功都不能替代其他步骤。

源码阅读路线图

若只用两小时了解项目，按以下顺序阅读：

1. `README.md`：产品边界与构建入口；
2. `Models.swift`：输入、输出、偏好和协议；
3. `Segmenter.swift`：拼音路径；
4. `IMEKitTests.swift`：从测试理解预期行为；
5. `Engine.swift`：词查询、评分和词格；
6. `SQLiteStore.swift`：真实数据访问；
7. `MyIMEInputController.swift`：系统事件与组合；
8. `CandidateWindow.swift`：候选 UI；
9. `IMKBootstrap.swift`：服务生命周期；
10. `SelfInstaller.swift`：跨机器安装；
11. `build-release.sh`：公开发布门禁。

若用一学期学习，则按本书正文顺序，从最小模型逐渐映射到这些文件。源码阅读时对每个方法写四行注释：输入、输出、持有状态、失败方式。对于任何 `@uncheckedSendable`、文件删除、外部进程启动和客户端调用，都额外写出安全假设。

本章小结

阅读真实工程的目的不是逐行记忆，而是建立地图：事件从哪里来，状态在哪里改变，候选为何出现，数据写到哪里，发布物怎样获得信任。

术语索引

commit, iv

IMKInputController, 2

IMKServer, iv, 2

InputMethodKit, 2

KSPC, 95

preedit, iv, 14

raw buffer, iv, 14

召回, 56

排序, 56

结束语：把每一次按键变成证据

输入法工程最迷人的地方，是抽象算法与具体系统行为在毫秒尺度相遇。一个候选分数可能来自百万级词库和图搜索，一个按键却仍可能因为事件路径不同而没有进入控制器。优秀的输入法开发者既能推导评分函数，也愿意在第二台机器的 Notes 里逐字验证。

MyIME 不是商业输入法能力的终点，却是很好的学习起点。读者不必照抄所有实现。真正需要带走的是一套方法：分离核心与系统、用最小模型解释状态、用可重复测试约束算法、用跨机器验收约束发布，并始终区分“设计意图”和“实测证据”。

本章小结

如果你已经能够独立回答“这个按键在哪里被消费”“这个候选为什么排第一”“这个用户数据写到哪里”“这个 DMG 为什么在另一台机器可用”，那么你已经不只是完成了一个输入法，而是完成了一次小型系统工程训练。

版本与勘误记录

本教材首版对应 MyIME 1.0.10。建议课程维护者在项目升级时记录：源码提交号、教材章节、受影响的代码清单范围、行为变化和重新验证结果。不要直接覆盖旧结论；教学材料需要保留“为什么过去这样写、后来为什么改变”的工程历史。

问题和勘误可通过项目仓库提交。报告应至少包含 macOS 版本、目标应用、输入序列、预期结果、实际结果和最小复现步骤。